

VECTOR

APL News and Reviews:

• KX London Users Meeting	30
• Forests in Finland	36
• Ian Clark reviews Mathematica	60
• Objects in Dyalog APL (part 3)	79
• Reiter on Apollonian Circles	106
• Sylvia Camacho on Chaos	117



*The Journal of the
British APL Association*

ISSN 0955-1433

www.vector.org.uk

Vol.22 No.4 December 2006

A specialist Group of the British Computer Society

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file or an HTML document. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2 or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from the website), and MS Word (any version, ideally using the Vector template which is also available from the website).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2006-2007

Category	Fee	Vectors
UK Private	£20	1
Overseas Private	£22	1
(Supplement for Airmail, not needed for Europe)	£4	
UK Corporate Membership	£100	5
Overseas Corporate	£110	5
Sustaining	£500	10
Non-voting Member (Student, OAP, unemployed)	£10	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: gill@apl385.com.

Contents

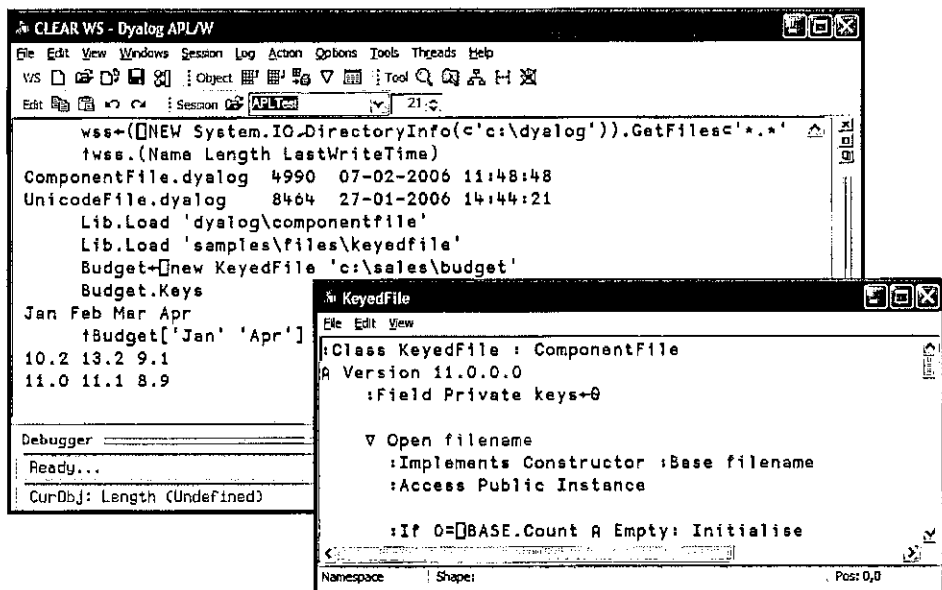
	Page
Editorial: Profiting from Iverson's work	3
RESPONSES	
APL Starmap	John R. Thorstensen 7
Memories of Ken Iverson and APL	Paul Barnetson 7
Kenecdotes	Sam Sexton 10
A new Web Server for APLers	Woody Butler 10
NEWS	
News from Sustaining Members	12
Product Guide	Gill Smith 18
KX in London, February 2006	
Meeting report	Adrian Smith 30
Forests in Finland, March 2006	
Meeting Report	Adrian Smith 36
APL and Bloomberg Data Control	Antti Korpela 46
Meeting of NYC JUG 20060314	
Meeting Report	Devon McCormick 50
DISCOVER	
APL/J/K are Top of the League	Graham Parkhouse 57
Review: Mathematica 5.2	Ian Clark 60
LEARN	
Objects for APLers (part 3 of 3)	Morten Kromberg 79
J-ottings 48: Perming and Combing	Norman Thomson 99
PROFIT	
Integer Apollonian Circle Packings in J	Cliff Reiter 106
The Supposed Descent into Chaos of the Logistic Equation	Sylvia Camacho 117
Complete Index to Volumes 13-22	129
Index to Advertisers	143

DYALOG

The tool of thought for expert programming

Dyalog Version 11.0 is the first commercially available APL system to provide a complete Object Oriented framework for APL. Object Orientation complements the functional and array tools already provided by APL. With version 11.0, you can

- ❑ Provide APL components to developers using other languages with the same ease as saving a workspace
- ❑ Use COM and Microsoft.Net tools as easily – and using the same syntax – as components written in APL



Since 1983, Dyalog Ltd. has developed a host of powerful language features and an unparalleled ability to interface with other software. This makes Dyalog the most powerful APL-based application development system available in the market today.

In addition to Object Orientation, Version 11.0 adds numerous enhancements to the APL language. 64-bit versions remove workspace limitations under Windows XP/64 and 64-bit Unix systems.

Dyalog APL is available for Windows from 98 to XP/64 and PocketPC, and for Linux and Unix.

Editorial: Profiting from Iverson's work

by Stephen Taylor

In our last issue we celebrated the life of Ken Iverson, without whom it seems unlikely that we should have anything resembling the APLs. Those of us who are still making a living with his tools, and the larger number who have done so, have a lot to thank him for.

We profit from his work in more ways than are marshalled into our 'Profit' section.

We have news this issue from *Kx Systems*, whose K language exploits Iverson's arrays-and-operators paradigm to provide short, fast analytical programs to performance- and data-hungry clients. In Finland the FinnAPL consultants continue to punch above their weight because APL spares their brains the unprofitable scut-work of writing loops. We have the news from their annual forest meeting earlier this year. And in New York the J User Group reports its proceedings from the Heartland Brewery.

At Project Euler (<http://mathschallenge.net>) members compete to solve mathematical programming problems. Competitors need both maths and programming savvy to succeed. There is a widely-held misapprehension that the APLs are not just good for mathematical applications, but good for little else. The results Graham Parkhouse reports from Project Euler do nothing to dispel this: the website describes APL/J/K as "the language of choice for the highest-scoring users". You might not want to compete, but you can read his report and bask in the glory reflected from such illustrious company.

It was always Ken Iverson's intention that maths teachers and their students should profit from his languages. Many have. Others have had the opportunity to use the *Mathematica* software, which draws on some but not all of his work. Some of our members say the overlap is strong enough that we should account *Mathematica* one of the APLs. In this issue Ian Clark reviews *Mathematica* 5.2. I invite your views: is *Mathematica* an APL we should report on here?

One of the more exciting developments this year is the release of *Dyalog APL 11* with its support for class-instance inheritance, and we carry here the third and last of Morten Kromberg's articles introducing OO to APLers. Objects are the dominant paradigm outside the APLs, and a common interface to operating-system resources. They also offer new ways to encapsulate and organise code. The blending of array and object operations opens new possibilities. In part we are

guided by the experience the OO programmers have accumulated. In part we are breaking new ground and must find our own way.

Stefano Lanzavecchia has already used Dyalog 11 to build a lightweight web server, from which he reports good results. And over the summer Gilgamesh Athoraya and I have rebuilt much of the core of a pensions application using objects and Morten's Simple APL Library Toolkit to keep code in Unicode text files. OO APL is changing the way we write applications.

And the way we write about them. Dyalog 11 ships with an IME (input method editor) that gives other Windows applications access to the APL keyboard mapping. I've spent the last week working on drafts of a new introduction to APL. Where once I was pleased to be able to cut and paste from the APL session into my documents, now I'm delighted to be able to work in my usual text editor, font set to Adrian Smith's *APL385 Unicode*, typing APL expressions direct into the XML documents. Gimme the simple life!

Our Profit section as usual shows people using the APLs to get results in their fields. Cliff Reiter explores the maths and the beauty of Apollonian packing solutions for circles. Sylvia Camacho tackles questions raised by Gérard Langlet in an earlier Vector concerning chaos theory and computational precision.

Agile systems development remains one of the most important ways in which people profit from Iverson's notation. With this in mind, I went this summer with Gitte Christensen and Morten Kromberg of Dyalog to XP2006 to talk to other programmers committed to agile development. There we found developers from other marginalised languages who recognised our stories of outstanding productivity and matched them with their own. We found common cause in our use of domain-specific notations constructed (or embedded) in the programming language, and are meeting in London this autumn to review the prospects for a separate track at XP2007, or a separate conference, or both.

We should have a report from this in the next issue. In the spirit of Christmas, 23:1 will be a bumper issue, with reports from both the Dyalog and APL2000 User conferences.

Quick Reference Diary

Date	Venue	Event
March 22-23, 2007	Finland	FinnAPL Forest Seminar
May 23-25, 2007	Ireland	Kx2007 International Users Conference at historic Dromoland Castle in Ireland

Kx will be holding its Kx2007 International Users Conference from May 23-25, 2007, at historic Dromoland Castle in the west of Ireland. This is their premier 2007 training and networking conference for the Kx user community.

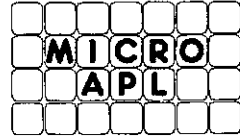
18-22 Jun 2007	Como, Italy	XP2007.
----------------	-------------	---------

The 8th International Conference on *Agile Processes in Software Engineering and eXtreme Programming* will be held in Como at the Palace Hotel conference centre, on June 18-22, 2007. This may well include APL-related material under a "Domain Specific Notations" track - see www.xp2007.org for more details as they appear.

Dates for Future Issues of VECTOR

	Vol.23	Vol.23	Vol.23
	No.1	No.2	No.3
Copy date	in press	in press	2nd March
Ad booking	in press	in press	9th March
Ad Copy	in press	in press	16th March
Distribution	January 2007	January 2007	May 2007

APLX *for*



Mac OS 9

Windows 95/98/ME

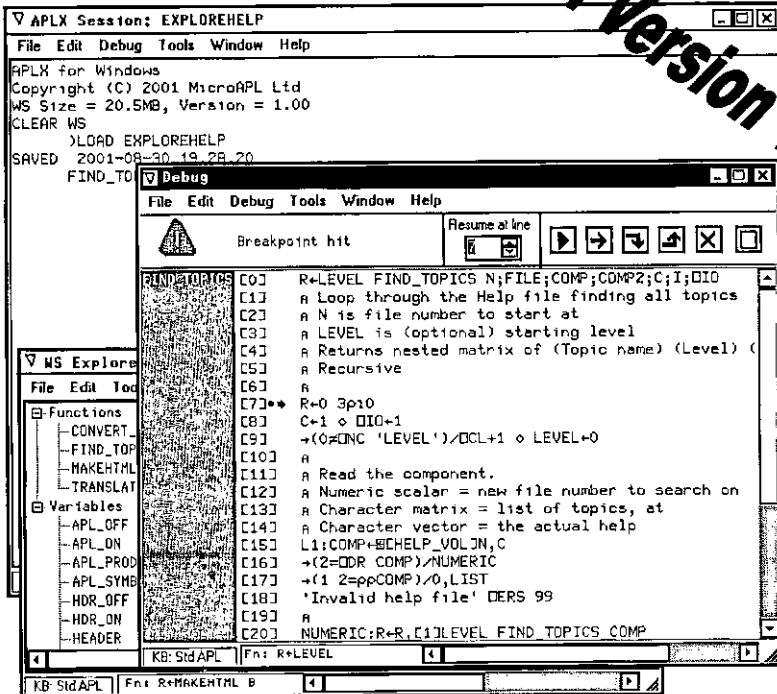
Linux

Mac OS X

Windows NT/2000/XP

AIX

New! Version 1.1



The cross-platform, full-featured, user-friendly,
GUI-enabled, cost-effective APL. *New! Version 1.1*
Check it out at <http://www.microapl.co.uk/apl>

APLX and MicroAPL are trademarks of MicroAPL Ltd. Other trademarks acknowledged.

RESPONSES

APL Starmap

From: John R. Thorstensen

It was a surprise and delight to find (in an idle moment of web browsing) that APL Starmap is still alive and kicking on the Web! I wrote that in the summer of 1973 between my 3rd and 4th years of College, when I happened to find lodging for a summer job in a spare bedroom at Paul and Sarita Berry's house in Swarthmore, Pennsylvania. Since then I've gone on to other things ...

<http://www.dartmouth.edu/~physics/faculty/thorstensen.html>

but I still enjoy writing code for the celestial sphere ...

<http://mdm.kpno.noao.edu/Manuals/doc/guimanual.htm>

John R. Thorstensen – Professor and Chair
Department of Physics and Astronomy
Dartmouth College
6127 Wilder Laboratory
Hanover, NH 03755

Memories of Ken Iverson and APL

From: Paul Barnettson (paul@barnetson.fsnet.co.uk)

30 September 2006

It was with great interest, and with many happy memories, that I read the Vol. 22, No 3 edition of Vector, with its theme of "*Remembering Kenneth Iverson*".

When I was young, and before I reached the "age of discretion", I used to irritate my friends and family no end by continually asking them what they meant by the words they used, and pinning them down to get precise definitions. It was to nobody's particular surprise that mathematics was my favourite subject at school and university, before I joined IBM to work with computers. And back in 1963, that was a burgeoning and very new field!

In 1968 I was sent on assignment to the USA for a couple of years, and while I was there I was introduced to APL, *"the language that forces the computer to think in the way that you do, rather than forcing you to think in the way the computer does, like other languages"*, as it was described to me.

I fell in love with it then, and on my return to the UK, set out to evangelise about it within IBM-UK. During the 70s I was IBM-UK's "APL-man", and some of your older readers might remember me in this job.

During this time Ken wore a beard, many of his US colleagues also did, the marketeer for APL in IBM-UK had one too, and as the technical man I also wore one. With vivid ginger hair at the time, I was the man with "the red one", and a hirsute appendage was expected of all male APL enthusiasts!

One of my contributions during this time was being secretary to the APL Standards Group, ably led at that time by Dave Ziemann; I think he eventually moved over to a language called Smalltalk. We used to spend hours arguing over whether a particular feature involved standardising the interpreter or the language – not an easy thing to decide – not to mention discussing whether we were standardising what had been done, or whether we were trying to lay down rules for the future.

In the early days, when APL was still only available on 2741 terminals, I would travel to various exhibitions and conferences to organise demonstrations of this marvellous new language. I was always accompanied by my carefully guarded box of APL golf-ball typewriter heads. This box was immediately labelled *"Paul's Balls"*, much to the amusement of my mainly female demonstrators – I really don't know why!

At another APL conference I was chairing a presentation (by Bob Bernecky I think), and a marketing company had given everybody a name badge with a number on. Prizes were being offered for delegates who could find somebody whose badge had the same number as their own. So I defined SET as the collection of all the badge numbers in the auditorium, and I invited everybody to re-organise themselves according to

SET←SET[⍋SET]

This way, they would find themselves sitting next to the person with the same number as themselves! If N was the number on my badge (I forget what it was now), I then wrote up

N∈SET

to find there was a match in the auditorium! We later collected our prize together.

I was lucky enough to meet Ken during this period, along with many of the contributors to this particular Vector. Adin Falkoff, Jim Brown, Gitte Christensen, Per Gjerløv, John Scholes, Morten Kromberg, Gérard Langlet, are all people who helped me along the APL way, not that there is any reason why they should remember me. I still correspond with Norman Thomson – he has tried to convert me to J! But you can't teach an old dog new tricks.

Now I have retired, I still use APL. One application is writing packages for charities; an example is *The Charlton Chase*, an overnight orienteering event for young people on the South Downs in the middle of winter. My package enables us to calculate when we could expect the teams at their next checkpoint, and so warn us if they are overdue. It also enables the young people to have the final results immediately the last team returns. I wrote an article on this package for Vector, a long time ago now.

As an enthusiast of recreational mathematics all my life, I have also used APL in innumerable problems concerning magic squares, prime numbers, and the like. I find these in New Scientist (Adrian – I also solved Enigma 1368, and in exactly the same way that you did!), and thanks to APL my name has appeared on three occasions as the lucky winner. I have also had similar problems published in Computer Weekly, which carries a "Puzzler" each week.

My only problem is finding a cheap single-user modern APL system; the ones on the market are expensive multi-user systems. Would you believe I'm still using a 1980s version of PC. APL, which somehow came home with me when I left IBM!

My regards to all my old APL friends and colleagues, and – like them – I can truthfully say that APL changed my life.

We'll miss you, Ken – you changed the world, and not many people do that.

Thank you for the chance to write this letter.

Paul Barnetson
22, Ferndale Rd
Chichester
W. Sussex PO19 6QJ

Kenecdotes...

From: Sam Sexton (Sam.Sexton@reuters.com)

11 October 2006

Although I've moved away almost completely from the world of APL at work, I still pick out the interesting (to me!) articles in Vector now and then and really enjoyed reading the tributes to Ken Iverson. If you have a follow-up of further memories, you may wish to add this:

I was at Ken's presentation in Helsinki in 1985 that Michel mentioned and I recall his "very critical marks" in response to questions about APL2 being put extremely politely along the lines of "well, these are the holes in it, it's illogical, but if that's what you want, it may be useful". However, the stinger was at the end, with his final words on the subject being "I'm just glad they didn't call it APL!". Warm applause and cheering ensued. It was an honour to meet him then and on my visits to IPSA in Toronto and I treasure my signed copy of A Programming Language - as it's red, perhaps it should be referred to as The Thoughts of Chairman Ken...?

Regards, Sam

Sam Sexton
Infrastructure Group
Transactions Group (Sales & Trading)

SAGE: A new Web Server for APLers

From: Woody Butler (woody@teambase.com)

1 October 2006

Over the past 3 years, Harold Ward from Netwings and I have developed a very robust Web Server engine called SAGE.

SAGE is written in APL2000 (runnnig on Windows) and is designed to interface with any language on the Windows server that supports standard COM methods.

Once an APL workspace is "registered" as a COM object on the server, SAGE can easily communicate with it... sending and receiving GET and POST strings from Internet users browsers and applications.

SAGE uses a back-end SQL SERVER DB for authentication and security control. So, by enrolling users in to tables (Members, Groups, and Web URL Resources)

you can easily protect specific URLs. SAGE URL Resources can also be given full Public Access, as well.. so user login is NOT required.

I use SAGE to call DyalogAPL through COM. My DAPL WS get's called upon a URL GET or POST to SAGE, and SAGE passes my [JLX autostart program a nested vector with parameter names and values. I run my desired back-end APL program and then return a simple HTML string back to SAGE through COM, and SAGE passes the HTML page back to the user.

We've been running SAGE in production for 2 years, and it is rock solid. We would be interested in working with other APLers who would like to write Web Applications using DAPL, APL2000 or any other "COM" supporting language that runs on MS Windows.

Please reply if you are interested, and we can discuss. Since we own a small ISP (GDEX.net), we have 100% control over the servers, network, firewall, DNS, database and development environment.

Woody Butler, President
TeamBase Corporation
GDEX Internet Services
APL Consulting & Hosting

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

**Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.**

Please note that Vol.1 No.2 is now out of stock.

Vector Back Numbers

Back numbers of Vector are available from:

British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

NEWS

News from Sustaining Members

MicroAPL Ltd

APLX for MacOS is now available as a native Intel binary, providing substantially higher performance for the new Intel-based range of Apple Macintosh systems. This has required a fairly large rework of the interpreter, and we would like to thank all our beta testers who have been helping us ensure the changeover is smooth. We strongly recommend any customers running APLX on an Intel-based Macintosh to upgrade to the new version. Not only is performance much better, but also Apple's Rosetta PowerPC emulation software has a number of bugs in its emulation of floating-point instructions, and these can affect results when running the PowerPC version of APLX on Intel systems.

Meanwhile, at the Array Languages Day hosted by Dyalog in Helsingør in October, Richard Nabavi of MicroAPL previewed some of the new features planned for APLX Version 4, which will be available next year. These include new object-oriented extensions to the APLX language, and the ability to use classes written in a variety of other languages including C# (and other .Net languages), Java, and Ruby.

Kx Systems

Kx Appoints Janet Lustgarten President

Janet Lustgarten, Chairman and former CEO, has been appointed to the role of President. Ms. Lustgarten, who will remain in the Palo Alto company headquarters, will lead the strategic direction of Kx, focusing on sales and customer relations.

The role of President was created to manage Kx through its next growth phase as the company's kdb+ database becomes increasingly deployed across the enterprise. Ms. Lustgarten, a co-founder of the company, said, "Use of the kdb+ platform is increasing in applications for financial services as well as other industries. We're in a high-growth time, and I am excited to be a part of this next phase."

Cantor Fitzgerald Makes Faster Market Decisions with Realtime Analysis Platform Built on Kx Systems

Kx Systems announced that Cantor Fitzgerald has completed a comprehensive system to handle historical data analysis and immediate querying of streaming market data based entirely on the Kx kdb+ database and "q" programming language. Cantor's traders use the solution to integrate their risk management and trading strategies and optimize short-term opportunities.

With this system, traders now track their portfolio risk and trades in realtime, in relation to other trades being made in the market, thus immediately gauging the momentum of price moves. This capability enables Cantor to make trading decisions instantly and get in and out of a market quickly - without compromising risk management.

"Everyone talks about realtime risk management, but with kdb+ we actually attain it," said Jacob Loveless, SVP at Cantor. "We've gone beyond simply knowing the risk to being able to act on it in the moment."

Cantor began working with Kx software in November 2005 and wrote a back-testing program in two days. Other capabilities were then added to build on the uniform kdb+ architecture for historical and real-time data. "The ability to integrate realtime publishing with historical searches made kdb+ the obvious choice for our new analysis system," said Loveless.

Because the entire system is written in the q language integrated with kdb+, there is zero latency between data capture and analysis. The unified platform architecture used by Kx eliminates the latency found where traditional applications connect to multiple external databases, and their various query languages, as well as to feed handlers and servers written in a variety of languages.

"Kx provides a unified platform that is unmatched for flexibility," added Loveless. "In this industry, our priorities for a programming language are simple - productivity and speed. Q delivers both, and it includes a realtime database that can keep up." Kdb+ eliminates the typical development scenario in which programmers must update several often incompatible databases and programs in order to develop, test, and release new code. For Cantor making changes to different parts of the system is fast and simple.

Cantor has also taken advantage of kdb+ support from Kx Systems' sales and services partner First Derivatives (FD), plc. "Support response using FD's email support system is fast," said Loveless. "Typically, I email a question, go get a cup of coffee, and have the response I need waiting for me when I get back to my desk."

"Cantor is the ideal example of how the world's most innovative investment firms are using Kx Systems products today," commented Simon Garland, Kx CTO. "It's no longer about building separate systems for tick data, historical queries, cross-asset data trading, and so on. It's about solving the entire puzzle at once, which is exactly what kdb+ was designed to do."

Developer Community Website

Kdb+ users now have access to a community code and information repository at <http://code.kx.com>. Code.kx contains an online reference manual, user-contributed code, a tutorial, cookbooks and more. Contributions and corrections/updates from the kdb+ community are welcome and encouraged. If you are a kdb+ developer and have not yet received your login, contact codeadmin@kx.com.

International Users Conference

Kx will be holding its Kx2007 International Users Conference from May 23-25, 2007, at historic Dromoland Castle in the west of Ireland. This is our premier 2007 training and networking conference for the Kx user community.

Attendance is by invitation only. To request an invitation, contact kx2007@kx.com. Visit www.kx.com/kx2007 for complete information on the conference—we hope that all users of Kx products will join us.

Dyalog Ltd

Thanks again to everyone who visited Dyalog'06, including our friends from APL2000, Kx Systems and MicroAPL, who joined us on the "Array Language Day" and showed how everyone is working hard to make Array Languages more competitive and available on a wide variety of platforms.

We hope that this tradition of running array language conferences in conjunction with one of the vendor conferences each year will be continued – and also that array language vendors will join us in making "APL and family" visible members of the Agile Community. Join us at XP2007 in June in picturesque Como, Italy!

Dyalog Version 11.0

It is more than a little embarrassing to look back see how many times we changed the predicted date for the availability of version 11.0, but it is now available! At the time this is printed in Vector, I expect 32- and 64-bit Windows to be generally available. Linux, Windows Mobile 5, and AIX versions should be close to release. Contact us regarding availability for other platforms.

If we say so ourselves, we believe it was worth waiting for! We accept that many of our existing users are very busy and will be slow to make use of many new features. However, the response to presentations at Dyalog'06 makes us believe that the object oriented features in Version 11.0, and the related support for writing Dyalog components which are stored in script form will revolutionize the way we do development, over the next decade or so.

If this sounds worrying, relax: You have our word that Object Oriented Dyalog will NEVER mean that users who enjoy functional array programming will be forced to create even a single class. And, even for those who do go for script-based development, it will be possible to develop and debug applications using the rapid iterative prototyping mode which we all demand.

The ability to integrate with industry standard development tools, have our source managed by popular source code management systems, embed code in web page scripts (etc), will also change the view that many non-APL developers have of us. If we produce the necessary examples and training materials, and make Dyalog easily available to anyone who wants to take a look, it should not take long for it to be accepted as a powerful scripting tool along with popular languages like Perl, Python and Ruby. We should be able to attract a large number of new users to take a look at what we have to offer, and we know that APL has a number of advantages over the other languages – at least for certain types of applications.

Low-cost Licenses

The free educational and low cost non-commercial licenses for Dyalog are now almost a year old, and we have been very pleased by the response. If only a fraction of the licenses shipped end up being used for real work, this initiative is going to lead to noticeable growth in our user community.

As we enter 2007, we expect the interest to increase significantly when we release version 11.0 for Linux under the same terms as the free and low-cost Windows licenses. We also plan to make it easier to request and subsequently download these licenses from our web page.

Training Courses

At Dyalog'06, we provided two full days of training in connection with the conference. Five different courses were provided; on the new object oriented functionality, on interfacing to Microsoft.Net, building web pages using APL, performing "office automation" tasks from APL, and on accessing databases from APL. We will be posting as much of this material as we can to our web page

during November, so that it is generally available. We are also considering re-running the courses in various locations across the globe if there is sufficient demand. Please contact Dyalog if your company or a local user group would like to arrange a course.

Causeway Graphical Systems Ltd

Firstly a recommendation – if you feel that you really need to get acquainted with this C# stuff but would much prefer to avoid Visual Studio and all that comes with it, search Amazon for “Gunnerson C#” and pick one of the early editions. I have the latest one (for .Net 2.0) and the very first one and I think for APLers learning this stuff for the first time, the early one is best.

Learning to work with .Net

It is an odd thing to say, but I reckon that writing APL with a view to compiling it results in better quality APL. Declaring argument types is good for you (and for those who come after you) and I suspect that ripping out ‘rank independence’ from utilities is actually a good thing. My old mainframe utility library spent far too much time conditioning arguments and far too little getting on with the job in hand for the common case.

The other thing I am finding is that writing with C# as a target makes me use all those helpful libraries ‘out there’, rather than staying with the old stuff. If I want to read a file, I get a *FileStream* and run its *ReadToEnd* method rather than calling my own code that wraps `□NREAD` with a few traps. This has the (obvious) advantage that the code just compiles (as long as it mentions `System.IO`) but also makes me long for the day when APL returns to its roots as a maths notation and leaves all this extraneous external stuff to the operating system where it belongs. At the Dyalog Users Meeting I showed a simple application with a Form, a *TrackBar*, and a *Chart* which was all written in standard CPro, but with no hint of a `□WC`. This has to be the way to go.

NewLeaf for .Net – progress report

As you know, rebuilding *RainPro* as *SharpPlot* took 2 of us around 18 months. We started on NewLeaf in February and made startlingly rapid progress, getting the basic typesetting running by the end of March. Marc then took on the RTF parser, and I took on Tables and my side of the project has been taking rather longer than I had hoped. *RainPro* was basically an exercise in tidying up, renaming a few variables, adding compilation tags and shipping. I took one long look at *leaf.table.Spread* and decided to ignore it completely and write a new one. Thanks

to a long history of users asking for features (and sometimes offering to pay for them) the current code does all sorts of things it was never really designed to do, and has got to the point where I really don't want to touch it ever again. So ...

Now that we have a well-tested *TextBlock* object (used for all the simple text flow) a *Cell* is just a wrapper around a *TextBlock* with some extra stuff like borders and vertical alignment. A *Table* is a matrix of *Cells* with some extra stuff like page-fitting rules. Once a *Cell* has been told how wide it can be, it can re-wrap its *TextBlock* and find out how deep it is. A row in a table is just a vector of *Cells*, and suddenly APL is really working with you. The little extension to Dyalog that allows you to type `[ /table row . depth` and find the deepest cell in the row is very, very nice! I really hope that the C# guys one day get the ability to work with arrays of objects, refer to element properties, and get back arrays of numbers. If we APLers can somehow get this idea back into the mainstream, we will have done the world a service.

The other interesting thing we have found out is that you can put 'traditional' DLL entry points into .Net DLLs. This means that we are building up a collection of `WCALL` definitions for APL+Win which make a simple (and very low-overhead) gateway from the old world to the new. As a very rough benchmark, the code to loop around *nlFlow* enough times to create 120 pages of PDF took approx 12 sec with NewLeaf, a little under 1 sec running the compiled code through `WCALL` and a little under 1 sec running the same test entirely in C#. Conclusion – it is possible to build a bridge from the old world to the new which has almost no impact on performance. Of course Dyalog already did it for us – the same test run direct from Dyalog 11 runs at exactly the same speed.

Where next

Finishing NewLeaf has to be the focus just now, but I am seriously beginning to wonder if I can rebuild *CPro* to run entirely off the .Net Gui controls. The *Grid* is the interesting one, and I am not even going to start reading about it until NewLeaf is well and truly out of the way. Most of the rest are easy.

Whether *CPro* (in its current form) can ever be compiled I rather doubt. It relies on being able to execute arbitrary APL expressions which are saved as data with each dialog definition. However .Net is moving towards XAML, where your dialogue definition is saved as XML data, and .Net 2.0 has partial classes (where the code for a class can be in several source files) both of which are big steps in the direction I want to go. I think the message for my *CPro* users is to hang in there and wait for the world to catch up.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.

Please help us keep it up to date!

All contributions and updates to the Vector Product Guide should be sent to:
Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385,
Email: apl385@compuserve.com

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADVOCORP Oy	APL+Win, APL+Link, APL+Linux, APL+Unix, APL*Plus Sharefile	poa	Complete APL2000 and Statgraphics(StatPoint) product range and links to various third party products.
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
APL Systems IDC SL	APL2000 APL interpreters and tools	poa	Distributor of APL2000 products for the UK Consulting and maintenance for APL applications
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyalog	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Download from IBM APL2 ftp site or contact APL Products.
	Workstation APL2 V2 Version 2	\$1500	AIX, Linux, Solaris, Windows Product 5724-B74
	APL2 Version 2	poa	Product No. 5688-228, Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229, Runtime environment for APL2 packages
Insight Systems	APL2000	poa	Leading distributor of APL2000 products in Denmark
	Dyalog Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
JSoftware Inc.	J on the Web online registration ...		
	J Professional (online reg.)	\$895	includes manual set and one year of updates
	J Standard (online reg.)	Free	Free for download only

Books and accessories			
	J Dictionary	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Fractals, Visualization and J	\$80	
	Concrete Math	\$40	
	Exploring Math	\$50	
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.
	APL+Unix	poa	
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APLW	poa	French distributor for Dyalog
MicroAPL	APLX for Windows/MacOS	499	Cross-platform APL development environment with GUI programming facilities. Interpreter modelled on APL2. Available for Windows 95/98/ME/NT/2000/XP, Mac OS 9 and Mac OS X.
	APLX Server Edition	poa	For running large multi-user APL applications on x86 Linux, RS/6000 AIX, and Windows NT/2000.
	APLX for Linux (commercial)	499	Linux desktop edition of APLX, compatible with Windows and MacOS versions, with full development environment and GUI programming facilities. Runs under RedHat, Debian, Mandrake and SuSe Linux distributions.
	APLX for Linux (personal)	Free	Full version of APLX, can be downloaded free for personal use.
	APL*PLUS	poa	Manugistics
	APL.68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Sollton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS products	poa	All APL*PLUS products including upgrades and educational.
	Dyalog and JSoftware products USA	poa	
	Dyalog and JSoftware products	poa	
ubJL GmbH (APL Software Team)	Dyalog APL and Dyalog Systems Ltd. Products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Acadvent	ARQUE	poa	Software for the quality assurance of university examinations.
APL Software/Services	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick_holt@email.com.
Assured Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)

Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in <i>NewLeaf</i> reports. Functionally compatible across Dyalog/W and APL+WIn.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+WIn.
	SharpPlot for Microsoft .Net	500	<i>RainPro</i> rebuilt as a fully-managed .Net DLL for use by APL and non-APL applications alike. May be an attractive alternative for Dyalog 11 and Santa Fe users, as it has much improved image support as well as running rather faster, and not cluttering your workspace with APL sourcecode.
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CoSy	K.CoSy	\$30 % yr sub	K.CoSy is a general purpose computing and programming environment constructed, all in open code, in Arthur Whitney's very high level, yet structurally transparent Array Programming Language, K, and its tightly coupled User Interface. K.CoSy is an extremely productive environment in one of the most powerful and fastest of APL's progeny, and therefore, likewise, of all languages. K.CoSy provides a workspace-like interactive development environment previously impossible in K. Because of its unique open construction within the language itself, this environment is clearly competitive in a large domain with the APLs from the other vendors. K.CoSy notepad nature, interactivity, and open K code vocabulary make learning Arthur Whitney's K far less daunting and far more productive than its raw console, or any external scripting method. If you are a client of Kx Systems, or are investigating the possibilities, Contact us. See CoSy/K.CoSy for more information.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/I users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
Dynamic Logistics Systems GmbH	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.

	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
	Olmec	poa	APL GUI environment, providing menu bar, tool bar, status area, navigation sidebar (with treeviews & listviews) and client area. All are configured by simple text files and require no programming. Client area has a "tab wizard" option to provide ordered transaction processing
	Nazca	poa	fast, flexible and reliable static database and editor.
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark
	<i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>		
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	QWIN	poa	Legacy DOS Windowing support for APL+Win
	ODBC/127	poa	IBM AP127-like ODBC Interface for APL+Win and Dyalog APL/W
Optima	ServiceLine	poa	A property management system which keeps a record of all outstanding tasks, produces an up to date list of work to-do and Scheduled reminders plus automated standard letters and basic financial control.
	TravelLine	poa	A system designed to control the workload allocation for a fleet of chauffeur vehicles plus a reminder system for fleet management and Local Authority requirements.
	BPA	poa	"Brand Performance Analysis" allows for the modelling of product/brand performance over time and comparison with competitor products.
	DBI	poa	"Database Interrogator" allows for the non technical user to ask sensible questions of a large database such as a questionnaire and obtain results tables and graphs quickly, easily and accurately.
Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL+PLUS PC & APL+PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.

APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADVOCORP Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to APL level interfacing to any system under Windows, TCP/IP network and database connectivity, APL based financial client/server applications, Cognos Planning and ReportNet consultancy.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development, Specialists since 1979 in Sharp APL, APL*Plus, APL+Win, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
Assured Systems	Consultancy	poa	APL+Win and/or J and/or VB custom systems development. Conversion of APL+DOS to APL+Win a speciality.
AUSCAN Software	Consultancy and Training	poa	APL software development, training
Ray Cannon	Consultancy	500/day +VAT	APL, C, C++, Assembler, Windows, Graphics on PC and IBM Mainframe. Experience in Insurance, Chemical, and Airline Industries
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
CoSy	Consultancy	poa	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modelling.
David Crossley	Consultancy	poa	Experienced in large APL system developments since 1969 for PC or mainframe.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyalog	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivatives plc	Consultancy	poa	Financial trading software in Q, K and kdb+
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APLW.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work. full members of DSDM consortium and Agile Alliance.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
INFOSTROY	Consultancy	poa, competitive	Broad experience in various APL platforms. Special skills and knowledge in developing complex applications for investment, financial and construction markets. Implementation of hybrid solutions based on APL, Delphi, C#, VBA, SQL servers.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP

APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.

KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Lambert Technology	Consultancy	poa	APL programming, consulting & training; web design and construction.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
Alastair Kinloch	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	poa	Business Forecasting & APL Systems.
Nussbaum gift	Consultancy	poa	IT Consultant with a strong focus on APL.
Optima	Consultancy, Training & Development	poa	We have been in business since 1990 and have a team of APL professionals with many years experience in pharmaceutical, industrial and financial systems on both PC and Mainframe platforms. In addition to pure APL we can also offer assistance with network integration, ACT! customisation and issues concerning the internet and/or web design.
Orbien Consulting Inc.		poa	Consultancy
Graeme Robertson	Consultancy & Training	250-500	Design and write custom software. Maintain and upgrade APL systems. Deliver customized APL training courses
Skelton Consulting	Consultancy	poa	K/Q Consultancy
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Rex Swain	Consultancy	poa	Independent consultant, 25 years experience. Custom software development, PC and/or mainframe.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.

	MACfns	MACfns has been released. A suite of over 100 high-accuracy, high-speed assembler code utility functions for APL+Win and APL+DOS. MACfns has been thoughtfully designed and implemented for APL programmers. It includes functions for segmented string manipulation, data restructuring, translation and datatype adjustment, and many numerical computations. Typical speedups range from 4-10 times, with reduced workspace usage, datatype conservation, identity detection, and greater numerical precision amongst the other benefits. Documentation of MACfns is extremely detailed and thorough.
ubJL GmbH (APL Software Team)	poa	Analyses, design and programming.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification. APL unicode fonts designed and built for all flavours of APL.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email acam@blueyonder.co.uk.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Right Seat Software	Vox Proxy	\$199 (comm) \$69.95 (edu)	Vox Proxy is authorware for PowerPoint(r) 2000 or 2002 which allows the use of Microsoft Agent Technology (3D talking animated characters) within slide shows. VP appears on PowerPoint's main menu and provides editing side-by-side with slides. Automated script-writing provides control of PowerPoint, allowing the use of characters for live presentations or fully-automated tutorials, demos, or training programs. Optional CD Prep program allows the user to create auto-starting CD's that will play on any version of PowerPoint with or without PowerPoint. Vox Proxy for educational use.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join Material fee 5,000yen a year
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	The APL Planet	Semi Annual meetings	\$13 (US), \$25 (non-US)
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
Toronto SIG	Toronto, Canada		Occasional meetings, APL Skills Database, Toronto Toolkit	

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
Acadvent Ltd.	Dr A M Sykes	171 Gower Road, Swansea SA2 9JH, UK. Tel: 01792-201776 Email: Alan.Sykes@which.net

ACM SigAPL		ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADVOCORP Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: adrian@apl385.com
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: curtis_jones@prodigy.net
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Consultants Japan	Kyosuke Saigusa	1507 shimomizo, sagamihara, kanagawa, Japan 2290015 Tel: +81-042-778-2127 Fax: +81-042-778-2205
APL Germany e.V.	Dr. Reiner Nussbaum	Buchenerstrasse 78, D-68259 Mannheim, Germany. Email: reiner@nussbaum-glft.de
APL Italiana S.p.A.	Stefano Lanzavecchia	via Monferrato 1, 20144 Milano, Italy. Email: stf@apl.it
APL Software\Services	Dick Holt	1570 Heather Field Lane, Earlysville VA 22936 USA. Tel: +1 434-973-4255 email: dick_holt@email.com
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: aplsi@starpower.net
Assured Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hlrlrj@melbpc.org.au
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@ca.inter.net
Beautiful Systems, Inc.	Jim Goff	PO Box 2235 Jenkintown, PA 19046, USA Tel: +1 (215) 635-0375; Fax: +1 (215) 635-9212 Email: info@beautifulsystems.com
Ray Cannon	Ray Cannon	21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
The Carlisle Group	Paul Mansour	544 Jefferson Avenue, Scranton PA, 18510 USA. Tel: +1 570-953-2036
Causeway Graphical Systems Ltd	Adrian Smith	Brook House, Gilling East, YORK, UK. Tel: 01439-788413 Email: adrian@causeway.co.uk
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolfi@cinerea.se
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33.4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Yrjökatu 11 D 17, 00120 Helsinki, FINLAND. Tel: +358 9 674 001 Email: sales@dinosoft.fi
Dittrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dyalog Ltd.	Morten Kromberg	Grove House, Lutyens Close, Chineham Court, Basingstoke, Hampshire RG24 8AG, UK. Tel: 01256-338461 Fax: 01256-316559 Email: sales@dyalog.com; support@dyalog.com
Dynamic Logistics Systems GmbH	Michael Baas	Wilhelm-Schöffer-Str. 29, 63571 Gelnhausen Germany. Tel: +49 6051 13067 Fax: +49 6051 16142. Email: info@dls-planning.com
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408) 782-6648 Fax: +1 (408) 782-6627 Email: info@DynArray.com
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mail.com
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivatives plc	Michael O'Neill	First Derivatives House, Kilmorey Business Park, Kilmorey Street, Newry, Co. Down, N. Ireland BT34 2DH. Tel. 028 3025 2242 Fax 028 3025 2060 Email: enquiries@firstderivatives.com
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com

General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 832463 E-mail: martin@gsoft.plus.com
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email:HMW@4xtra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.ltd.uk
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: acam@blueyonder.co.uk
IBM APL Products	Nancy Wheeler	APL Products, IBM Silicon Valley Lab, Dept H36/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOSTROY	Alexey Miroshnikov	11 Bolshaya Monetnaya Str., suite 6, St. Petersburg 197101 Russia. Tel: +7 812 325-9797 Fax: +7 812 233-9685 Email: aim@infostroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 25 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
Japan APL Assoc	Julchiro Takeuchi	Keio Univ. Faculty of Sci.&Tech. 3-14-1 Hiyoshi, Kohoku-ku Yokohama Kanagawa, Japan 223-8522. Tel: +81(045)563-1411 Fax: +81(045)566-1617 E-mail: takeuchi@ae.keio.ac.jp
JSoftware Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@jsoftware.com
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 55, 00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: kimmo.kekalainen@kjk-tieto.com
Kx Systems	Simon Garland [Europe]	555 Bryant St., No. 375, Palo Alto CA 94301 USA. Tel: +1 866 kdb fast email: info@kx.com (in Europe simon@kx.com)
Lambent Technology Ltd	Stephen Taylor	81 South Hill Park, London NW3 2SS, UK. Tel: +44(0)20 7813 3786. Email: stj@lambenttechnology.com
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.nfl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Alastair Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0) 7802 430202 Email: alastair.kinloch@btinternet.com
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Millinta Inc.	Dan Baronet	Contact Dan Baronet at dbaronet@millinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: apl@ellismorgan.co.uk
Dr. Nussbaum gift mbH	Dr Reiner Nussbaum	Buchenerstrasse 78, D-68259 Mannheim, Germany Tel: +49 621 7152190, Email: reiner@nussbaum-gift.de
Optima Systems Ltd	Paul Grosvenor	Optima House, Mill Court, Spindle Way, Crawley, West Sussex, RH10 1TT, UK. Tel: 01293 562700 Fax: 01293 562699 Email: paul@optima-systems.co.uk
Orbien Consulting Inc.	Ros Fovargue	33 Cherryhill Ave., Toronto, Ontario, Canada, M1C 1W9. email: rfovarg@interlog.com or Ros.Fovargue@sympatico.ca
Qualedi Inc.	Nicole Schless Georges Brigham	121 West Main Street, Milford, CT 06460, USA. Tel: +1 (203) 874-4334 Fax: +1 (203) 876-9083. Email: sales@qualedi.com; info@qualedi.com
Renaissance Data Sys	Ed Shaw	P.O. Box 511, Botsford, CT 06404, USA. Tel: +1 (203) 270-9729 Email: apibooks@earthlink.com
Right Seat Software, Inc.	Tom Atkins	1110 12th Street, Golden, CO 80401, USA. Tel: +1 303 278 2244. Fax: +1 303 278 6967. Email: info@voxproxy.com
Graeme Robertson	Dr. Graeme D. Robertson	15 Little Basing, Basingstoke, Hants RG24 8AX, UK. Tel: +44 (0) 1256-364071 Email: GraemeDR@aol.com nildram.co.uk
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldec@aol.com

Skelton Consulting GmbH	Charles Skelton	Weinlinge 9, 70329 Stuttgart, Germany. Email: c.skelton@skelton.de
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Email: bernecky@acm.org
Soliton Associates		Soliton Inc, 44 Victoria Street, Suite 2100, Toronto, ON M5C 1Y2 Tel: +1 (416) 364 9355 Email: sales@soliton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Russian Chapter of SIGAPL		P.O. Box 330, Excelsior, MN 55331 USA Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Strand Software Inc		8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
Rex Swain	Rex Swain	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1-818-347-5779 Email: roysykes@earthlink.net
Sykes Systems Inc	Roy Sykes Jr	P.O.Box 39, Mansfield, Ontario, L0N 1M0, Canada, email: info@torontoapl.ca
Toronto SIG	Richard Procter	
ubJL GmbH	Kai Jäger	Thusneldastr. 22, 90482 Nürnberg, Germany. Tel + 49 911 482512; Fax: +49 911 482518 Email jaeger@ubjl.de
(APL Software Team)		
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2 <ftp://software.ibm.com/ps/products/apl2>

WORLD WIDE WEB SITES

Acadvent Ltd.	www.acadvent.com/
ACM SigAPL	www.acm.org/sigapl/
ADVOCORP Oy	www.rett.fi/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.apl385.com
APL Forum	www.aplforum.com
APL Germany e.V.	www.apl-germany.de
APL Italiana S.p.A.	www.apl.it/
APL Journal, Germany	www.rhombos.de/shop/a/show/article/?216
APL Borealis Inc.	www.aplborealis.com
APL Consultants Japan	http://aplcons.com
APL to ASCII	see Waterloo Archive
Assured Systems	www.assuredsystems.com.au/
AUSCAN	http://home.ca.inter.net/rjp/auscan/
Eke van Batenburg	www.bio.leidenuniv.nl/~Batenburg/index.html
Carlisle Group	http://carlislegroup.com/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyalog Ltd	www.dyalog.com/
DynArray	www.dynarray.com/
Dynamic Logistics	www.dis-planung.de
Systems GmbH	www.dis-planning.com

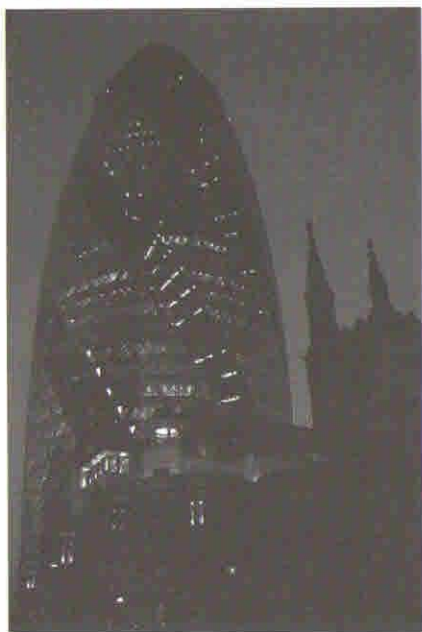
FinnAPL	www.pyr.fi/apl/
First Derivatives plc	www.firstderivatives.com
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/awdtools/apl
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.ae.keio.ac.jp/lab/soc/takeuchi/japla/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
kx systems	www.kx.com
Lambent Technology	www.lambenttechnology.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo-allegro.com/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Nussbaum gift	www.nussbaum-gift.de
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualedi.com
Renaissance Data	www.aplbooks.com/
Right Seat Software, Inc.	www.voxproxy.com
SigAPL	www.acm.org/sigapl/
Skelton Consulting GmbH	www.skelton.de
Snake Island Research Inc.	www.snakeisland.com
Soliton	www.sofiton.com/
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.ca/
ubJL GmbH (APL Software Team)	www.aplteam.de
Waterloo Archive	www.math.uwaterloo.ca/apl_archives/Welcome.html
Jim Weigang	www.chilton.com/~jimw/

The February 2006 KX Users' Meeting

reported by Adrian Smith (adrian@apl385.com)

Location

The meeting was back at the Merchant Taylors' Hall, convenient for the many city attendees, and the reception afterwards was at the very top of the Gherkin, which has to be the best view in London, particularly on a gloriously clear night. Thanks to Arthur and the Kx guys for inviting Vector to cover the meeting, and join the crowd (over 50 people attended) at the reception afterwards.



The Seminar Programme

I think I can honestly say that all the talks were interesting, and several would have been worth the trip on their own. I particularly enjoyed Niall Dalton on the future of Moore's Law and parallel processing, but then again I learned APL on a 370/148 with 'vector assist' so I can enjoy the sight of the wheel turning full circle!

Chris Purves (Barclays Capital) on Kdb+, Java and Program Trading

This was a nice example of a small team delivering fast results on a mixed environment. The BATS team is a couple of traders, a couple of IT guys, and "our tame stats professor" who advises on the details of the algorithms. Kdb is used to hold the data, and Java is used to write the front-end applications. Chris was asked "why Java?", and gave the very sensible reason that they knew Java very well, the bank was very comfortable with it, and there were some good tools freely available (like the 3D visualiser he showed later on). The basic requirement is to store, manage and analyse high-frequency market data. They started with 32-

bit systems, but are now running on AMD Opteron systems each with 32Gb of RAM and around 4Tb of disk space. The Kdb engine is used "out of the box" except for a few extra feeds, The Java server-side software has the trading algorithms, and the Java client software is used for monitoring.

The statistics were carefully written to be independent of tick frequency, so that no distinction is made between realtime and historical data. The output from the analyses can easily be 100 times more than the input, so this goes into Kdb too! The system then allows for flexible charting of back-tests, and the front-ends allow the users to create and save 'dashboards' to get a quick overview of comparative performance.

The Java engine streams information from Kdb, orders the events, and passes the data through various time-series transforms using 'grids' of parameters to search the strategy-space quickly. The OO capabilities of Java are well used here as there is a standard 'strategy' interface into which new strategies can be plugged really easily to test out ideas. In summary, Kdb is essential for the volume and speed, Java is good for rapid development and deployment. Any speed issues could be dropped down into q, but the trading algorithms are very iterative, so it is likely that the (compiled) Java code would be as fast as the (interpreted) q code here.

Jamie Grant (First Derivatives) on building a trading infrastructure

First Derivatives are the main support operation for Kdb and q developers in Europe. This talk outlined their work on taking many of the common components from the wide range of systems they support, and making these components available as a standard 'starter kit' for implementing a robust trading system. It will use a TCP/IP hub with a 'publish/subscribe' architecture to handle logging and alerts, and will provide a simple 'execution engine' to handle the actual mechanics of trading, and a matching 'risk engine' to validate requests before they are actioned. The Gui will be implementation-specific and will be the only part not written in q.

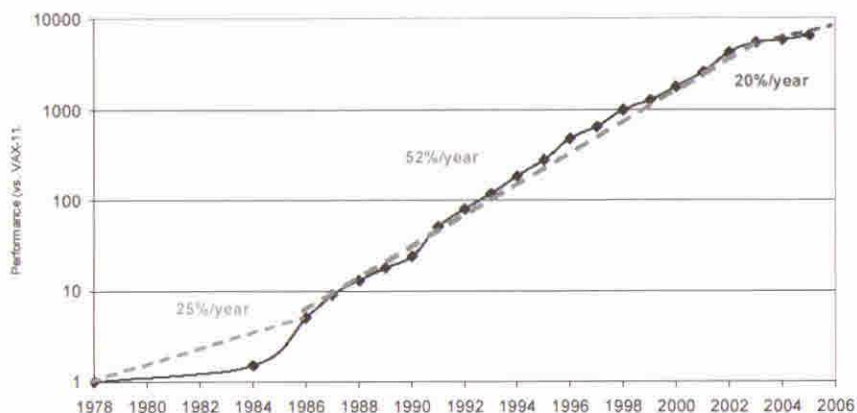
In answer to the question "How long?" Jamie replied that the design more or less fell out of an afternoon's discussion, and that the basics were working in a month. Then of course there was some cleanup to do.

Niall Dalton (niall@xrnd.com) on the death (or not) of Moore's Law

For the original set of slides, see <http://www.xrnd.com/kxtalk.pdf> where Niall has thoughtfully loaded up the PDF of his talk. He may well write it up in full for us in time for the printed edition of this Vector, but meanwhile here are my notes and quite a lot of the charts!

XRND sounds like a fascinating company to work for! They are a small group focussed on getting fast systems to run even faster, so Niall was in a very good position to comment on the impact of Intel's designs on calculation throughput over the years. He started by showing various 'quotes' of Moore's Law and asking the audience to identify what Moore actually said! To our credit, most of us picked "The number of transistors in an integrated circuit doubles every 24 months" - which is the correct answer and which is still holding rather well. The problem is "What are we doing with them?" which is a very different issue, and shows that processor throughput has very nearly levelled out.

UniProcessor Performance (SpecInt)

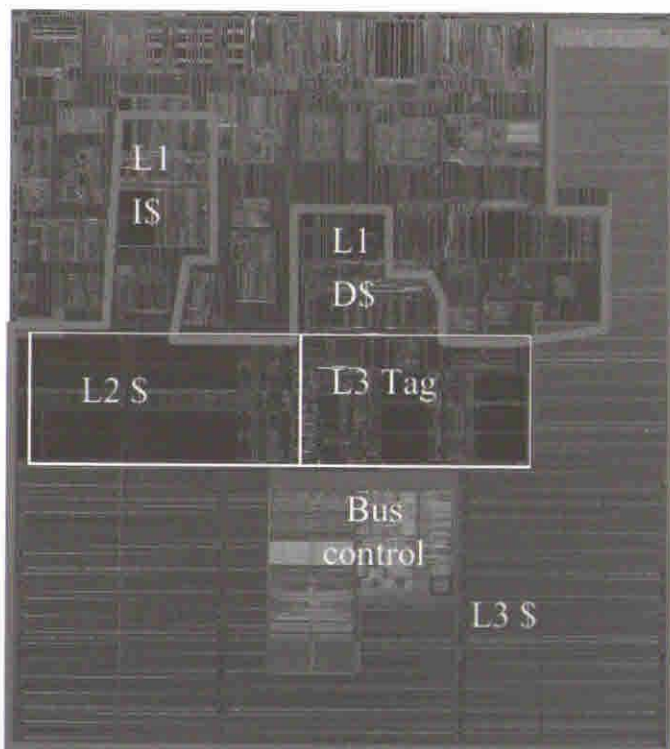


- VAX : 25%/year 1978 to 1986
- RISC + x86: 52%/year 1986 to 2002
- RISC + x86: 20%/year 2002 to present

Intel in particular have been working hard to give an illusion of parallelism by adding more and more tricks to the chip to try to keep the instruction pipeline full. Things like branch prediction, speculative execution, caching all serve to keep the processor running at full speed, but are incredibly complex, hard to maintain, and burned into the transistors. All this work uses power and generates *heat* whether or not the actual CPU is really busy. As Niall put it "The P4 is a very interesting processor, mainly for the things Intel got wrong". They were hoping for 10GHz but we have flatlined at 4GHz, and even that requires a lot of cooling. In fact we have nearly reached the limits of air-cooling, which means that something *has* to change in our thinking.

To rub in the point, he showed this picture of the Itanium II chip:

State of the art?



Of the 211M transistors, around 99% move and store data, only 1% actually *change* data! At 1GHz, this chip dissipates 130 Watts of power over a die size of 421mm^2 which is a lot of heat in a small space! Niall showed a lot more charts to reinforce the point that we really have reached the limits of scalar processing, and that multi-core chips are the only way ahead. Which is where languages like APL and K are suddenly very interesting. Here is a trivial algorithm that illustrates the problem with conventional (scalar) code:

```
int find (float*x,int n,float y){  
    int i;  
    for (i=0;i<n;i++) if(y==x[i])return i;  
    return -1;  
}
```

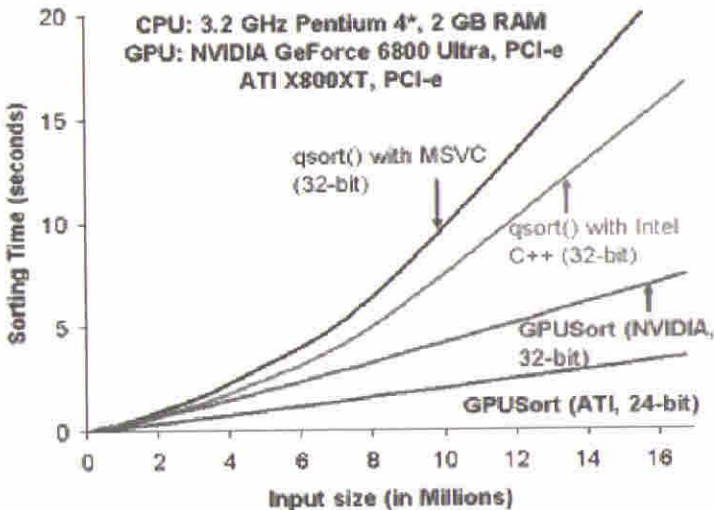
Now in q you could write something more like

```
find: {[x,y] first where x = y}
```

which a 'clever' compiler could spot as intrinsically parallel and farm out among the processors. With only 32 elements this is still nearly 3× faster with parallel execution. In the C code, it is almost impossible to tell what can be safely run in parallel - even an apparently obvious example like adding two arrays *could* have been coded with two pointers iterating down data that overlapped in memory. A bad piece of programming, maybe, but quite possible in a pointer-based language like C. Even 'good' C programming can be really awful when it comes to moving data around - chasing pointers through linked lists is probably the *worst* thing you can do these days! Garbage collectors also get badly in the way - a language that uses reference-counting and does its own cleanup will always win on this type of hardware.

Oddly, the really pioneering applications are the 3D shaders embedded in today's leading graphics chips. Even more strangely, people are learning to express simple algorithms like sorting and relational queries in the languages meant for painting triangles! The performance results can be astonishing:

Running a simple sort on graphics hardware



The next Playstation uses IBM's Cell hardware, which is 8× parallel, and can be 100× faster, but the launch date is going back and back. Maybe the reason is that

no-one has yet discovered the right tools for high-level programming in a truly parallel architecture? Apart from q, there are SmallTalk, R, Matlab, Mathematica and almost no others which are widely known. It looks as if q could have a big advantage, as we already know the hardware likes it, and the simplicity of the syntax means the compiler needs very little help to implement it in a genuinely parallel way.

"q is one of the best languages I can think of that maps from the problem domain to what the machine wants to do"

Which sums it up nicely!

Simon Garland joining the Dotz

This was a short tutorial on the utilities available in the z. namespace, mostly for simple socket programming.

Stephen Taylor on the danger of falling under the spell of IT

Stephen has seen the creeping menace of the IT professionals roll over many a productive APL shop, and (as he put it) the trading floor may be the last redoubt of the skunk works. His talk was born out of a lifetime of escaping 'formal methods' in favour of just sitting down with the users and getting on with whatever they see as important today. Hopefully he did at least make the audience aware that they were in possible danger, although my feeling is that this particular bunch of rabid programmers can fight off the IT steamroller more easily than most of us could. They have the power to make or lose millions in a day, and in the end that should be enough to keep their defences intact. Let's hope so, because we all want to see APL prospering, making money for its users, and sending programmers home with a spring in their step. From the atmosphere at this meeting, I would say that Kdb and q do this better than most.

The Reception

Was more than just a fabulous view, good snacks, and free drinks. It was also a good place to talk to the Java guys about data visualisation, the First Derivative guys about life in Ireland, and almost anyone about the fun of array programming. Interestingly, I got quite strong feedback that Stephen was warning of a danger many of them were aware of, but were confident they could defeat. Maybe if you are responsible for *that much* of your organisation's profit (or of course loss) you really can tell the IT crowd where to stuff their silly rulebook? I hope so, for the sake of the Kx business, and for APL programmers in general.

The 11th Annual Forest Seminar

reported by Adrian Smith (adrian@apl385.com)



Location

Another great location, but the weather was less kind to us than last year. The one small patch of clear weather which gave me time to run around with the camera was a brief relief from continuous dull light, with the temperature just far enough above freezing to leave everything dripping, or sliding off roofs.

The location was really good, just an hour from Helsinki and in a collection of cabins centred around a friendly hotel and small conference centre. Thanks again to our Finnish friends who made life easy by facing the rush-hour traffic to collect us on the first morning, and for dropping us back in town on Friday evening.

Numbers were a little down this year, with just over 20 attendees. I think this was mostly to do with the later date colliding with children's holidays - certainly several stalwarts had apologised in advance for not being there, and Richard was bribed with a glass of beer to do a shortened version of his Rowan presentation in the hotel in Helsinki on Wednesday night!

Selected Highlights

Many of the papers (especially on day-1) were in Finnish, so I can really only report a flavour of the subject matter, and a feel for what the audience response was. Hopefully we may see some of these coming around in Vector one day in translation.

Olli Paavola and Barcodes

Worth knowing about if you ever need to hook up an APL system to a barcode reader, or generate codes to print on output documents. The talk was more an illustration of the possibilities than a detailed technical 'how to' presentation, but useful to know that this can be done, and quite easily and reliably.

Veli-Matti Jantunen with some Sudoku solutions

This one really does need the author to write it up for us. I get the flavour of an algorithm that was sifting the 'entire' puzzle-space rather than approaching it serially as an exercise in logic. This also had a very nice Gui that showed the development of the solution as the algorithm gradually became certain about the contents of the cells.

Pertti Kalliojärvi on Dyalog 11



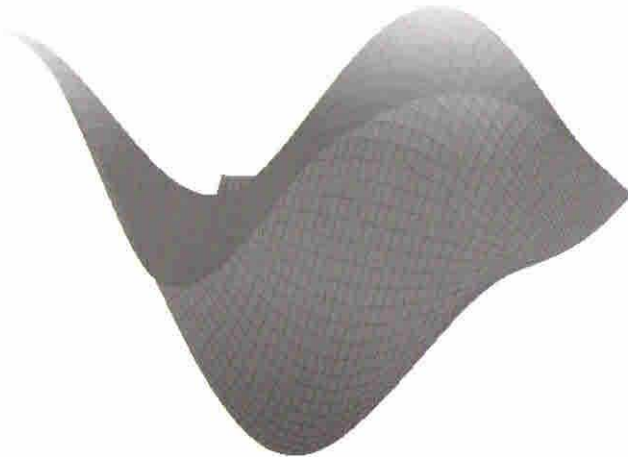
This was an excellent introduction to the new features in Dyalog 11, and the general principles of object-oriented design. Hard to know how much this will influence the current generation of APL developers - I still get the strong feeling of

"very interesting, but what has it got to do with me?" from the audiences at these gatherings. I also suspect that APL still doesn't go quite far enough to *really* get us using these new concepts. I find it quite hard to adapt to the C# world (which had this stuff from the beginning) and keep going back to the books to refresh my mind as to the differences between virtual functions and interfaces, or the nuances of private, internal and so on as access mechanisms. This stuff is quite a lot to get your brain around, and probably impossible to appreciate fully until you have built something real with it. Anyway, well done Pertti for trying!

Jouko Kangasniemi on data visualisation

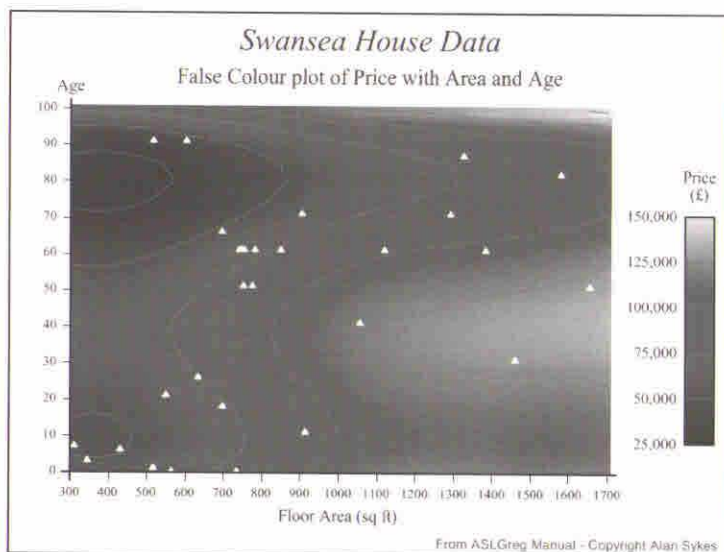
Lots of data series analysed with good quality, simple charts. One day I maybe should investigate the work Jouko has done on converting RainPro output to Flash (with the addition of some nice transitions and animations) but for now I am content to sit back and admire his work.

Adrian Smith on new toys in RainPro



Two new facilities in RainPro are the ability to use smooth curves instead of straight lines to join points (in the 3D charts and contour plots as well as in simple timeseries) and the addition of altitude colouring both to 3D charts (as shown in the picture) and - more importantly - in contour plots. This stuff is also available to Dyalog 11 users as SharpPlot, and it adds quite a few handy features (like transparency support in PNG files) for those who are willing to move to .Net graphics.

Please see the Vector website for COLOUR versions of the these examples!



This feature was requested by a Norwegian user who has sea-surface temperature data from a large number of sensors, and wanted to draw a simple map showing the patterns of variation. This is traditionally done with a set of 'false colours' which map relatively intuitively to the human conventions for 'hot' and 'cold'. However the tricks to fit the surface to the spot-values are exactly what I was already using to make 3D meshes for the cloud chart, so Rainpro needed relatively few new properties to extend this capability to flat contour plots. The colour for each tile is computed at a fairly coarse scale by draping a gaussian-weighted mesh over the x-y grid and using the averaged z-value to select from a vector of graduated colours. Then we assume that each of these rather coarse tiles is linear, and just break it up into 4 or 16 smaller tiles by simple interpolation.

The chart shows Alan Sykes' data from ASLGreg, as you may have never seen it before! Suddenly it is really clear that there is a region of expensive housing around 40 years old, with a floor area of 1500 square feet and above. I feel that this could be a really good tool for data-mining, and I look forward to seeing what Jouko and his colleagues can do with it in time for next year's seminar!

Richard Smith on Rowan (*Pihlaja* in Finnish) - a new APL for .Net

I think Richard chose a good name for his personal implementation of Iverson's Notation - the first piece in Sibelius' *Five Piano Pieces Op.75* is *Kun pihlaja kukkii* and the Rowan is a sacred tree in Finnish folk tales. The language has already been well described in Vector, but there were a couple of new ideas which are worth illustrating here.

Rowan now supports named arguments in addition to the traditional APL convention of *alpha* and *omega*, for example in the simple case, the two versions of *mean* are exactly equivalent:

```
mean+{(+/ω)÷ρω}
mean 3 4 5

4
mean+{data: (+/data)÷pdata}
mean 3 4 5

4
```

Note that the white space after the colon is essential here! For functions of one argument (or two) the syntax just adds a little readability, but for functions which are required to take a fixed argument list, it has the big benefit that the inevitable error is tripped on calling the function incorrectly, rather than somewhere deep inside it:

```
area+{ a b c: s+0.5×+/a b c; √×/s,s-a b c; }
area 3 4 5

6
area 3 4
R.LengthException: Argument list of area (3 items)
different length to passed args
```

The result of any Rowan function is the result of the last statement which had a value (which is a slightly different rule from A+ where it is always the result of the last statement). I could have written *mean* as:

```
mean+{data: (+/data)÷pdata; }
mean 3 4 5

4
```

... and it still works as I hoped. There are actually 2 statements here (the second one is empty) but Richard very sensibly lets me add the final semi-colon just in case I have my C# brain plugged up here!

Another nice idea in Rowan (which Dyalog could copy rather easily) came from reading a very early Iverson paper and realising that there was little point in

computing huge boolean arrays if all you want is the index set of the *true* values. What we have is a new operator called *where* which modifies any boolean-producing function to return the index values instead ...

```

      qq←112
(1 2 3 4 5 6 7 8 9 10 11 12)
      qq>17
(8 9 10 11 12)
      'Hello, world'=1'o'
(5 9)

```

I know Dyalog works around many of these cases with idiom recognition, but having the alternative clearly shown in the code has to be a better idea.

It was the ease of interfacing to .Net that really caught the audience's attention. Suppose you want to read a simple text file into your Rowan workspace - there is an excellent set of utilities already in the .Net framework to handle this sort of thing, so you simply use what is out there:

```

      #using "System.IO"
      sr ← #new $StreamReader "c:/temp/fluff.txt"
System.IO.StreamReader
      sr:ReadToEnd
Hello again file

```

Rowan is very similar to Dyalog-11 here if for *#new* you read *quad-new*, but Richard chose to use a second prefix character to mark a word as a .Net type. This makes it very clear to the interpreter that it should look 'elsewhere' for the *StreamReader*, rather than taking the Dyalog approach of trapping the *VALUE ERROR* on first encountering a new name, looking it up and adding it to the symbol table if it finds it. I think that the only 'reserved words' in Rowan are 'true' and 'false' which I can live with.

Having discovered how to read a data file, the next step would probably be to create a simple Gui front-end to allow a user to interact with it. Now that Rowan supports the C# concept of 'delegates' (see Vector 22.1 for the technical details) it is remarkably easy to drive the .Net Forms library. Here is a little demo that changes the form colour as you wave the mouse around:

```

#using "System.Drawing";
#using "System.Windows.Forms";

gui.f_onMouseOver ← { f msg: // Simple 2D colour-chooser
  frx←msg:X ÷ f:Width;
  fry←1-msg:Y ÷ f:Height;
  c←$Color:FromArgb((lfry×frx×255) (lfry×255-frx×255) 0);

```

```

    f.BackgroundColor=c;
  };

  gui.b_onClick ← { sender args: =sender:Text };

  gui.createForm ← { // Create a simple GUI testpiece
    f←#new $Form [];
    f.Text←"Rowan form";

    b←#new $Button [];
    b.Text←"Press me!";
    b.Click←#delegate $EventHandler b_onClick;

    b2←#new $Button [];
    b2.Text←"Hello, world";
    b2.Click←#delegate $EventHandler b_onClick;
    b2.Location←#new $Drawing.Point(b.Size.Width,0);

    f.Controls:Add b; f.Controls:Add b2;
    f; // Return ref to form
  };

```

Running the script defines the function to create the main form (and two buttons) and sets up some event handlers. A couple of notes on the script file are in order here - the `←` character (called left vane) is required to assign the functions globally into the workspace, as any ordinary assignment arrows are local to the script file (which is treated by Rowan as if it were a function). Note the use of monadic `=` in the handler for the buttons - this simply echos its argument to the session. The facing brackets `[]` are the Rowan way of specifying an empty array.

Named arguments really come into their own here, as the event-handlers are typically passed a reference to the object and a message of some kind. The buttons ignore the message and just echo their respective captions to the session. In the case of the colour-chooser, the handler needs the form `f` and the `X` and `Y` properties of the mouse-move msg to compute a new colour. Once you have run the script to create the handler functions and make the main form, you can:

```

ff←gui.createForm;
ff.Show;
ff.Opacity←0.6;
ff.MouseMove←#delegate $MouseEventHandler gui.f_onMouseOver

```

You really need to go to Richard's Rowan webpage at www.redcorona.com/rowan and download a copy to see the full effect - this example is included with various sample scripts for you to try. You will have to get used to writing `'` to invoke methods and refer to properties, but otherwise you should find that everything you read in the .Net documentation just "works out of the box" as there is really

nothing between what you type into the session and the .Net runtime engine, except a very thin layer of Rowan parser! To show just how thin a layer this is, try:

```
1234.45:ToString "#,##0.00"
1,234.45
```

Even a simple scalar value can run the standard *ToString* method, passing a picture formatter. The syntax has been slightly special-cased with *each* to have the methods extended to vectors:

```
qq←23 45 12345
(23 45 12345)
qq":ToString "#,##0.00"
(23.00 45.00 12,345.00)
+/qq":ToString " #,##0.00"
23.00 45.00 12,345.00
```

The last line should be a little bit of a puzzle, until you realise that C# overloads the + symbol to mean catenation of strings, so Rowan followed along with this slightly bizarre convention!

Extending the Rowan function set

Richard also showed just how easy it was for a C# programmer to extend the language, so that any unused symbol in the APL Unicode set could be allocated a new meaning. To illustrate this, he made a little example which used the ⌵ characters to read and write text files:

```
using System;
using System.IO;
using R;
using RedCorona.Rowan;

public class FilePlugin : IRowanPlugin {
    public void Load(IRowanEngine engine){
        engine.AddFunction("⌵", new VerbInfo(new MonadicFunction(ReadFile), null,
            null, null, 0, 0, 0, false, null));
        engine.AddFunction("⌶", new VerbInfo(null, null, new
            DyadicFunction(WriteFile), null, 0, 0, 0, false, null));
    }

    Symbol ReadFile(IEngine engine, Symbol fname, object data){
        StreamReader sr = new StreamReader((String)fname.Value);
        string s = sr.ReadToEnd();
        sr.Close();
        return new Symbol(s, ObjectType.String);
    }

    Symbol WriteFile(IEngine engine, Symbol textSym, Symbol fname,
        object data){
```

```
string text = (String)textSym.Value;  
StreamWriter sw = new StreamWriter((String)fname.Value);  
sw.Write(text);  
sw.Close();  
return new Symbol(text.Length);  
}  
}
```

There are plenty of handy symbols out there that no known APL ever used, so this would be a really good way to integrate one of the many excellent (usually free) maths libraries into Rowan, if you really needed a fast fourier transform, or the eigenvalues of a matrix as part of your interactive environment. This was definitely a worthwhile outing, as Richard & I came away with plenty of new ideas for the Rowan session, and a simple project management capability modelled on the J talk that followed on day-2 (and from discussions with Anssi Seppälä in the hotel on Wednesday evening). It's only a tiny download, so give it a go and tell us what you think of it!

Free time, Sauna, and Dinner

The lake was frozen so hard that there was no possibility of a post-sauna plunge, so we had to content ourselves with throwing snowballs at the trees. Shame!

Sami Laitinen APL and Excel (part II)

Sami works for Nordea, and uses APL to calculate mortality curves for estimating the future value of pension funds. He was driving this through the APL COM interface, but large organisations have a habit of making life hard for in-house developers and now he can no longer register COM servers as part of an installation. So he has fallen back on using a simple DyalogRT setup, writing the parameters to a text file, starting APL, and waiting for the results to compute before resuming processing in Excel. He also mentioned a nice little piece of work where he translates APL to VBA code, using cell references for variables and calling a library of VB functions to implement the APL primitives. I hope he and I may be able to co-operate a little here one day, as this is very like the work Causeway has been doing with C# translation.

Antti Korpela on reading the Bloomberg data streams in APL

As Antti said in his introduction, this was a very short paper because he just tried out the stuff and it worked! The surprising thing was that the data structures returned by the Bloomberg OCX were wonderfully suited to APL and could be used more or less directly as a nested vector of matrices. See page 46 for the full paper.

Esu Lippu on software testing with J



This was valuable stuff, partly for the J examples, but mostly for the concepts. This system allows 'before/after' comparisons of database tables (read via the J ODBC interface) which have been populated around the time of a major software update, and it then flags up unexpected changes in values. These can be picked out analytically with some rather simple J expressions, or visually with some nice 3D plots,

Typically there will be a raft of minor changes which are fine (after all, we have fixed the software) but the occasional major systematic change which is almost always an alert for a new bug.

Tomas Gustafsson goes boating (virtually)

All you need is a graphics card with a level-2 shader, a managed C++ .Net assembly that can compile HLSL (High-level Shader Language) and APL to do the physics of just how a boat bounces off your generated waves, and off you go!



This really is getting very good indeed, with a frame-rate better than 30fps complete with flying spray, reflections, and wonderfully realistic water. Like all games engines, DirectX cheats like mad to fake realistic effects, but mostly it works (except when Tomas decides to frighten the audience by driving the boat straight through a very solid-looking island!) An excellent round-off to a most enjoyable two days.

APL and Bloomberg Data Control

by Antti Korpela (*Antti.Korpela@nordea.fi*)

This is a short "getting started by examples" description on how to get Bloomberg data directly into APL arrays.

What is Bloomberg? The following quote is copy-pasted directly from the Bloomberg home page:

About Bloomberg

Bloomberg is the leading global provider of data, news and analytics. The BLOOMBERG TERMINAL and Bloomberg's media services provide real-time and archived financial and market data, pricing, trading, news and communications tools in a single, integrated package to corporations, news organizations, financial and legal professionals and individuals around the world.

(source: www.bloomberg.com)

Background

About two years ago I needed to do some investment portfolio analysis at work and I happened to have (among other things) Bloomberg and Dyalog APL in my workstation. Almost immediately I got the feeling that it would be practical if I could get the data directly into APL-arrays. After a quick look at the documentation I realised that there are plenty of examples on how to get the data into Excel. The obvious conclusion was that then it shouldn't be much more difficult to get the data into APL either. So I decided to try it out.

Requirements:

- Bloomberg Terminal
- Any APL supporting Windows OCX-Controls

Example Function to Connect the Bloomberg Data Control

Ok. Let's get started. The OCX is a graphical object. The following function takes an object name, position and size as arguments and returns a name for the created instance. If the data control does not exist, the result is empty character vector.

```

▽ bloom←(bb)LoadBloomberg arg;ob;p;s;v
[1]
[2]  A| Loads the Bloomberg Data Control
[3]  A| ob = parent object
[4]  A| p = position
[5]  A| s = size
[6]
[7]
[8]  A| bloom = name for the created instance
[9]  A| bloom = '' if Bloomberg not available
[10]
[11]  (ob p s)+arg
[12]
[13]
[14]  :If 2*~NC'bb'
[15]      bb←'#.bbdatacontrol'
[16]  :End
[17]
[18]  :Trap 11
[19]      bb ~WC'OCXClass' 'Bloomberg Data Control'
[20]      bloom←ob
[21]  :Else
[22]      bloom←''
[23]  :End
[24]
[25]  :If ''#bloom
[26]      bloom ~WC bb
[27]      bloom ~WS('Posn'p)('Size's)
[28]  :End
▽

```

The next step is to try it out:

```

'f' ~WC 'Form'('Size' 10 10)
    ''##.gBLOOMOB+LoadBloomberg ('f.bbexample')(0 0)(100 100)
1

```

If everything worked out, we'll get the GUI:



.... Now we should be ready to access the data.

Example Function to Load Historical Price Data

The following function loads daily price data for given securities. The result is a three dimensional data cube for which the dimensions are number of days, number of securities and two for date and price.

```

▽ rslt←bloom BloomHistory arg;tickers;fields;start;curr;end
[1]
[2]  A| bloom ≡ name for an instance of Bloomberg data control
[3]
[4]  (tickers fields start curr)←arg
[5]
[6]  :With bloom
[7]    ActivateRealtime←0
[8]    Periodicity←bbDaily
[9]    DisplayNonTradingDays←AllCalendar
[10]   NonTradingDayValue←PreviousDays
[11]   rslt←BLPGetHistoricalData2(tickers fields start curr)
[12] :End
▽

```

Time to test the function – for example Nokia's price from last 12 days in euro currency:

```

arg←('NOK1V FH Equity' 'PX_LAST' '20060331' 'EUR')
M←#.gBLOOMDB BloomHistory arg

pM
12 1 2

```

```

display -3 1 2:M

```

2006 4 9 0 0 0 0	17.02
2006 4 10 0 0 0 0	16.97
2006 4 11 0 0 0 0	16.81

The output is very convenient for an APL programmer – a simple cube of data, time in `YYMMDD` format and no garbage to filter away. And as we can see from the example function, Bloomberg provides many useful options on the data content.

Another Example – the Subscribe Function

In addition to price data, the Bloomberg database contains huge amounts of all kind of information. For example, the following small cover function might be useful:

```
▽ rs!t←bloom BloomSubscribe arg;tickers;fields
[1]
[2]  A| bloom ≡ name for an instance of Bloomberg data control
[3]
[4]  (tickers fields)←arg
[5]
[6]  :With bloom
[7]    rs!t←BLPSubscribe arg
[8]  :End
▽
```

Let's try that out as well:

```
mybonds←'RFGB 2.75 07/06 Govt' 'SPGB 3.0 07/07 Govt'
fields←'NAME' 'CPN' 'MATURITY'
display M-#.gBLOOMOB BloomSubscribe mybonds fields
```

FINNISH GOVERNMENT	2.75	07/04/2006
BONOS Y OBLIG DEL ESTADO	3	07/30/2007

Conclusion

Independent of what one is up to (descriptive information, prices, cash flows ...) it turns out that the output is always in a very practical shape from an array programmer's perspective. The interface is easy to use and provides another example of that it's quite often worthwhile examining what you already have on your workstation.

Meeting of NYC JUG 20060314

reported by Devon McCormick

We had a good, meandering meeting that somehow managed to touch on all the agenda items. I'll present them in the order of the agenda though that's not necessarily the order in which we touched on them.

For the "Beginner's regatta", I presented an update of a topic from last month's meeting: different ways of waiting for an external command to finish. As is often the case in these matters, there are routines in the standard J libraries superior to my hand-crafted solutions.

In this case, the functions in *task.ijs*, *fork* and *shell*, do a fine job of invoking a command and either returning immediately or waiting for it to finish. Using the "wait" command I've been invoking to test for any lag in waiting for a command to finish, one could enter

```
load 'task'
_1 fork 'cmd /c wait 5'
```

where the `_1` left argument tells *fork* to wait for the command to finish. The small disadvantage of invoking the command processor "cmd" (necessary because "wait" is an internal DOS command), is that it brings up a DOS window. I haven't figured out a way around this. For example tests, see *cmdWaitTestingFork.txt* on the web site.

In the "Show and Tell" section, we looked at a problem posed by Mike Day on the J Forum: find soln to $a^2 + b^2 + c^2 = (a - b)(b - c)(c - a)$. There were a few replies to this, after which Mike clarified that he was looking for solutions over integers.

```
NB. [Session started: 2006 2 22 18 27 38.253]
eqn0=: 3 : '+/*:y.' NB. Set up equations
eqn1=: 3 : '*2-/y.,{y.'
diffeqns=: 3 : '(eqn0 y.)-eqn1 y.' NB. and differencer.

NB. Pick wide range of random arguments...
(<./,>./)rndn=. (_1e4+?1e5$20001)%>?1e5$1e3
_9879 9658

(<./,>./)|diffs=. 3 diffeqns\rndn
0.010086928 5.5282426e10

close1s=. 10{./:|diffs NB. Pick 10 closest results as
```

```

seeds=. closeis(3[\rndn NB. starting points...
_0.63838384 0.37931034 _17.451852
_6.4783599 5.4468339 6.1741868
. . .
_0.10789474 0.10091743 0.3027933

diffeqns"1 seeds
0.010086928 0.011770021 _0.023991408 ... 0.096197183

NB. Start with 1st seed and look a little bit around it in 3-space:
$>,nn=. (0{seeds) +>./ <steps _1 1 11
+---+---+
|11|11|11|
+---+---+
NB. where
steps=: 3 : 0
NB.* steps: vector of numbers from num, to num, in numsteps steps.
'from to numsteps'=. y.
from+(to-from)*(numsteps-1)%-i.numsteps
)

NB. Make 11x11x11 neighborhood of seed (use odd number-->0 0 0 to
NB. include original point so we can do no worse).
$>,>,>./>./ nn
11 11 11 3

<./,|dd=. diffeqns$> >,>,>./ nn
0.006545809
NB. This is an improvement. Point on which it is based:
>ix=. <($dd)#:(,|dd)i.<./,|dd
5 5 0

NB. Value of the new point:
>ix{>,>,>./ nn
_0.63838384 0.37931034 _17.461852
diffeqns >ix{>,>,>./ nn NB. Check it...
0.0065475394

NB. Use this point as new centroid for tighter neighborhood:
nn=. (>ix{>,>,>./ nn) +>./ <steps _0.1 0.1 11
<./,|dd=. diffeqns$> >,>,>./ nn
0.00054704513
NB. Is further improvement...
>ix=. <($dd)#:(,|dd)i.<./,|dd
5 5 4
>ix{>,>,>./ nn NB. New point...
_0.63838384 0.37931034 _17.481852

NB. and so on until no significant improvement.

NB. Start with another seed...
nn=. (1{seeds) +>./ <steps _1 1 11
<./,|dd=. diffeqns$> >,>,>./ nn
0.011770021
diffeqns 1{seeds

```

```

0.011770021
NB. No improvement, so tighten the search neighborhood:
nn=. (1{seeds) +&.>/ <steps _0.1 0.1 13
<./,|dd=. diffeqns&> >, &.>/&.>/ nn
0.0029753954
>ix=. <($dd)#:(,|dd)i.<./,|dd
8 4 4
>ix(>, &.>/&.>/ nn
_6.4450266 5.4135006 6.1408534

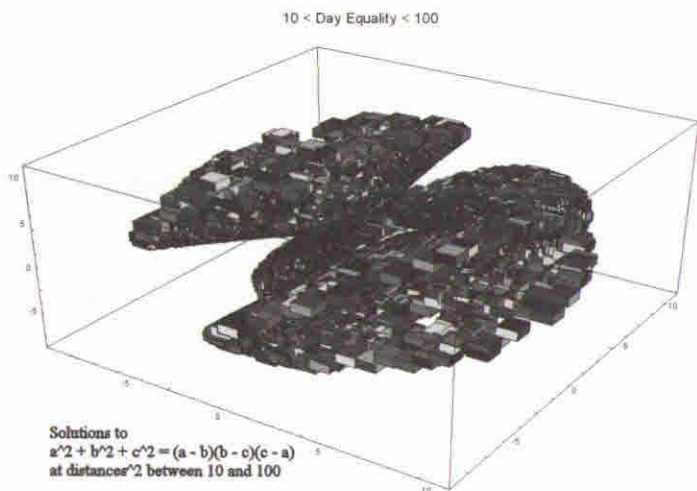
NB. Tighter around better point...
nn=. (>ix(>, &.>/&.>/ nn) +&.>/ <steps _0.001 0.001 13
<./,|dd=. diffeqns&> >, &.>/&.>/ nn
9.0611054e_5
>ix=. <($dd)#:(,|dd)i.<./,|dd
3 12 12
>ix(>, &.>/&.>/ nn
_6.4455266 5.4145006 6.1418534

NB. And so on...

```

However, I had a method for real numbers that I liked but became interested in the shape of the solution-space. To see this, I needed a way to plot 3-dimensional points.

Fortunately, Oleg Kobshenko provided a way to do this with his *plot3d.ijs* contribution to the forum. The plots are somewhat interesting and there are a number of useful lessons to be learned from different aspects of solving the system of equations, but I intend to write these up later.



Regular Expression Classification Table

We explored a technique I've been using with good success: table-driven file classification using regular expressions. This is a table consisting most essentially of a column of regular expressions and another column of executable expressions. Files sent to a central server during the day are processed by a routine that classifies files according to which regular expression they match, then running the corresponding expression on the files thus selected.

```

FTHDR=: 'DESC'; 'REGEX'; 'PGM'

NB.* FT: file-types: description, reg expression for name, program to run.
FT=: tablf2mat 0 : 0
VP funds monthly totals      ^COPY OF RPT MAINSTAY VP NAV \{2\}\.XLS
  (VPDIR, 'History')&moveFlsToSaveDateDir
VP daily positions          ^VP(CONS|MOD|MODGRO|GRO)[0-9]{6}\.CSV
  moveVPHldFlsToDateDir
VP Trade orders ^TRADE-VP(CONS|MOD|MODGRO|GRO)-[0-9]{6}\.CSV
  moveFlsToSaveDateDir
AAF positions ^AA FUNDS POSITION REPORT [0-9]{2}\.[0-9]{2}\.[0-9]{4}\.XLS
  movePosnsToDatedDir
VP Subscriptions/Redemptions ^SUBREDVP 20[0-9]{6}\.CSV
  moveVPSubRedToDatedDir
VP daily cash ^{57200|57210|57220|57230} [0-9]{1,2}-[0-9]{1,2}-[0-9]{2,4}
  CASH.XLS moveVPCashFilesToDatedDir
AAF daily cash ^{56580|56750|56760|56650} [0-9]{1,2}-[0-9]{1,2}-[0-9]{2,4}
  CASH.XLS moveCashFilesToDatedDir
IncMgr daily cash ^57524 CASH [0-9]{1,2}-[0-9]{1,2}-[0-9]{2,4}.XLS
  (IMFHDR;<IMF)&moveCashFilesToDatedDir
IncMgr Accounts ^{57522|57523|57525|57526|57527} CASH [0-9]{1,2}-[0-9]{1,2}-
  [0-9]{2,4}.XLS (IMFHDR;<IMF)&moveCashFilesToDatedDir
AAF daily positions ^{CONS|MOD|MODGRO|GRO}[0-9]{6}\.CSV
  moveHldFlsToDateDir
Trade order ^TRADE-(CONS|MOD|MODGRO|GRO)\.XLS moveFlsToSaveDateDir
PACE performance ^PERFORMANCE ANALYSIS - ASSET ALLOCATION FUNDS*
  moveFlsToSaveDateDir
Asset Manager collateral ^ASSET MANAGER [0-9]{2}-[0-9]{2}-[0-9]{2}.XLS
  moveFlsToNameDateDir
Asset Manager collateral update ^COLLATERAL UPDATED [0-9]{1,2}-[0-9]{1,2}-
  [0-9]{2}.*\..XLS moveFlsToNameDateDir
AAF trades ^AA FUNDS TRAN[S]?ACTION[S]? (REPORT)?[ \.]+[0-9]{2}\.[0-
  9]{2}\.[0-9]{4}\.XLS ('.';DODIR,'Old\')moveFlsToNameDateDir
Asset Manager daily cash ^{56525|56526|56527|56528} CASH [0-9]{2}-[0-
  9]{2}-[0-9]{2}.XLS moveFlsToNameDateDir
Recent trades in Income Manager ^INCOME MANAGER TRADE LOG\.XLS
  moveFlsToSaveDateDir
)

FTarg1_test_=: 'Copy of Rpt Mainstay VP NAV
(2).xls'; 'VPMoGro060222.csv'; 'TRADE-VPGro-060215.CSV'; 'AA FUNDS POSITION
REPORT 05.24.2006.XLS'; 'SUBREDVP 20060524.CSV'; '57210 05-24-2005
CASH.xls'; '56580 12-27-2005 CASH.xls'; 'cons051223.csv'; 'TRADE-
CONS.XLS'; 'Performance Analysis - Asset Allocation Funds - -

```

```
98173243919528.CSV'; 'Asset Manager 12-23-05.xls'; 'Collateral Updated 12-27-
05.xls'; 'AA FUNDS TRANSACTIONS.12.23.2005.xls'; '56525 cash 12-27-
05.xls'; 'Income Manager trade log.xls'; '57524 CASH 01-23-06.xls'; '57523 CASH
01-30-06.xls'; 'NotRecognized'; 'TRADE-CONSMOD.XLS'
```

```
FTtestComplete_test_=: 3 : 0
```

```
0 FTtestComplete y. NB. if 1=x., show details even if all tests
passed.
```

```
:
  coinsert 'base'
  if. 0=#y. do. y.=. FTarg1 end.
  y.=. toupper&y.
  reconfis=. _1-:({.@:)>>{1{"1 FT rxmatch&.>/y.
  sumrecon=. +/reconfis
  try. FTtest sumrecon;<reconfis
    if. x. do. FTshowTests sumrecon;reconfis;<y. end.
  catch. FTshowTests sumrecon;reconfis;<y.
  end.
)
```

```
FTtest_test_=: 3 : 0
```

```
'sumrecon reconfis'=. y.
NB. Unrecognized test strings: FTarg1_test_#--+. /reconfis
assert. sumrecon *./ . e. 1 0 NB. No more than 1 regex/match
assert. 1*./ . <:+/|:reconfis NB. All regexes tested once or more
assert. 0 1 *./ . e.~ sumrecon NB. No duplicate classification
1
)
```

```
FTshowTests_test_=: 3 : 0
```

```
'sumrecon reconfis args'=. y.

ntre=. 0=+/|:reconfis NB. Untested regexes
if. 0<pl=. +/ntre do. pl=. (1~:pl)#'es'
  smoutput 'Untested regex',pl,':'
  smoutput >(<' '),&.>ntre#1{"1 FT
end.

retm1=. 1<+/|:reconfis NB. Regexes tested more
than once
if. 0<pl=. +/retm1 do. pl=. (1~:pl)#'es'
  smoutput 'Regex',pl,' tested more than once:'
  smoutput >(<' '),&.>retm1#1{"1 FT
end.

dupcl=. -.0 1 e.~sumrecon NB. Duplicate classification
if. 0<nd=. +/dupcl do.
  smoutput 'Duplicate classification','(s'#~1~:nd),' ':'
  for_dix. I. dupcl do.
    smoutput FT#~dix{"1 reconfis
    smoutput ''
  end.
end.

if. 0<pl=. +/0=sumrecon do. pl=. (1~:pl)#'s' NB. Unclassified targets
```

```

        smoutput 'Unclassified target string',pl,':'
        smoutput >(<' '),&.>(0=sumrecon)#args
    end.
)

metaTest_test_=: 0 : 0
NB. Force duplicate classification:
FT=: FT,('Phony';(57524|57525) CASH [0-9]{1,2}-[0-9]{1,2}-[0-9]{2,4}.XLS';')
FT=: FT,('Phony2';(56525|56526) CASH [0-9]{2}-[0-9]{2}-[0-9]{2}.XLS';')
)

```

A most important restriction is that each file be classified by only one of the regular expressions; also, each expression should pick up a likely file. This quickly highlights the necessity of building the test suite alongside the code, a common practice in "Extreme Programming". However, as the test suite grows in complexity, it, too, needs to be tested. This raises the question of how we test the tests. For instance, we need to be sure we'll pick up a duplicate classification but we don't want to pollute our good table with deliberate errors. Dan had some good suggestions on dynamically modifying the table to accomplish this.

Learning/Teaching Array Thinking

We discussed a problem from the "TopCoder" website involving finding the longest prime-free intervals for a given range of positive integers.

You are given an int A and an int B. Return the number of far from primes numbers between A and B, inclusive. The number N is considered far from primes if there are no prime numbers between N-10 and N+10, inclusive, i.e., all numbers N-10, N-9, ..., N-1, N, N+1, ..., N+9, N+10 are not prime.

- A will be between 10 and 100000, inclusive.
- B will be between A and 100000, inclusive.
- (B - A) will be between 0 and 1000, inclusive.

Example: (3328,4100) returns 4

The far from primes numbers are 3480, 3750, 3978 and 4038.

See http://www.topcoder.com/stat?c=problem_statement&pm=6074&rd=9812 on the original website for a fuller explanation (You will have to register first).

```

qts''
2006 2 24 9 29 28.637

p: 10
31
p: i.10
2 3 5 7 11 13 17 19 23 29

```

```

prms=. p: i. 2000
p: 2000
17393
prms=. prms#~prms<:10000
$prms
1229
difs=. 2--/\prms
>./difs
36

prms=. prmsBtw 3328 4100
I. 20<difs=. 2--/\prms
18 53 80 88
0 1+/I. 20<difs=. 2--/\prms
18 53 80 88
19 54 81 89
prms{~0 1+/I. 20<difs=. 2--/\prms
3469 3739 3967 4027
3491 3761 3989 4049
-:/prms{~0 1+/I. 20<difs=. 2--/\prms
3480 3750 3978 4038
prms=. prmsBtw 19240 19710

+ /<:--/10 _10+"(0 1)prms{~0 1+/I. 23<difs=. 2--/\prms
53
ffp
3 : '+/_21+--/\prms{~0 1+/I. 22<2--/\prms=. crnumvec y.'
ffp=: 3 : '+/<:--/10 _10+"(0 1)prms{~0 1+/I. 22<2--/\prms=. crnumvec y.'
ffp&.>3328 4100;10 1000;19240 19710;23659 24065;97001 97691
+---+---+---+
|0|0|53|19|87|
+---+---+---+
ffp=: 3 : '+/<:--/10 _10+"(0 1)prms{~0 1+/I. 21<2--/\prms=. crnumvec y.'
ffp&.>3328 4100;10 1000;19240 19710;23659 24065;97001 97691
+---+---+---+
|4|0|53|20|89|
+---+---+---+

ffp=: 3 : '+/<:--/10 _10+"(0 1)prms{~0 1+/I. 21<2--/\prms=. crnumvec y.'

crnumvec=: 3 : '(<./y.),(>./y.),~prmsBtw (<./,>./)y.'

prmsBtw=: 3 : 0
y.=. (<./,>./)y.
pt=. p: nt=. >. 1.2*(i{y.}%^1{y.
while. pt<i{y. do. pt=. p: nt=. nt+100 end.
prms=. p: i. nt
prms=. prms#~(prms>:0{y.)*.prms<:1{y.
)

```

We compared the looping solutions presented with more array-oriented approaches but were stymied to come up with any sort of general scalar-to-array method. Much of array-oriented thinking comes from a different initial approach that might just require lots of practice.

DISCOVER

APL/J/K are Top of the League

by Graham Parkhouse

When I mentioned to a friend that I programmed in APL he informed me APL was currently top of the league of programming languages, as found on mathschallenge.net, under *Project Euler*. This is "a series of challenging mathematical/computer programming problems that require more than just mathematical insights to solve." The introductory message on the website continues:

"Although mathematics will help you arrive at elegant and efficient methods, the use of a computer and programming skills will be required to solve most problems. Please be warned that the problems are challenging and you are unlikely to make much progress if you have no knowledge of programming. Having said that, this is both the perfect environment and opportunity to learn new techniques and refine your programming skills."

The website is very user friendly and the problems, which range from pretty easy to very difficult, are all well conceived. Points are awarded for each problem solved according to the difficulty of the problem and the server will award you these points when you key in the correct answer. The typical answer to a problem is many digits long, so guessing is out of the question. And the server will block your entries for a while if you have too many tries at once. There are currently 126 problems, each worth between 2 and 20 points, which are being tackled by 1372 active users. Active in this context means those of the 3125 registered users who have solved a problem within the last 180 days. A problem or two are being added every few weeks.

As I write the highest possible score of 1915 points is held by 21 people. Roger Hui has 1895 points, so has a rating of 99%. He is 20 points short because he hasn't got around to solving the most recent problem. Obviously these statistics are all gradually changing with time. These figures are for the middle of August 2006. In August last year there were only 619 active users, showing a growth of more than 100% per year. There were then 102 problems.

To become a registered user you must provide a name, a password and an email address. There is no charge. You may also name your preferred programming language. Your options are:

	No of users	Ave rating		No of users	Ave rating
Pencil/Paper	52	6%	ECMAScript	5	59%
APL/J/K	29	46%	Ada	3	15%
Assembler	7	38%	PHP	24	11%
BASIC	52	19%	Maple	3	5%
C/C++	347	19%	LISP	34	17%
C#	52	17%	Spreadsheet	9	4%
Delphi	52	31%	Matlab	22	20%
Haskell	40	26%	Fortran	2	48%
Java	137	21%	Tcl	4	79%
Mathematica	28	34%	Magma	1	45%
Perl	56	24%	ML	3	35%
Python	184	30%	Smalltalk	3	29%
Ruby	62	15%	Erlang	2	40%

You must be very careful not to name APL/J/K as your preferred language when you first register, because your score at this moment will be zero and your rating will be 0%. Then, as soon as you solve a problem and become an active user you will have a detrimental effect on APL/J/K's statistics! You don't have to declare a preferred programming language: 159 of the 1372 active users have not declared a preference. If you ever reach the stage where your rating is above APL/J/K's average rating then you can edit your profile and declare your preference!

Comparatively few active users seem to have realised this. What is more, you can deregister your preference and join the 159 others of unknown preference at any time. Many active users, all those with less than the average rating for their

preferred language, could enhance this average rating by deregistering their preference.

You can see from the table that, of all the more popular languages, APL/J/K has the highest average rating. It is officially recognised by the website as "the language of choice for the highest scoring users." There is a rule that before a language can be considered for this honour it must have at least 1% representation, i.e. have at least 14 users. Those languages that do not qualify have been shaded. So, as the title of this article proclaims, APL/J/K are top of the league according to Project Euler. But is this a significant achievement? I think it is. High scorers in Project Euler need to be both intelligent and diligent. An intelligent person is not going to be diligent with blunt tools. There must be some pleasure in the programming if such a person is to spend so many hours without pecuniary reward.

I registered on 9 July 2005 and solved the ten easiest problems on the first day. 37 days later I had solved 77 problems and had a rating of 70%. I was working hard. Then I continued my normal life and now, a year later, I have solved 91 problems but have a rating of only 66%. High scoring is a challenge and anyone achieving it is spending valuable time programming. Had APL been half as good my score would have been much less than half as good.

Review: Mathematica 5.2

by Ian Clark

Introduction

When I sent away for the free *Mathematica* trial disk, version 5.2 hadn't yet been released. When it came out I upgraded my system. As a rank beginner addressing other rank beginners where Mathematica is concerned, this is hardly a review of 5.2 so much as a review of Mathematica itself. But Mathematica needs no promotion from me – its user base is huge and extensive, far beyond the provenance of Mathematics, pure and applied, as a typical university department understands it. This article therefore is aimed at jobbing APLers like myself whose exposure to Mathematica may be slight.

I first came across Mathematica at an educational exhibition nearly 15 years ago. I remember thinking then that insofar as it was like APL – it was good, insofar as it wasn't – it was bad. And I was averse to typing: `Plot[Sin[Exp[x]], {x, 0, 4}]` ... as any APLer might be who had been forced to use C/C++.

How blinkered I was! Nobody criticises Dreamweaver because HTML is so ugly to type. For the six months I've had Mathematica to play with, I've scarcely typed a line of Mathematica language like the above. For nowadays the Mathematica engine has a top end!

I ought to have known better, of course. In 1973 at the Peterlee scientific centre I came across Salah Mandil and his symbolic differentiator. This was built on top of a macro-processor, not unlike the C preprocessor. I was awed by the power of the technique. As a mathematician I should have been suspicious of the Babbage school of computing which saw a mathematical formula as the (unwieldy) specification for a series of instructions to be performed in sequence on a numeric accumulator. The symbolic differentiator never handled a mathematical variable as a binary quantity at all – it certainly never performed finite difference, in circumstances fraught with rounding errors, and pretended it was doing calculus. But I had been retreaded as a computer programmer and I'd somehow got it into my head that this was how it had to be done.

I quickly got the idea and tried writing a symbolic Fourier analyser. Artificial Intelligence was then all the rage, at least among those of us short on the natural variety, and the idea appealed to me of applying a communications engineering technique of looking for a signal-in-noise in a system which actually had symbols

for what it was looking for – in other words one that had “thoughts”. It never got very far, but I saw no barriers that couldn’t have been overcome if I’d had the time to spend on it that Salah had done on his differentiator. Glancing at the publications produced by the Mathematica team, judging by the titles alone it seems they’ve had the same idea, and I look forward to getting hold of Mathematica for keeps in order to wallow in their investigations.

Everything is an Expression

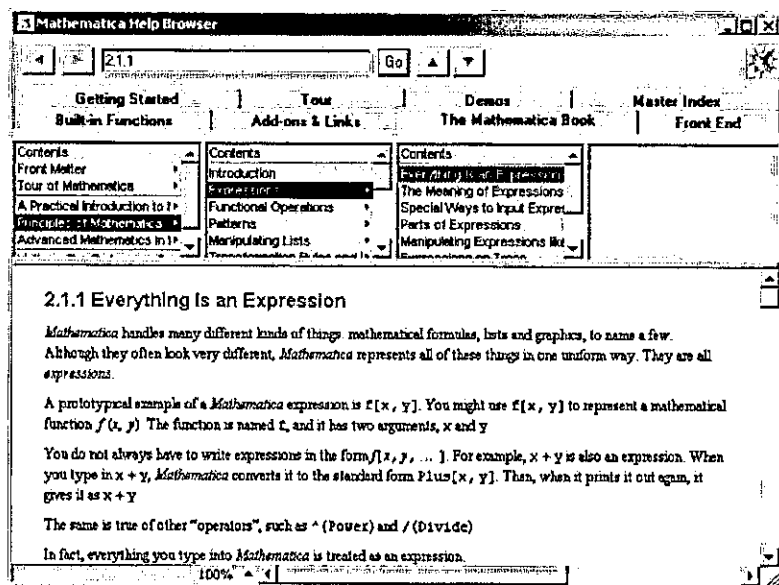


Figure 1: The Help Browser, showing “Everything is an expression”. Note the tabs, which list the chief sources of information to learn about Mathematica.

In Mathematica everything, but everything, is an “expression”. And an expression is a string of ASCII characters to be filtered (you can see the C/Unix world view coming through strongly here). An expression breaks down into function calls, like Sin[...] and Exp[...], and if at the end of the day you want the system to do something more than simply spit out a string of characters you have what used to be called “semantic” functions (although I haven’t seen the term used in the vendors literature) – functions like Plot[...] and Play[...], plus all the GUI furniture you’ll ever want. If you want the system to give you numbers, then at some stage a semantic function like N[...] must cut-in and start converting symbolic numerals

to binary. Although one of the chief features of release 5.2 is full support for 64 bit computing, I imagine the binary stage is put off as late as possible. Indeed if you compute 1023^{1023} or $1000!$, what comes out is an extremely long integer, suggesting to me that it's all done symbolically.

This contrasts strongly with the APL view of the world, where the whole time you're conscious of handling vectors of binary "num" elements.

It's refreshing to meet a product again with such a brutally logical backbone of design – something that was much sought-after in the Sixties and exemplified by systems like LISP and Prolog. Who nowadays would think to build a GUI on top of a string processing engine and aim to represent every meaningful GUI event as a semantic function processed by the engine? Certainly none of the Mac/Windows-bemused design teams I've worked with over the last 20 years, who thought it clever to chop your application up and stuff it into event handlers. But if you go the Mathematica route to software architecture you're no longer held hostage by your platform.

I recall many an experimental project which tried to encapsulate some classic APL interpreter being brought to its knees because the APL was built on the assumption that the user had a "break" key to press at any given moment. Or that the user was freely able to compose overstruck characters by multiple use of the forward and backspace key in any order which brought the golfball back over the same spot. This is what I mean by giving hostages to your platform. Having seen countless man-hours wasted in porting heritage systems across five generations of end-user interface, exemplified in APL by $\square$, $\square SM$, $\square SVO$, $\square WC/\square WI$ and $\square NC$ 9, forgive me for being edgy about the ill-patrolled border between application and user-interface.

Mathematica seems to have sidestepped all that. It shows. I got the chance to put its much vaunted portability to the test, shuttling work-in-progress between Windows and Mac. It was unbreakable. Microsoft please copy with Office. Although, like getting to Tipperary, if they really wanted to do that, they wouldn't start from where they are now.

The Notebook (.NB)

The fundamental file format of Mathematica is the Notebook (.NB). It is common to the whole range of Wolfram products: a notebook created with one product can be used with another. There is no space here to do more than briefly touch on a few. For the whole range see the Wolfram Research website [3].

Publicon [6], aimed at the problems of typesetting mathematical documents. More elaborate palettes like Figure 3, a library of templates for research papers, handouts and any form of document you can think of, import/export into a range of other formats. But Publicon does not evaluate expressions.

webMathematica [7], billed as a "one-stop solution for putting interesting science on-line". In passing let me note the existence of MathML, a proprietary HTML-extension standard to embed formulae into a web-page, which Wolfram products support.

If you drop a notebook onto MS Notepad you'll see that it is nothing but a text file in Mathematica language (and in printable ASCII only). On the principle that everything is an expression, a notebook consists basically of the expression: "Notebook[...]", plus a lot of comments. Some of those comments are cached data. Some are helpful instructions on how to edit the file by hand, including how to disable the cache (simply remove the comment: "(*CacheID: nnn*)") and let Mathematica regenerate it. Mathematica users are treated like grown-ups.

When you import an item of miscellaneous data into Mathematica, such as a JPEG or a WAV, it is cached inside the notebook and is available even with MathReader, without the original JPEG or WAV needing to be accessible. You can even append the notebook to an email (it is after all printable ASCII only) and send it to a colleague. The notebook itself shows the necessary instructions to strip it off the email and use it effectively.

Without the magic password, which is essentially what Wolfram Research sells to you over the Web, Mathematica operates in a crippled mode called MathReader. You can also download MathReader for free [1]. I recommend everybody to do so. With it you can view any notebook as it is meant to appear – this includes viewing the plots and playing the sounds. You thus have access to the whole knowledge-base [2], which for me was like a ticket to a first-class technical library. I was tempted to send myself back to university and learn the subject properly this time.

More importantly, for an APLer with time to tinker, if you write an APL workspace to construct a notebook, you get a free viewer with many of the features of a web browser, able to portray mathematical notation. Plus hypertext and a range of pretty notebook formats. Figure 2 shows my favourite notebook format (a pleasant pinky-brown), but this is a matter of taste. It is not however the one used in examples below (the default) which is black/white only.

MathReader allows you to change format according to a meagre repertoire, but the notebook formats provided with Mathematica proper are much more varied.

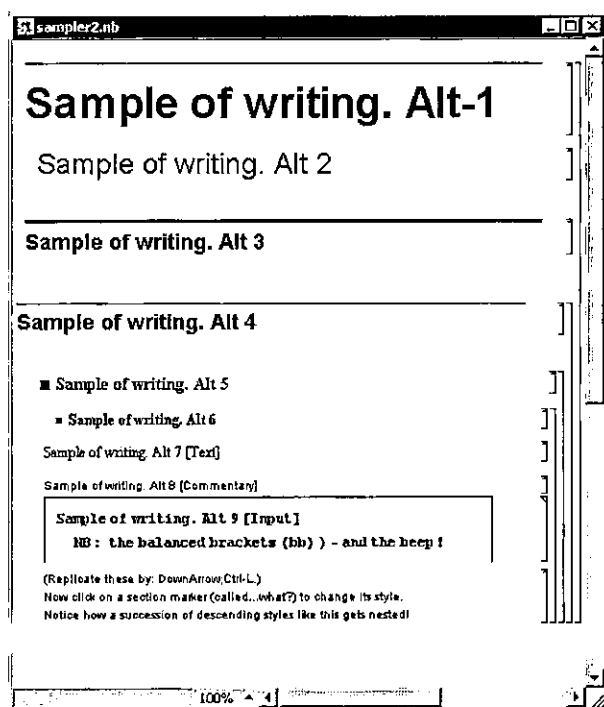


Figure 2: The various built-in paragraph styles, displayed for one of the many notebook formats. The template of a notebook can be changed with a key-click, even whilst composition is underway. The "Alt n" suffix shows which key-chord to hit to get the style in question.

The vendors seem to recognise that it is possible, indeed easy, to write your own notebook-processor and don't put blocks in your way. They reckon of course that they have already written tools to do whatever you may fancy doing. Suffice to say that I've abandoned my own little APL project to produce a mathematical display for pupils to tweak in order to do things like cross multiply, cancel out and collect terms.

All the instructional material, whether manuals or online help, comes in the form of notebooks. Nothing better demonstrates the vendors' confidence in their own product, its flexibility and suitability as a vehicle for expository material. Which APL vendor would have the chutzpah to ship all the online help, sample output and instruction manuals as workspaces (or session-log dumps)? An extensive

collection of notebooks come with the free CD or the downloadable product installer. See also [4].

Composing and Executing Expressions

You can write all your expressions in InputForm (as in Figure 10), which I suppose was the original way Mathematica was used. You can run the Mathematica engine, the "kernel", on its own and interact with it like APL. But the top-end gives you better tools, notably a collection of palettes, the chief one being Figure 3.

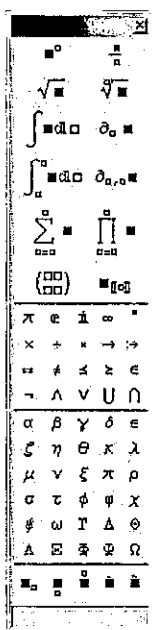


Figure 3: One of the palettes to help you compose classic mathematical expressions easily.

When you run Mathematica, the above palette appears, as does a windoid across the top of the screen consisting of nothing but a menu (Figure 4). When using the palette, the little squares ("holes" I suppose you could call them) in the symbolic construct you've just pasted-in become accessible using the tab-key. You can then employ the whole of the palette to insert a nested formula. Thus, for an expression with a complicated numerator and denominator you'd click the symbol on the top-right, then tab to each hole in turn and compose a separate expression in each.

You'll notice a matrix of four holes on the 6th row of symbols. You're not limited to 2x2 of course – further holes and rows can be added by pressing various key-combinations.

When the windoid of Figure 4 is showing, you can click through onto the desktop to open any notebook. Or you can do it the boring old way and open the File menu. Mathematica provides an indexing service to locate all the notebooks on your machine, assuming that you'll want to keep your lifetime's work in extensive libraries, but I didn't examine this. (But it seems something that APL vendors might usefully copy.)



Figure 4: The Mathematica topbar.

When I launched Mathematica for the very first time, I said to myself: "It's an APL...", so I promptly typed $1+1$ and pressed Enter. Nothing happened! Good start, I thought... but I was being crass. That works with the kernel, but the user metaphor for a notebook is a word-processor. You just get a new line.

What I should have done was press Shift+Enter! But by now I decided I didn't really know what I was doing, so instead of thrashing around I took the Ten Minute Tutorial – which Mathematica invites you to do in a hard-to-miss banner when it launches. I think this does its job splendidly. It's so effortless and simple-stupid I suspect they spent a lot of time and effort getting it right.

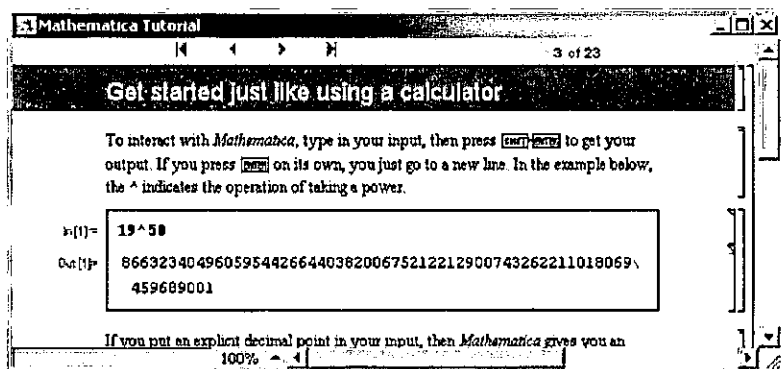


Figure 5: The super-efficient "Ten-Minute Tutorial": the best introduction to Mathematica for a rank beginner.

Note that this Tutorial is nothing special in the way of notebooks, but of a sort you can easily write yourself. From Figure 5 you'll see (rather better than with the all-blank default notebook format which you get on startup) that a notebook consists of a series of cells. Each cell can be a paragraph (or some other construct like a link or a bullet item), but the cell to focus on is a nested pair of input/output cells. By default you start creating an input cell. When you press Shift+Enter, it completes the input cell and calls the engine to create an output cell just below it to hold your answer: a number, an expression, or a plotted figure. You can range up and down the notebook inserting, deleting and cloning cells in any order, and you can re-execute existing cells – whereupon the output cell recomputes. It seems a lot more orderly and flexible than a typical APL session, where you need a lot of experience to mix active and passive text confidently, but it's no more cumbersome in use. Apart from remembering to hit Shift+Enter, you can build up a notebook as if it's APL.

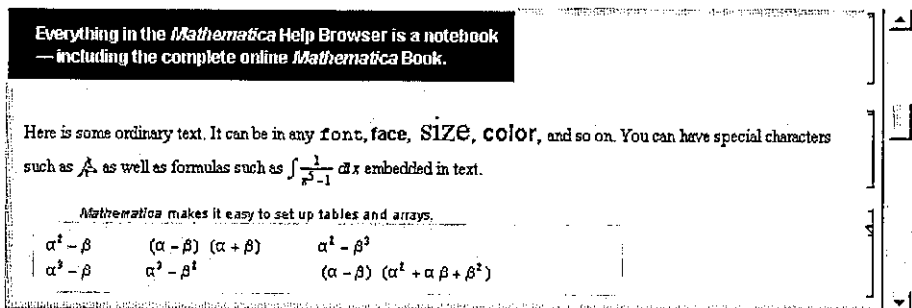


Figure 6: From the Help Browser: "Tour".

Let's play...

Mathematica is an APL in the modern sense of the word: an "array-processing language". It happily applies its mathematical operations to data items like: *z* whether or not *z* is a single number, a list of numbers or a matrix. Below we assign *z* as a list:

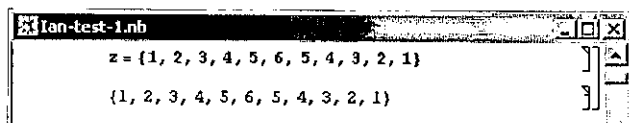


Figure 7: Inputting a list of numbers by hand.

This is a little bit artificial: a practical application will typically get its numbers from some data source like a database, a TXT, an XLS, a WAV, a BMP or a range of other formats.

Note the gubbins in the right hand side of the window. This delineates the scope of the input and output cells, and the cell which contains them both. Input and output cells often appear in different formats. The main formats used with a notebook (there are many others) are:

1. **TraditionalForm**: the one you see in Mathematics textbooks. It happens to be ambiguous (I didn't know that!) ~ so Mathematica offers you:
2. **StandardForm**: a two-dimensional notation resembling TraditionalForm, but robust, unambiguous and a bit clunky-looking. You can compose it directly using the palette, or with modified keystrokes (Ctrl+, Alt+, etc.) if you prefer not to mouse around. Mathematica uses it by default for output cells.
3. **InputForm**: what you can type-in as visible keystrokes, if you're so inclined.

There are quick ways of taking an output (or an input) cell and re-entering it as a consecutive input cell: a thing you're always wanting to do in a mathematical investigation, and you can switch a cell between forms by a simple keyclick. It's instructive to do so: it shows you how Mathematica handles a conventional-looking expression internally.

To begin with I wondered if Mathematica kept all expressions in StandardForm and simply let the top-end reformat the cell appropriately. However the following examples show this is not so: the displayable data is kept in a form appropriate to how it is displayed, the kernel being relied upon to do any required string conversion when you change a cell's format. Here is the text of a notebook I managed to whittle down to one (input) cell, expelling everything inessential:

```
Notebook[{
  Cell[BoxData[
    FormBox[
      SuperscriptBox[
        RowBox[{"(",
          RowBox[{SubscriptBox[StyleBox["x", "TI"], "2"], "-",
            OverscriptBox[StyleBox["x", "TI"], "_"]}], "2"],
          StandardForm]], "Input"]
    ]
  ]
}]
```

and here is how Mathematica/MathReader displays it:

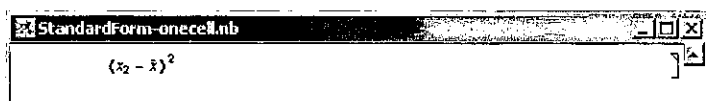


Figure 8: How Mathematica displays the one-cell hand-edited sample notebook: StandardForm-onecell.nb

If you edit StandardForm-onecell.nb to change the word StandardForm to TraditionalForm, this is what you get: a more graceful font worthy of the Enlightenment from which modern Mathematics sprang:



Figure 9: How Mathematica displays the one-cell hand-edited sample notebook: TraditionalForm-onecell.nb

Here is an example of InputForm, a linearised notation which shows up in bold in the notebook format we have selected:

$$(-b + \text{Sqrt}[b^2 - 4ac]) / (2a) /. \{b \rightarrow 0, c \rightarrow 1, a \rightarrow 2\}$$

Figure 10: An example of InputForm, as Mathematica displays it.
(The connector: "/" is to be read as: "such that".
You type-in the right-arrow as "hash/greater-than")

Note the way in which values are temporarily substituted for the "mathematician's" variables: a, b and c, viz. not by "assignment" (=) as was done for z above. The variables only take these values for the scope of the expression whilst it is being executed. Mathematica is more hard-line in its InputForm syntax than APL, demanding commas between numbers, plus curly brackets round the lot to denote a list (but it's amazing what can be listed). Its strong C/C++ accent betrays its background: "=" meaning assignment and "==" meaning "equals", the one concession to a programmer's view of notation rather than an algebraist's.

The difference is an important one. Variables a, b, c, here are "mathematician's" variables whereas z is like an APL global var – a "programmer's" variable. The former are placeholders which take the same value throughout an expression – whatever you, the mathematician, decide they should take. The latter isn't – it can take part in mathematically-absurd expressions like: z=z+1. It's really just a label for a storage location.

Mathematician's variables behave themselves when you save the notebook and reload it later. A programmer's variable like z doesn't – its value is not remembered between saves! Apart from this one silly little pitfall, you can reload a notebook and everything works – the plots re-plot, the sounds replay, the imported data is still there – it restores to the exact state in which you saved it. Better still: I moved my notebook from a Windows machine to a Mac – and not only did it look the same but everything worked the same – including the sounds! I was most impressed.

But not the "assigned" variables! They had to be re-evaluated by re-executing the last line which assigned them. This isn't just sloppiness, or an uncharacteristic oversight: it's an example of the rigorous logical thinking that informs the design of Mathematica throughout. You've got to imagine some nazi archangel up there saying: "you want a programmer's variable? – you get a programmer's variable! – and if you don't like it, try learning some Mathematics!"

Incidentally, I like the way Mathematica structures its error messages. A series of Alert boxes would be just plain maddening, so mostly you just hear a beep. At your leisure you pull down the Help menu and select "Why the beep?... Shift+Ctrl+H".

I must confess that as soon as I saw that menu item I selected it, hoping to get some idea how Mathematica structured its error information. I got the bald message: "Mathematica hasn't beeped yet!" I could feel that nazi archangel frowning down on me, shaking its head in disappointment. (...So why not grey-out the menu item?)

Now that we've assigned a value (a list of numbers) to z , let's do something with it:

```
z + 2 Pi
{1 + 2  $\pi$ , 2 + 2  $\pi$ , 3 + 2  $\pi$ , 4 + 2  $\pi$ , 5 + 2  $\pi$ ,
 6 + 2  $\pi$ , 5 + 2  $\pi$ , 4 + 2  $\pi$ , 3 + 2  $\pi$ , 2 + 2  $\pi$ , 1 + 2  $\pi$ }

N[%]
{7.28319, 8.28319, 9.28319, 10.2832, 11.2832,
 12.2832, 11.2832, 10.2832, 9.28319, 8.28319, 7.28319}
```

Figure 11: Adding a quantity to a list of quantities.

This illustrates nicely the fact that Mathematica processes an expression symbolically until you force it to substitute figures and evaluate an instance of the

expression. That's quite different from a programming language in the Fortran line-of-descent, which translates a formula into a sequence of operations on storage locations containing numerals. It's a powerful approach, which neatly sidesteps rounding errors in intermediate calculations. If you want decimal numerals, you apply the N[...] operator to the expression.

Note the use of "%" which stands for the expression you've last been working on. Or you can lift the whole expression for a new cell – let Mathematica do it, because OutputForm isn't necessarily appropriate to just copy/paste into an input cell. But Mathematica will make the adjustments.

Next I thought I'd play about with the roots of a quadratic:

```
(-b + Sqrt[b^2 - 4 a c]) / (2 a)

$$\frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

(-b + Sqrt[b^2 - 4 a c]) / (2 a) /. {b -> 0, c -> 1}

$$\frac{\sqrt{-a}}{a}$$

(-b + Sqrt[b^2 - 4 a c]) / (2 a) /. {b -> 0, c -> 1, a -> 2}

$$\frac{i}{\sqrt{2}}$$

Expand[(a b - c d)^2 + (a d + b c)^2]

$$a^2 b^2 + b^2 c^2 + a^2 d^2 + c^2 d^2$$

Simplify[a^2 b^2 + b^2 c^2 + a^2 d^2 + c^2 d^2]

$$(a^2 + c^2) (b^2 + d^2)$$

Factor[%]

$$(a^2 + c^2) (b^2 + d^2)$$

```

Figure 12: Investigating the roots of a quadratic equation.

Notice the powerful functions: Expand[...], Simplify[...] and Factor[...] (the last two giving the same answer here). Other powerful functions are Solve[...] and Reduce[...]:

```
Solve[x^2 + 2 x + 1 == 0]
{{x -> -1}, {x -> -1}}

Solve[x^5 - 1 == 0]
{{x -> 1}, {x -> -(-1)^(1/5)}, {x -> (-1)^(2/5)}, {x -> -(-1)^(3/5)}, {x -> (-1)^(4/5)}}
```

Figure 13: Playing with quadratic and quintic equations

I tentatively tried to get Mathematica to solve Fermat's Last Theorem, but I couldn't pin it down with all the assumptions it expected me to clarify. (I rather think it was flannelling...! :)

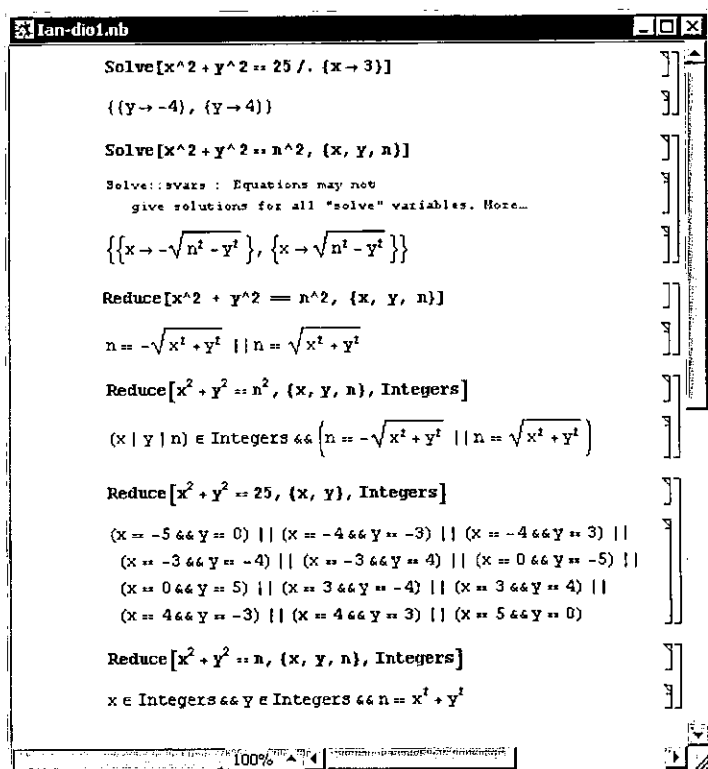


Figure 14: Playing with Diophantines

Growing bolder, I thought I'd use the palette (Figure 3) instead of the boring old keyboard. Unlike APL, Mathematica has a robust notion of infinity – several in fact. It happily sums series to infinity and integrates over an infinite range. As it ought to, of course. (Amazing what you can do with symbolic processing...)

```

∞ ∞
∞
∞^∞
ComplexInfinity

```

Figure 15: To Infinity – and beyond!

...Yes, that is infinity times infinity (which is infinity of course). You can multiply variables, numbers or expressions by putting a space between them. And the second input cell is indeed infinity to the infinite power.

Now for a bit of lightweight statistics, I said to myself:

```

x = {11, 12, 33, 24, 45}
{11, 12, 33, 24, 45}


$$\frac{\sum_{k=1}^n (x_k - \bar{x})^2}{n - 1}$$


Part::pspec: Part specification K$42 is neither an integer nor a list of integers. More...
Part::pspec: Part specification i is neither an integer nor a list of integers. More...
Part::pspec: Part specification K$49 is neither an integer nor a list of integers. More...
General::stop: Further output of Part::pspec will be suppressed during this calculation. More...


$$\frac{\sum_{i=1}^n (-25 + \{11, 12, 33, 24, 45\}[[i]])^2}{-1 + n}$$


```

Figure 16: Trying to talk statistics to Mathematica (without loading the requisite pack).

It wasn't clever to sum to n without saying what n was. So the kernel grouched at me. Notice that its grouches have clickable "More..." to explain its reasoning. But it still went on and did the best it could.

However one thing soon became apparent. I had defined x as a simple series, but that didn't mean that Mathematica automatically knew what x-overbar and x_i were. I had to tell it, which you do like this (Figure 17). The construct ":=" is a special case of a more general pattern-matching facility, enabling you to extend

notation – or even design your own. Mathematica comes with a collection of add-ons called “packs”. These, like notebooks, are written as text files having the file extension: “.M” instead of “.NB”, but like notebooks they consist entirely of Mathematica expressions. They are mostly “:=” lines, plus flag assignments.

Note that the first two lines of Figure 17 are not both needed. They simply show equivalent ways of specifying what $\bar{x}$ is to mean.

```
OverBar[x_] := Mean[x]
x̄ := Mean[x]
Mean[x]
25
Subscript[x_, i_] := x[[i]]
x[[2]]
12
x2
12
```

Figure 17: Extending the syntax which Mathematica recognises in a notebook.

Of course all these standard notational definitions are done for you if you load the statistics package.

Integration and differentiation are there, and very powerful they are too. I used the palette to input each expressions in StandardForm, then converted to InputForm to show how Mathematica handles them as standard operations.

```
Integrate[1/x, x]
Log[x]
D[Log[x], x]
1/x
```

Figure 18: Elementary integration and differentiation.

By now I was eager to see some functions plotted. Why start with something too easy? So I gave it the Topologist's Sine Wave – guaranteed to break most people's

naive postulates about functions, and indeed about continua, limits, and so on, with its bizarre behaviour in the neighbourhood of zero. What would Plot[...] make of it? – I wondered.

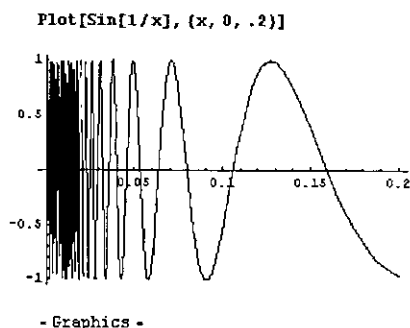


Figure 19: The Topologist's Sine Wave – blows the mind of a non-topologist.

...well, I had to say I was impressed. The pixels perished first – and it looks like Mathematica approached to the very brink of the black-hole at zero and didn't fall in.

Now for something three-dimensional. How about an egg-box?

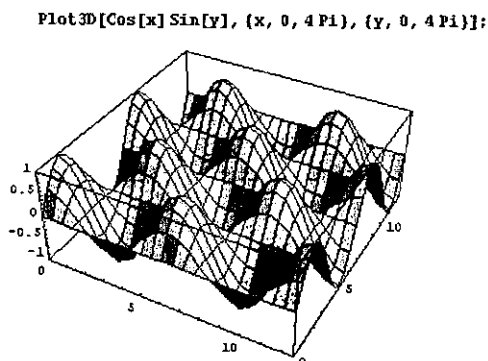


Figure 20: Plotting an egg-box.

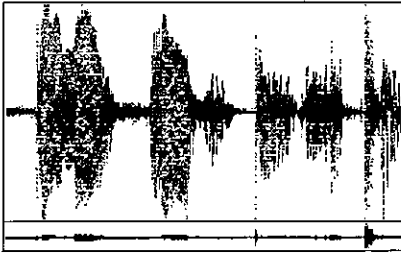
One of Mathematica's party tricks is to treat a sound file as a continuous function. Or alternatively to treat a function (like the Riemann Zeta Function) as a sound to

play (it sounds way-out, man!). Here is the author's voice, supplying an error signal for a well-known product (which exhibited a touching faith in its customers' intelligence):

```
fool = Import["C:\\ADRYTUM\\SYSTEM\\fool.wav"]
```

- Sound -

Show[fool]



- Sound -

```
ListPlot[Abs[Fourier[fool[[1, 1]]], PlotJoined -> True,  
PlotRange -> {{0, 5000}, All}];
```

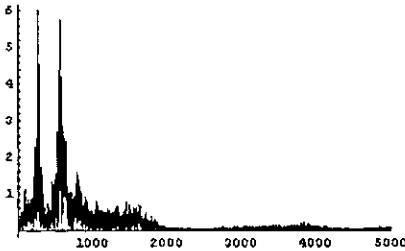


Figure 21: Signal analysis of the author uttering:
"Only a fool would do that!"

If you click the output cell (or select menu: Cell/Play sound), it plays the sound. (This works in MathReader too). Next I Fourier-analysed it. All this took a few brief moments to compute.

What I Didn't Like

My one big grouse – and it's maybe something they can't do a thing about – is that MathReader does not allow the contents of cells to be altered or re-executed in any way – except to redraw plots and replay sounds. You need the fully-functioning Mathematica to alter cells. Fair enough, you might think – but this simple fact prevents you placing a working calculator in the public domain – one containing a formula to which anybody can set their own numbers. I know this might be impossible to achieve with Mathematica as it is constructed: because the boundary between notebook composition and formula evaluation is such a hazy one. Even to contemplate such a possibility betrays my Babbage way of thinking – I'm assuming that there is some sub-engine inside the Mathematica kernel which can be disabled on its own, which substitutes values into variables and then mills the formula through a 64 bit arithmetic accumulator. One single accumulator? – then what about lists, what about vectors and matrices?

I still can't escape the feeling however that were Wolfram Research to crack this problem and still not give away the Crown Jewels, they could corner the market in public domain calculators. They already have CalcCenter [5] – a step along the way, and the webMathematica product [7] can be seen as directed towards this issue. But to make it possible for anyone owning a Mathematica licence to publish a free working calculator would in my opinion have a tectonic effect on world education and the use of higher mathematics in all walks of life. It could make Excel as obsolete as the Abacus, which gives me, for one, a wicked sense of glee.

APLers are familiar with the problem as it concerns the Execute function. Can we give the passengers plastic knives and forks to cut up their rubber chickens, but not to slit the pilot's throat? Or would it turn out to be easier to fund Wolfram Research as a UN agency and give its products away free to the world?

Where Does this Leave APL?

As a committed APLer (still) – I want to know why I shouldn't abandon APL in favour of Mathematica for all purposes. I think it's unworthy of an intelligent person to bury my head in the sand over this, or dismiss the problem by saying "each to his own". There are applications I could write easily in APL which I wouldn't contemplate in Mathematica. Is this just because an old dog can't be taught new tricks? Mathematica has more in common with J than a "classic" APL, insofar as the programmer's work is embodied in a text file, a direct descendant of the humble session log of a 1960s interactive interpreter, rather than a black-hole workspace, so what I'm about to say applies even more to J.

Is Mathematica really the way to go to express our creative thoughts – viz back to a 17th century notation designed for parchment instead of forward to a twentieth-century one designed for computers? Or to slant it differently: a widely understood and internationally accepted notation, rather than a “starved palette” of terminally overloaded nouns, verbs and adverbs which will only ever really be understood by a few narrow specialists? For individuals and groups the issue may boil down not to quality and fitness for purpose, but to culture, training, background, job-protection, and ultimately elitism and even name-calling. This of course applies to the language of Mathematics itself every bit as much as to APL or J.

This is something I’d like to explore – and I invite others to explore – in a future article. What is it about APL that gives me an edge over somebody armed with Mathematica (or any of its rivals, like MAPLE) when it comes to programming a sophisticated application?

Conclusion

Well – it’s a Weapon of Maths Instruction! Send in the inspectors!

References

- [1] MathReader, free download: <http://www.wolfram.com/products/mathreader/>
- [2] Mathematica Information Center: <http://library.wolfram.com/infocenter>
- [3] Wolfram Research: <http://www.wolfram.com>
- [4] Full documentation for Mathematica: <http://documents.wolfram.com>
- [5] CalcCenter, calculators repository: <http://documents.wolfram.com/calccenter/>
- [6] Publicon: <http://www.wolfram.com/products/publicon/>
- [7] webMathematica: <http://www.wolfram.com/products/webmathematica/>

LEARN

Objects for APLers (Part 3 of 3)

by Morten Kromberg

Editors Note on Sections 1 - 8

The first four chapters of the *Introduction to Object Oriented Programming For APL Programmers* were published in Vector 22.1, and the next 4 in Vector 22.2. As Dyalog Version 11 has matured through the pre-release phase, there have been plenty of minor revisions, and some significant new capabilities introduced. Please use the latest download of the full document from the Dyalog website as your reference material for the earlier sections.

This series now concludes with the final 4 chapters. These are fully in line with the released version of the software, and no revisions are expected.

9. Deriving from Dyalog GUI Classes

In addition to the classes which you can write in APL, version 11.0 allows you to work with:

- Dyalog GUI classes, including OLE Servers and OLE Controls (ActiveX components)
- Microsoft.Net classes

In version 11.0, these classes can be used in exactly the same way as instances of classes which have been implemented in APL. You create instances of them with `NEW`, and you can manipulate the public properties, methods and events which they expose (but unlike APL classes, you cannot see or modify the implementation).

For example:

```
MyForm←NEW 'Form' (('Caption' 'Hello World')('Size'(10 10)))

XL←NEW 'OleClient' (←'ClassName' 'Excel.Application')
XL.ActiveWorkbook.Sheets[←'Sheet2'].Index

2

Using←'' A Set the .Net search path
now←System.DateTime.Parse ←'2006-06-06 06:06:06.666'1

unow.(Year Month Day Hour Minute Second Millisecond)
2006 6 666
```

Note that the class *name* for built-in GUI objects is provided as a string rather than as a reference to a name in the workspace. The above objects were all accessible in version 10.1 of Dyalog APL, using slightly different mechanisms (which are still available in 11.0, in order to allow existing applications to continue working). The big difference is that:

- Indexing of default properties is supported (see the Sheets example above)
- Most importantly: you can write APL classes which derive from built-in and external classes, which is what this chapter is about.

In version 11.0, it is not possible to create instances of OLEControl objects using NEW (or classes which derive from OLEControls). This limitation will be lifted in a subsequent release.

A Standard Dialog Box

Dyalog GUI Classes provide basic building blocks for GUI applications. Each class, be it a Form, Button, List- or Combo-box, Edit, Grid, Treeview (etc.), is a very general component with a large number of properties which can be used to provide a variety of different behaviours depending on your application requirements. In any given application, you are likely to use certain behavioural patterns frequently. If you write an APL class which uses a GUI class as its base class, you can create “custom controls” which are easier to use in your particular application context, than the original Dyalog classes.

Imagine that your company has a corporate standard for the appearance of dialog boxes:

¹ If you are reading this, the world apparently did not come to an end as the clock ticked past 6/6/6.

- The form will have standard dialog box look (EdgeStyle Dialog, Border 2)
- Each form will have a "Quit" and a "Save" button in the bottom left corner ("attached" to corner if the form is resized).
- The form cannot be closed except by clicking on one of these buttons
- The form Caption and its Size must always be provided when an instance is created.

We can create a suitable class which derives from 'Form': (you can find this class in the workspace `DerivedGui` in the `OO4APL` folder):

```
:Class Dialog : 'Form'
A Implement Company Standard Dialog Form

A Has 'Save' and 'Quit' buttons at bottom left
A Coord Pixel, Cannot be Closed except by pressing Quit or Save

A Usage: □NEW Dialog (Caption Size formProps)
A Set Save.onSelect to take control of Save button behaviour

:Field Public Save
:Field Public Quit

▽ Create(cap size formprops);z
:Access Public
z+('Coord' 'Pixel')('EdgeStyle' 'Dialog')('Border' 2)
z+z,('Caption'cap)('Size'size),formprops
:Implements Constructor :Base z

onClose+~1 A Disable Close event
z+('Size'(22 100))('Attach'(4p'Bottom' 'Left'))
Save+□NEW'Button'(('Caption' 'Save')
('Posn'((Size[1]-30),10)),z)
Quit+□NEW'Button'(('Caption' 'Quit')
('Posn'((Size[1]-30),120)),z)
Quit.onSelect+~doQuit'

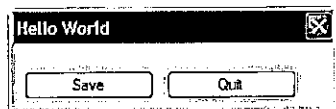
▽

▽ doQuit args
:Access Public
Close
▽

:EndClass A Dialog
```

Which allows:

```
f1+NEW Dialog ('Hello World' (50 250) 0)
f1.BCol+192 192 255 A Pale Blue
```



Note that the name of the GUI class is quoted when it is used as a base class, in the same way as it would be in the argument to `NEW`. Apart from that, all the principles we have discussed so far about the use of the public members of the base class apply in the same way as if the base class had been written in APL. For example, we can set the `BCol` property to change the background colour.

In the class code, note that we can refer to the `onClose` property of the form directly in our constructor: It is a public property exposed by our base class and thus immediately available to derived class code – and to any user of the derived class. Our `doQuit` function can call the `Close` method of the form directly to close the form. With the above class in our arsenal, we can start each dialog from a slightly higher level than a raw `Dyalog Form` object.

A Labelled Edit Field

One pattern which is often repeated in applications is an edit field with an attached label. It would be nice for application code to be able to treat such a pair as a unit, rather than have to position, size and track them separately. We can achieve this with a class which derives from the built-in `Edit` class and adds some right-justified text just to the left of the edit box:

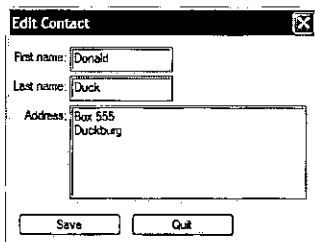
```
:Class Editfield : 'Edit'
A An Edit field with an associated Label on the left
A Usage example: NEW Editfield
  ('Price:' '10.5' (10 30) (0 50) (<'FieldType' 'Numeric'))

:Field Private Label A Ref to a Text object

▽ Make(label text posn size editprops)
:Access Public
:Implements Constructor :Base ('Text' text)
  ('Posn' posn)('Size' size),editprops
Label+##.NEW'Text'(('Text' label)('Points'(posn+3 3))
  ('Halign' 2)) A Created in container (##)
▽
:EndClass A Editfield
```

Of course, a more sophisticated implementation might have options for the positioning and alignment of the label, but this class illustrates the principle. Creating "company standard" dialog boxes for data entry is now a bit easier than before:

```
f1=NEW Dialog ('Edit Contact' (200 300) 0)
f1.First+f1.EDIT EditField ('First name:' '' (10 60) (0 100) 0)
f1.Last+f1.EDIT EditField ('Last name:' '' (38 60) (0 100) 0)
f1.Address+f1.EDIT EditField ('Address:' '' (66 60) (90 230)
                          (c'Style' 'Multi'))
f1.(First Last Address).Text+'D...' 'Duck' ('Box 555' 'Duckburg')
```



Our Dialog class exposes the Save button as a public field, so we can collect the data by defining a function and assigning it to the Select event:

```
▽ Update(button event);form
[1] form+button.## A Find the form
[2] ContactInfo+form.(First Last Address).Text
[3] form.Close
▽

f1.Save.onSelect+'Update'
```

Finally, we could collect our notes, tidy up and put the whole thing up into a little Contact class with an Edit method:

```
:Class Contact
:Field Public FirstName+''
:Field Public LastName+''
:Field Public Address+0p+''

▽ Edit;f1
A Uses #.Dialog and #.EditField
:Access Public
f1=NEW #.Dialog('Edit Contact'(200 300)0)
f1.First+f1.EDIT #.EditField('First name:' ''(10 60)(0 100)0)
f1.Last+f1.EDIT #.EditField('Last name:' ''(38 60)(0 100)0)
f1.Address+f1.EDIT #.EditField('Address:' ''(66 60)
                          (90 230)(c'Style' 'Multi'))
```

```

f1.(First Last Address).Text+FirstName LastName Address
f1.Save.onSelect+'Update'
□□Q'f1'
▽

▽ Update(button event);form
  A Private method used by Edit as callback on Save button
  form=button.##
  FirstName LastName Address+form.(First Last Address).Text
  form.Close
▽
:EndClass A Contact

```

We have not created a constructor for this class, so new instances will have the values declared at the start of the class definition. Register your first three friends as follows:

```

Friends+3*□NEW Contact
Friends.Edit

```

This pops us three modal dialog boxes in a row, one after the other – which is hardly ideal. We'll improve on that in the next chapter.

10. Interfaces

As we have seen, a *base class* provides core functionality which other classes can build upon. The core functionality is available from each derived class, unless the derived class intentionally overrides all or part of it. This makes it possible for programs which “know about” the base class behaviour to use most, if not all, classes derived from the same base. If necessary, a user of a derived class can *cast* the instance to the base class using dyadic □CLASS. The following expressions call the List method of ExcelWorkbook via an instance of PlanBook.

```

w*□NEW PlanBook 'c:\temp\widgets.xls'
(ExcelWorkbook □CLASS w).List 'c:\temp'

```

There are situations where it is also useful for classes which do *not* derive from a common base class to expose common behaviour – an *interface*. In the same way as with classes which derive from a common base class, a program written to use a particular interface should be able to use any class which *implements* the interface.

For example, we can define an interface called `iEditable` which requires an object to be able to:

- Paint itself inside a subform at a given location on a GUI Form and return a reference to the subform which it created for itself
- Pick up new values for its properties when asked to do so

We can then write an editor which was able to edit *any* instance of *any* class which supported this interface. By convention, the names of interfaces begin with a lowercase `i`. Our `iEditable` interface definition might look like this:

```
:Interface iEditable

  ▽ SubForm+Paint(Container Position)
  A Paint instance in Container at Posn, return ref to SubForm used
  ▽

  ▽ Update
  A Update properties of instance from Painted controls
  ▽

  ▽ UnPaint
  A Remove any references to resources created by Paint
  ▽

:EndInterface A iEditable
```

The interface definition contains no code, which will leave many APL programmers wondering what it is for! In most other languages, an interface definition would contain a little more: Declarations of the types of all the arguments and results. APL interface definitions *can* also include type information, if we want to export them for other languages to use – see the final sections on Microsoft.Net for more about this. There *are* other reasons why an interface definition is useful, which we will investigate in a moment.

Let us modify the `Contacts` class we created in the previous chapter, and replace the existing `Edit` and `Update` methods with an implementation of `iEditable`:

```
:Class EditableContact : iEditable
  :Field Public FirstName+''
  :Field Public LastName+''
  :Field Public Address+Opc''
  A --- iEditable implementation
  :Field Private idSubForm A ref to subform is stored

  ▽ {SubForm}+Paint(container position)
  :Signature SubForm+iEditable.Paint Container,Position
```

```

SubForm=idSubForm+container.□NEW 'SubForm'
  (('Posn'position)('Size'(120 200))('BCol'container.BCol))
SubForm.First+SubForm.□NEW #.EditField
  ('First name:'FirstName(10 60)(0 100)0)
SubForm.Last+SubForm.□NEW #.EditField
  ('Last name:'LastName(38 60)(0 100)0)
SubForm.Address+SubForm.□NEW #.EditField
  ('Address:'Address(66 60)(48 130)(c'Style' 'Multi'))

```

▽

▽ Update

```

:Signature iEditable.Update
FirstName LastName Address=idSubForm.(First Last Address).Text

```

▽

▽ UnPaint

```

:Signature iEditable.UnPaint
idSubForm=0

```

▽

:EndClass A Contact

With this class defined, we can create a form and arrange our contacts on it. Note that, in order to access the interface, we have to cast each instance of `EditableContact` to `iEditable`, in the same way that we might cast to a base class if we wanted to access that. This is in order to make it possible for a class to add an interface without the risk of name conflicts between interface member names and members of the class itself:

```

contacts+31□NEW EditableContact
e_contacts+iEditable □CLASS"contacts A Cast each to iEditable
e_contacts[1].□nl -3 A Each one exposes 3 methods
Paint UnPaint Update

```

```

f1=□NEW Dialog ('My Contacts' (480 300) 0)
e_contacts[1].Paint f1 (0 0)
e_contacts[2].Paint f1 (150 0)
e_contacts[3].Paint f1 (300 0)
A or:      e_contacts {α.Paint f1 w}"(0 0)(150 0)(300 0)

```

At this point, pause to fill in the form:

The image shows a window titled "My Contacts" with a close button (X) in the top right corner. Inside the window, there are three identical-looking groups of input fields. Each group consists of three labels with corresponding text boxes: "First name:", "Last name:", and "Address:". The first group contains the text "Donald", "Duck", and "Duckburg". The second group contains "Dolly", "Duck", and "Duckburg". The third group contains "Scrooge", "McDuck", and "Alaska". At the bottom of the window, there are two buttons labeled "Save" and "Quit".

When we are ready, we run the update functions:

```
e_contacts.Update A Runs iEditable.Update on each instance
contacts.(FirstName LastName)
Donald Duck    Dolly Duck    Scrooge McDuck
```

We could also have defined a function to do the update and connected it to the Save button:

```
upd←{e_contacts.Update}
f1.Save.onSelect←'upd'
```

The reason we need an `UnPaint` method can be illustrated by the following sequence. Note that the form does not disappear when expunged. This is due to the references (in the private fields called `idSubForm`) inside each contact:

```
⌞ex 'f1'
e_contacts.UnPaint
```

Of course, if `contacts` and `e_contacts` were local to the function doing the above work, or if we expunged these, the references would also disappear.

Avoiding Name Conflicts

Through the ages, many APL developers have “independently discovered” the need for something like interfaces, and written code which (for example) checks the class of the name 'Paint', and if the function is present deduces that “the interface is available”. However, this is dangerous: a class might have a method called Paint which has nothing to do with “being iEditable”. Even if we are rigorous and check all the names in the interface, there is a risk of confusion with similar interfaces. In addition to this: What if our class already has a method called Paint and we want to add the interface to it?

The important reason for having an interface definition is the avoidance of these name conflicts between interfaces, or between an interface and names used by the class. In the EditableContact class, each of the interface functions has a :Signature statement, for example:

```
▽ {SubForm}←Paint(container position)
... stuff snipped ...

:Signature SubForm←iEditable.Paint Container,Position
```

It is the name iEditable.Paint in the :Signature statement which declares that the function implements the Paint method of the iEditable interface. The name of the actual function is not actually used. It makes sense to use the same name as the interface function, but if we needed to avoid name conflicts we could call it ie_Paint or foo – anything we like.

Conclusion

The chapter on interfaces concludes the introduction to object oriented concepts as implemented in Dyalog APL version 11.0. The following chapters show how object orientation can be used to integrate APL with other tools which inhabit the Microsoft.Net Framework.

11. Using The Microsoft.Net Framework

Operating systems are designed to hide the details of the machine hardware from developers, making it possible to write applications without understanding the details of how the chips which manage memory, disk, keyboard, screen and other peripherals are controlled. As operating systems have evolved, they have tended to provide ever higher levels of abstraction. Environments like Microsoft.Net

provide object-oriented encapsulations of everything from low level hardware interfaces to high level GUI and Databases.

If you are running Dyalog APL on a machine where the Microsoft.Net Framework is available, you have access to a vast collection of tools for application building, which are supplied by Microsoft. All of these classes, any classes you have acquired from third parties, plus the classes you write yourself in a language which supports .Net (Visual C#, Dyalog APL, and dozens of other languages), can be used in the same way as you would use the APL classes described in the preceding chapters.

All of the “.Net classes” mentioned above, whether they are something you have written yourself or they were provided by Microsoft, reside in files called *Assemblies*. Microsoft has decided to reuse the extension “.DLL” for these files, so they have the same extension as traditional Dynamic Link Libraries. There can be dozens, or hundreds of assemblies on your machine, so a mechanism is required to name the assemblies that an application would like to use.

`⎕USING` contains a list of assembly names (see the Dyalog APL documentation for details). If one of the elements is an empty vector, this is taken to mean the Microsoft.Net Framework classes contained in the assembly called `mscorlib.dll`. This core library contains the most commonly used classes. To give an impression of the types of services provided by .Net, the remainder of this chapter will explore a (very) small selection.

System.Environment

`System.Environment` is a class which contains a number of useful shared properties and methods which provide information about the platform on which the application is running:

```

⎕USING←'' ⌞ This is interpreted as ⎕USING←,c''

System.Environment.⎕nl -2
CommandLine CurrentDirectory ExitCode HasShutdownStarted
MachineName NewLine OSVersion ProcessorCount StackTrace
SystemDirectory TickCount UserDomainName UserInteractive
UserName Version WorkingSet

se←System.Environment
se.(Version OSVersion MachineName)
2.0.50727.42 Microsoft Windows NT 5.1.2600 Service Pack 2 GOLLUM

se.SpecialFolder.⎕nl-2
ApplicationData CommonApplicationData CommonProgramfiles Cookies
...etc...
```

```

se.(GetFolderPath SpecialFolder.ProgramFiles)
C:\Program Files

se.(t{w (GetFolderPath SpecialFolder#w)}"SpecialFolder.[NL-2]
ApplicationData      C:\Documents and Settings\mkrom.INSIGHT\...
CommonApplicationData C:\Documents and Settings\All Users\...
...etc...

[avise.NewLine
4 3

```

We could have named the System namespace in `□USING`. This would have allowed us to leave the System prefix out of our references to classes:

```

□USING+ 'System'  A Equivalent to □USING+ ,c' System'
Environment.Version
2.0.50727.42

```

The author's personal preference is still to use the fully qualified names – I suspect this may change as I (and the rest of the APL community) start using these classes more frequently, and start to recognize Environment as meaning System.Environment. **However:** Note that APL will only search for .Net classes if the name which is used would give a VALUE ERROR in the APL workspace. The use of so-called “namespace prefixes” in `□USING` increases the likelihood of a name conflict between your own names and those in an assembly which you are trying to use (for example, Version is more likely to conflict with a name in the workspace than is System.Version). We'll return to this topic in the next chapter.

System.Globalization.CultureInfo/DateTimeFormatInfo

The DateTimeFormatInfo class contains information about the date and time formats for a given culture. You can get hold of an instance of DateTimeFormatInfo for the current culture, or for a specific one:

```

current←System.Globalization.CultureInfo.CurrentCulture
current.(Name EnglishName)
da-DK Danish (Denmark)

dtf←current.DateTimeFormat
dtf.[n] -2
AbbreviatedDayNames AbbreviatedMonthGenitiveNames
AbbreviatedMonthNames AMDesignator Calendar CalendarWeekRule
CurrentInfo DateSeparator DayNames FirstDayOfWeek ...etc...

dtf.MonthNames
januar februar marts april maj juni juli august ...

dtf.FullDateTimePattern

```

```
dddd, dd MMMM yyyy HH:mm:ss
```

```
    dtf.GetShortestDayName"0 1 2 3 4 5 6
    sø ma ti on to fr lø
```

```
    dtf.NativeCalendarName A SO much valuable information!
Den gregorianske kalender
```

```
    de=(NEW System.Globalization.CultureInfo('de-DE')).DateTimeFormat
    de.DayNames
    Sonntag Montag Dienstag Mittwoch Donnerstag Freitag Samstag
```

de-DE means German as spoken (or rather, written...) in Germany.

System.DateTime and System.TimeSpan

DateTime and TimeSpan provide tools for working with timestamps, including doing arithmetic on them:

```
    +may29+NEW System.DateTime (2006 5 29)
29-05-2006 00:00:00
    +aday+new System.TimeSpan (24 0 0) A 24-hour timespan
1.00:00:00
    +may28+may29-aday
28-05-2006 00:00:00
    +\5p aday
1.00:00:00 2.00:00:00 3.00:00:00 4.00:00:00 5.00:00:00
    (+\5paday).Days
1 2 3 4 5
```

```
    nextweek+may28++\5p aday
    ,[1.5]nextweek
29-05-2006 00:00:00
30-05-2006 00:00:00
31-05-2006 00:00:00
01-06-2006 00:00:00
02-06-2006 00:00:00
    nextweek.(Month Day)
5 29 5 30 5 31 6 1 6 2
```

```
    nextweek>may29+aday
0 0 1 1 1
    (nextweek-System.DateTime.MinValue).Days
732459 732460 732461 732462 732463
```

System.IO.DirectoryInfo

The DirectoryInfo class contains methods for inspecting the contents of file system folders:

```

temp=NEW System.IO.DirectoryInfo ('c:\temp')
temp.CreateSubdirectory='subdir'
c:\temp\subdir
  (temp.GetDirectories='*.*').(Name CreationTime)
DyalogWebSite 30-05-2006 15:35:40
js            24-02-2006 22:30:06
subdir        12-06-2006 14:17:14
  (temp.GetDirectories='subdir').Delete 1

files=temp.GetFiles('a*.dws')
  ffiles.(FullName CreationTime LastAccessTime)
c:\temp\A.DWS      18-01-2006 17:17:01  05-06-2006 21:57:06
c:\temp\ab.DWS     12-04-2006 15:54:36  05-06-2006 21:57:06
c:\temp\ado.DWS    17-06-2005 13:22:08  05-06-2006 21:57:06

,[1.5] CLASS files[1]
System.IO.FileInfo
System.IO.FileSystemInfo System.Runtime.Serialization.ISerializable
System.MarshalByRefObject
System.Object

```

The final result above shows that each element of the result is an instance of `System.IO.FileInfo`, which derives from `System.IO.FileSystemInfo`, which implements the interface `System.Runtime.Serialization.ISerializable` derives from `System.MarshalByRefObject`. At the end of the day, everything derives from `System.Object`.

System.IO.FileInfo

As we have seen above, the `Getfiles` method of `DirectoryInfo` returns instances of `System.IO.FileInfo`, which is a companion class for `DirectoryInfo`:

```

(a←files[1]).Onl -2
Attributes CreationTime CreationTimeUtc DirectoryName
Exists Extension FullName IsReadOnly LastAccessTime
LastAccessTimeUtc LastWriteTime LastWriteTimeUtc Length Name

a.(Name Exists IsReadOnly)
a.DWS 1 0

z←a.(CopyTo='c:\temp\z.dws')
z.(FullName CreationTime)
c:\temp\z.dws 14-06-2006 16:28:30
z.Delete

```

Summary

The above examples represent a very small subset of the classes provided by the Microsoft.net framework. There are classes for handling web requests in HTTP and FTP format, for compressing files, sending and receiving e-mail, GUI,

database access, graphics and printing, the list is almost endless. It is our intention that, following the release of version 11.0, Dyalog will produce sample code for many of the most useful classes, to demonstrate how applications written in version 11.0 can tap into this vast resource.

In addition to providing the framework itself, the .Net environment specifies calling conventions which mean that classes implemented in all .Net languages – including Dyalog APL version 11.0 – are fully interoperable. A class written in any language can use, derive from, be called, and be used as a base class by any other .Net language. This allows us to integrate APL with other languages and tools more easily than ever before.

12. Using APL Classes from .Net

In the workspace called `DotNet` in the `OO4APL` folder, there is a class with two “mathematical” methods in it:

```
)copy DotNet
DotNet saved ... etc ...
Maths.Round (of) 2
3.14
Maths.Fib 10
55
```

The definition of the class is:

```
:Class Maths

  ▽ r←Round(n decimals);base
    :Access Public Shared
    base←10*decimals
    r←({0.5+n*base})÷base
  ▽

  fibonacci←{ R Tail-recursive Fibonacci from ws "dfns"
    α←0 1
    ω←0:0pa
    (1↓α,+/α)▽ ω-1
  }

  ▽ r←Fib n
    :Access Public Shared
    r←fibonacci n
  ▽

:EndClass R Math
```

We can make this class available to all Microsoft.Net applications if we copy it into a workspace and subsequently export the workspace as a .Net *assembly*. As a service to users of typed languages like C#, we probably want to add type declarations of the .Net types of the parameters and results of the public methods before we do the export. This is not strictly necessary, but without it all types will default to System.Object. The result of this will be that most users have to cast data to or from System.Object in order to use it. Our assembly will be more pleasant to use if we can declare everything using the simplest or closest .Net type.

To make the declarations, we must first add a :Using statement which allows us to find .Net data types (this is usually done at the top of the class script):

```
:Using System
```

Next, we must add one line to each of our methods (a class containing these declarations exists in the workspace DotNet under the name MathsX). Note that dynamic functions can not be used as public methods in a class, but must be "covered" by a traditional function which supports declarative statements.

```

v r=Round(n decimals);base
:Signature Double=Round Double N, Int32 Decimals

v r=Fib n
:Signature Double=Fib Int32 N

```

The first :Signature declares that Round returns a result of type Double (which means a double-precision floating-point number), and takes two parameters. The first is a Double which is called N (most .Net development tools will make this name visible to a developer using our class), and the second parameter is a 32-bit integer called Decimals.

After adding the signatures, we are ready to export our class:

```

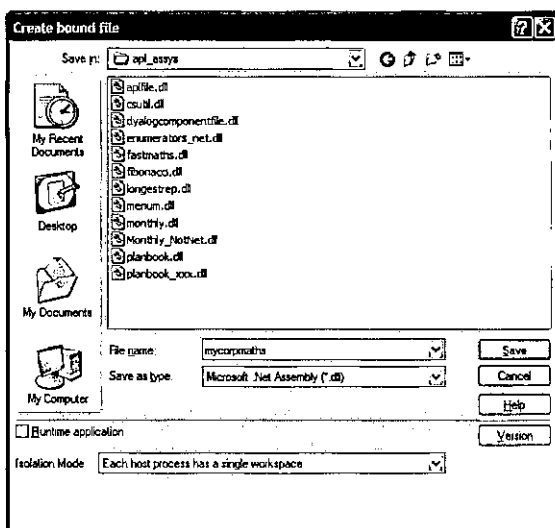
)clear
clear ws
)NS MyCorp
#.MyCorp
)CS MyCorp
#.MyCorp
)copy dotnet Maths
...dotnet saved Mon Jun 19 14:01:18 2006

```

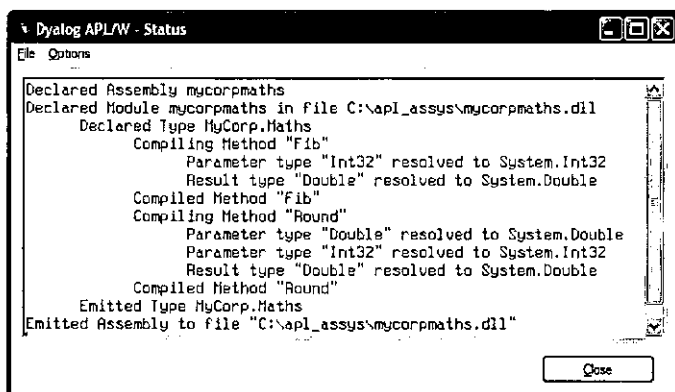
In the above example, we created a namespace MyCorp and copied Maths into it. This will export our class inside a .Net namespace called MyCorp, so that the class can be referred to as MyCorp.Maths. It is customary to embed classes within at

least one level of namespaces – often two levels, typically a company name followed by a product name – in order to organize classes in applications which use classes from a number of different sources.

Select File | Export from the session menu, select “Microsoft .Net Assembly” as the file type and probably uncheck “Runtime Application”:



When you press Save, the following text should appear in the Status window:



The output allows us to check that the two methods Fib & Round were exported, with the expected parameter and result types. We can now write a C# programme which uses our class, for example:

```
public class Test
{
    static void Main()
    {
        System.Console.WriteLine("Round(2/3,2) = ");
        System.Console.WriteLine(MyCorp.Maths.Round(0.6666,2));
        System.Console.WriteLine("Fib(10) = ");
        System.Console.WriteLine(MyCorp.Maths.Fib(10));
    }
}
```

The workspace DotNet in the OO4APL folder contains a programme called CSC, which can be used to call the C# compiler. If we save the above programme in a file called c:\apl_assys\testmaths.cs (or copy the file by this name from the OO4APL folder), we can compile it by calling:

```
'winexe' CSC 'c:\apl_assys\testmaths' 'c:\apl_assys\mycorpmaths.dll'
0 Microsoft (R) Visual C# .NET Compiler version 7.10.6001.4
for Microsoft (R) .NET Framework version 1.1.4322
Copyright (C) Microsoft Corporation 2001-2002. All rights reserved.
```

The left argument of 'winexe' instructs the C# compiler to build a Windows Executable rather than an assembly. The right argument contains the name of the source file (without the .cs extension), optionally followed by any assemblies which it needs. The result is a file with the same name as the source file, but with an extension of .exe. We call it from APL using □CMD:

```
□CMD 'c:\apl_assys\testmaths.exe'
Round(2/3,2) = 0.67
Fib(10) = 55
```

It is important to note that our assembly (mycorpmaths.dll) can be used from ANY language which can use Microsoft.Net. The list includes APL – see the web page <http://www.gotdotnet.com/team/lang/>. In fact, we can use the assembly from APL, in the same way that we would use any other .Net assembly:

```
)clear
clear ws
    □using~',c:\apl_assys\mycorpmaths.dll'
    MyCorp.Maths.Round (01) 3
3.142
```

13. Inheriting from a .Net Base Class

In the same way that you can inherit from one of the built-in Dyalog GUI classes, you can write a class which derives from a .Net base class. You can derive from classes which you write yourself in a .Net language (including Dyalog APL ©), classes purchased from a 3rd party tool vendor, or from Microsoft .Net Framework classes. Very many of the Framework classes are *sealed* for security and performance reasons, which means that they can not be used as base classes, but quite a few are intended for use as base classes.

We will now look at writing our own class in C#, as an optimization of our fibonacci function. The Fibonacci series is generated using a recursive algorithm which does a VERY small amount of processing for each level of recursion. Even though tail-recursive dynamic functions are highly efficient, a compiled, strongly typed language like C# is going to be able to execute this particular type of algorithm significantly faster than APL can do it.

If we build our Maths class on a base class written in C#, we can easily substitute some of the methods by methods written in C#. For example, we can write the following (ugly but fast) C# class:

```
using System;

namespace OO4APL
{ public class Maths
    { public static double Fib(int n)
      { double n1 = 0, n2 = 1, r = 0;
        int i=1;
        while (i<n)
        { i++; r = n1 + n2;
          n1 = n2; n2 = r; }
        return n2;
      }
    }
}
```

If we save this code in a file called fibonacci.cs (which can be copied from the OO4APL folder), and compile this class using our CSC function:

```
CSC 'c:\apl_assys\fibonacci'
0 Microsoft (R) Visual C# .NET Compiler version 7.10.6001.4
  for Microsoft (R) .NET Framework version 1.1.4322
  Copyright (C) Microsoft Corporation 2001-2002. All rights reserved.
```

We can now write a new class `FastMaths`, which derives from `OO4APL.Maths`. It contains the methods which are still written in APL, the rest will be inherited from the base class:

```
:Class FastMaths : OO4APL.Maths
:Using ,c:\apl_assys\fibonacci.dll

    ▽ r←Round(n decimals);base
      :Access Public Shared
      base←10*decimals
      r←([0.5+n*base)÷base
    ▽

:EndClass R Math
```

In version 11.0, a class which derives from a .Net class must be exported as a .Net class before it can be used. This restriction may be relaxed in a future release. If you want to test the class quickly, without creating a .Net assembly as a .dll, it is sufficient to export the class to memory using the menu item *File | Export to memory*, after which it can be called:

```
FastMaths.Round(01)3
3.142

FastMaths.Fib 10
55
```

The first method runs in APL, and the second in C#.

Conclusion

This concludes the Introduction to Object Oriented Programming for APL Programmers. If you want to learn more, it is time to read the Release Notes and the Language Reference for Version 11.0, both of which contain a number of additional examples.

J-ottings 48: Perming and Combing

by Norman Thomson

No, not an accidentally misdirected submission to *The Hair Stylist*, but rather an extension of one of Eugene McDonnell's token reduction examples (see Vector vol. 22 no.3) which involves generating systematic lists of permutations, where a permutation of order n is to be understood as a list of integers $i..n$ in some order, for example 3 0 2 1 4 .

Permutation Lists

A permutation list is a list of lists in which all of the $n!$ possible permutations occur once and once only. The most interesting lists of this kind are those in which the ordering is in some way systematic, the most obvious being the lexical order which Eugene discusses, and which is readily obtainable using A. (Anagram Index) as shown below. This ordering is also the result of Lehmer's algorithm, so Llist can be taken to denote either lexical or Lehmer.

Llist=: i.@! A. i.

NB. Lehmer/lexical listing

```

|:Llist 4
0 0 0 0 0 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3
1 1 2 2 3 3 0 0 2 2 3 3 0 0 1 1 3 3 0 0 1 1 2 2
2 3 1 3 1 2 2 3 0 3 0 2 1 3 0 3 0 1 1 2 0 2 0 1
3 2 3 1 2 1 3 2 3 0 2 0 3 1 3 0 1 0 2 1 2 0 1 0

```

(Note: In many of the illustrations which follow, transposition appears as the leftmost verb in the J line for the purpose of compactness of display. In these cases individual permutations should thus be read as columns reading from top to bottom.)

Factorial Digit Bases

Eugene defines a factorial digit base (that is, number base in the J sense) as:

```

fdb=.,}:@(>:@i.@-)
fdb 4
4 3 2

```

NB. factorial digit base

Each digit from 0 to $n-1$ has a unique representation in this base, as shown by:

```
fact=.fdb#:#i.@!
|:fact 4          NB. 0 to 23 as factorial digits
0 0 0 0 0 0 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3
0 0 1 1 2 2 0 0 1 1 2 2 0 0 1 1 2 2 0 0 1 1 2 2
0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
```

A permutation can be transformed to corresponding factorial digits by taking its items from the left and, for all but the last, recording the number of smaller digits which appear to its right:

```
nld=.#/:@:({. > }. )      NB. number of lesser digits
ptof=.{.}`(nld,ptof@.)@.(1&<@#)  NB. perm -> fact digits

ptof 3 0 2 1
3 0 1

each=.%>
|:ptof each <"1 Llist 4
0 0 0 0 0 0 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3
0 0 1 1 2 2 0 0 1 1 2 2 0 0 1 1 2 2 0 0 1 1 2 2
0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
```

The inverse of `ptof` is also obtainable recursively, this time by joining each factorial digit to the immediately prior permutation list, copying any lesser digits and adding one to the others. To those who know J, this is probably described more clearly in J than in English!

```
incgte=.] + <:          NB. increment if gtr or eq, else copy
ftop=.{. , -.}`({. , {. incgte ftop@.)@.(1&<@#)
                                NB. fact digits -> perm

|:ftop each <"1 fact 4
0 0 0 0 0 0 1 1 1 1 1 2 2 2 2 2 3 3 3 3 3
1 1 2 2 3 3 0 0 2 2 3 3 0 0 1 1 3 3 0 0 1 1 2 2
2 3 1 3 1 2 2 3 0 3 0 2 1 3 0 3 0 1 1 2 0 2 0 1
3 2 3 1 2 1 3 2 3 0 2 0 3 1 3 0 1 0 2 1 2 0 1 0
```

The above array demonstrates that ordering by factorial digits is equivalent to lexical ordering.

Factorial digits used with `ftop` resemble the left arguments of `b.` inasmuch as each digit in the appropriate range represents a function. For example, a factorial digit of integers descending to 1 such as `3 2 1` represents *rotate* (`|.`), a string of all 1s represents *1-shift*, a string of 2s followed by a 0 represents *2-shift*, a string of 3s followed by two 0s represents *3-shift*, and so on.

Other Systematic Orderings of Permutations

Another systematic ordering for permutation lists involves using a recursive method to generate what is known as the Tompkins-Paige ordering, first published in 1960:

```

      TPlist=.3 : 0
      if.y.>1 do.
        r=.,/(i.y.)|."1 each<(TPlist<y.),.<:y.
      else. r=.1 $0 end.
    )
    |:TPlist 4
0 1 1 0 2 2 1 0 2 2 0 1 2 2 0 1 1 0 3 3 3 3 3 3
1 0 2 2 0 1 2 2 0 1 1 0 3 3 3 3 3 3 0 1 1 0 2 2
2 2 0 1 1 0 3 3 3 3 3 3 0 1 1 0 2 2 1 0 2 2 0 1
3 3 3 3 3 3 0 1 1 0 2 2 1 0 2 2 0 1 2 2 0 1 1 0

```

In the above array, focussing on how the 3s are blended with repetitions of the next lower order permutations list is the easiest way to see how the Tompkins-Paige ordering is formed. In time efficiency terms TPlist and Llist are broadly similar.

There are obvious ways in which orderings such as Llist and TPlist can be used to generate further systematic orderings, for example

```

      |:3 - TPlist 4
3 2 2 3 1 1 2 3 1 1 3 2 1 1 3 2 2 3 0 0 0 0 0 0
2 3 1 1 3 2 1 1 3 2 2 3 0 0 0 0 0 0 3 2 2 3 1 1
1 1 3 2 2 3 0 0 0 0 0 0 3 2 2 3 1 1 2 3 1 1 3 2
0 0 0 0 0 0 3 2 2 3 1 1 2 3 1 1 3 2 1 1 3 2 2 3

```

Another means of generating systematic orderings is to use TPlist n as an index to any one of its component permutations, for example:

```

      |:(TPlist 3){1 0 2
1 0 0 1 2 2
0 1 2 2 1 0
2 2 1 0 0 1

```

More generally, the component permutation can be randomised as in the following tacit verb train:

```

      rlist=.TPlist { ?@! { Llist NB. list based on random perm
      |:rlist 4
2 3 3 2 0 0 3 2 0 0 2 3 0 0 2 3 3 2 1 1 1 1 1 1
3 2 0 0 2 3 0 0 2 3 3 2 1 1 1 1 1 1 2 3 3 2 0 0
0 0 2 3 3 2 1 1 1 1 1 1 2 3 3 2 0 0 3 2 0 0 2 3
1 1 1 1 1 1 2 3 3 2 0 0 3 2 0 0 2 3 0 0 2 3 3 2

```

Parity of Permutations

A basic property of permutations is that of *parity*, which means the oddness or evenness of the number of switches required to bring the permutation back to $i.n$. This property is related to factorial digit representations in the following way:

```

parity=.2&|@+/@ptof          NB. rem((sum of fact digs),2)
parity each<"1 TPlist 3
0 1 0 1 0 1
  parity each<"1 TPlist 4
0 1 0 1 0 1 1 0 1 0 1 0 0 1 0 1 0 1 1 0 1 0 1 0

```

Parity can also be obtained by considering the cycle form of a permutation as given by `C.` which returns a list of boxed lists (cycles) whose lengths are given by `#eachC.` :

```

C.t=.6 4 2 7 1 3 0 5
+-----+
|2|4 1|6 0|7 5 3|
+-----+

```

A cycle of even length has odd parity and a cycle of odd length has even parity so

```

#each C.t          NB. cycle parities are E O O E
1 2 2 3

```

Parities combine according to the 'not-equals' rule, so in the above case the overall parity is

```

par=.-:/@:-.@(2&|)@(#eachC.)    NB. parity by cycles
par t
0

```

Minimising Switches Between Adjacent Permutations

For `rlist 3`, the permutation list parity pattern is either `0 1 0 1 0 1` or `1 0 1 0 1 0` which means that parity alternates between successive permutations. For `rlist 4` the parity pattern is always one of

```

0 1 0 1 0 1 1 0 1 0 1 0 0 1 0 1 0 1 1 0 1 0 1 0

```

or

```

1 0 1 0 1 0 0 1 0 1 0 1 1 0 1 0 1 0 0 1 0 1 0 1

```

where in both cases, there are some consecutive permutations with the same parity. In programs which loop extensively through permutations computed in advance, the efficiency gained by minimising the changes between successive permutations can sometimes be worth considering. In 1963 S.M. Johnson of the RAND Corporation showed how to achieve such an ordering. A recursive technique is based on taking a single permutation at the next lower level, and creating $n+1$ further permutations by inserting n into all possible gaps starting either from the right or the left :

0 1 2 3	3 0 1 2
0 1 3 2	0 3 1 2
0 3 1 2	0 1 3 2
3 0 1 2	0 1 2 3

In array terms, the incoming value n is stitched to a skewed array of multiple copies of each of the next lower order permutations followed by a reverse skew which brings n onto the diagonal:

<code>rskev=:i.@-@# .> <"1</code>	NB. right skew
<code>lskev=:i.@# .> <"1</code>	NB. left skew
<code>mcopy=:\$(~ (,~ >:))@#</code>	NB. multiple copies
<code>agr=:lskev@(rskev@mcopy@[,.])</code>	NB. start inserts from right
<code>agl=:rskev@(lskev@mcopy@[,.])</code>	NB. start inserts from left

Now choose between right and left on the basis of parity:

<code>ag=.agr`agl@.(parity@[)</code>	NB. all gaps (right or left)
--------------------------------------	------------------------------

and finally bring everything together in

```

Jlist=:3 : 0
if.y.~:2 do.
  r=.,/(<"1 Jlist <:y.)ag each <:y.
else. r=:0 1,:1 0 end.
)
|:Jlist 4
0 0 0 3 3 0 0 0 2 2 2 3 3 2 2 2 1 1 1 3 3 1 1 1
1 1 3 0 0 3 2 2 0 0 3 2 2 3 1 1 2 2 3 1 1 3 0 0
2 3 1 1 2 2 3 1 1 3 0 0 1 1 3 0 0 3 2 2 0 0 3 2
3 2 2 2 1 1 1 3 3 1 1 1 0 0 0 3 3 0 0 2 2 2 3

```

This ordering not only involves just one switch between consecutive permutations, but additionally all such switches are between immediately neighbouring elements. However, the time to compute a `Jlist` is orders of magnitude greater than that required for a `Tplist` or an `Llist`, and so as, noted above, this method only pays off when there is a considerable amount of looping through pre-computed permutation lists.

Combinations and Permutations of r from n

To find ordered permutations of r items from n , make every possible selection (that is combination) of r elements from $i..n$, and then transform each of these into a list of its $r!$ possible permutations using any of the permutation list methods. Obtaining systematic combination lists is not quite as tidy a matter as finding permutation lists. One possible technique is to express $i..n$ as binary numbers and select those whose digit sum is r :

```
cnos=i.:@(2&^)@]      NB. integers from 0 to 2^n -1
bins=.#:@cnos         NB. binary nos from 0 to 2^n -1
mark=.].@((= +/"1@bins) # cnos) { bins      NB. marks for bins with digitsum = x.
combs=.mark #"1 i.@]   NB. transform marks to combinations
|:2 combs 4
0 0 0 1 1 2
1 2 3 2 3 3
```

Now obtain a permutation list appropriate to every combination:

```
perm=.,/(<@TPlist@[ {&> <"1@combs)      NB. r from n
|:2 perm 4
0 1 0 2 0 3 1 2 1 3 2 3
1 0 2 0 3 0 2 1 3 1 3 2
```

An Alternative Method of Obtaining Combinations of r from n

The following technique is another way of obtaining the defining binary array. mk is mark with its component binary number lists in a different order. The method is based on the fact that the combinations of r from n consist of all the combinations of r from $n-1$, together with n added to all the combinations of $r-1$ out of $n-1$. In binary number terms:

$r\ mk\ n$ is $(r\ mk\ (n-1)$ stitched with 0) joined to $((r-1)\ mk\ (n-1)$ stitched with 1)

For largish values the double recursion makes this method less appealing without adaptations which could obscure the fundamental principle. The double recursion also leads to the two stopping values embodied in the first two lines:

```
mk=.4 : 0
if. x.=y. do. r=.(1,y.)$1 return. end.
if. x.=1 do.r=.=i.y. return. end.
r=.(x. mk<:y.),.0),(<:x.)mk<:y.),.1
)
```

```
coms=.mk #"1 i.@]
|:4 coms 6
0 0 0 0 1 0 0 0 1 0 0 1 0 1 2
1 1 1 2 2 1 1 2 2 1 2 2 3 3 3
2 2 3 3 3 2 3 3 3 4 4 4 4 4 4
3 4 4 4 4 5 5 5 5 5 5 5 5 5 5
```

... and Finally a Bang !!

Anyone who has delved into such matters must know the phrase 'combinatorial explosion' which describes how soon space and time boundaries are exceeded for even quite modest values of the parameters. This article has outlined the basic arithmetic of permutations and combinations *en masse* for whose exposition I is particularly suitable. Skilful tinkering and use of parallel processing can extend these boundaries, usually at an understandable loss of some clarity – that's where programming ingenuity kicks in ...

PROFIT

Integer Apollonian Circle Packings in $\mathbb{J}$

by Cliff Reiter (reiterc@lafayette.edu)

Introduction

In recent years there has been an explosion in interest in packing of special tangent circles into circles. When three circles are tangent to each other in pairs, there are two circles that are tangent to those three (where lines may be thought of as circles of infinite radius). When a packing of circles includes those two tangent circles for every triple of tangent circles, the packing is called Apollonian. These tangent circles are also related to one another via a notion of circle inversion which goes back centuries, but the explosion of interest in *Apollonian* packings has occurred because of the beauty of the configurations and the rich structure that these configurations have recently been found to satisfy. We have no intention of attempting to recount the rich history or theory of these structures, but we do want to provide readers with enough background so that they could run some $\mathbb{J}$ experiments and see some of the beauty. Readers interested in further exposition on the recent developments and the underlying mathematics may want to start with [1]. Some of the beautiful recent results may be found in [2-6].

Descartes Configurations Give Others

Figure 1 shows an example of a basic configuration of four circles where each circle is tangent to the other three. This is known as a *Descartes* configuration. The curvature of a circle is the reciprocal of the radius and we have labelled those circles in the figure with their curvatures. If the curvatures are denoted b_1 , b_2 , b_3 , and b_4 , then the Descartes Circle Theorem says that

$$b_1^2 + b_2^2 + b_3^2 + b_4^2 = \frac{1}{2}(b_1 + b_2 + b_3 + b_4)^2.$$

Soddy published a poem recording his discovery of the theorem [9].

We check the theorem for the configuration in the figure as follows.

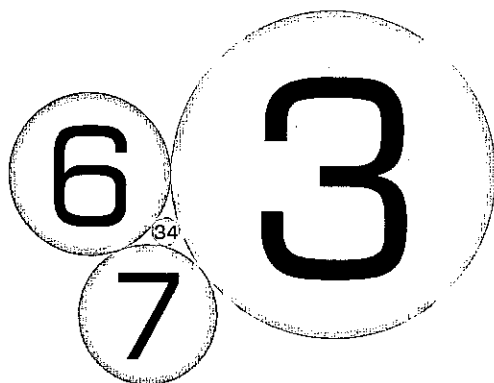


Figure 1. A Descartes configuration with curvatures 3, 6, 7, and 34.

```
bs=:3 6 7 34
+/*:bs
1250
-:*/+/bs
1250
```

If one curvature in the theorem is viewed as an unknown, say b_4 , then the quadratic formula can be used on the Descartes Circle Theorem to find the fourth curvature from the other three, but there are two possibilities that arise from the quadratic formula. One recovers the curvature we had, b_4 , and a new one appears. The new curvature b'_4 may be expressed in terms of the original 4 curvatures as $b'_4 = 2b_1 + 2b_2 + 2b_3 - b_4$.

Notice that if the original curvatures were integer, then the new curvature also will be integer. For the example above, we see

```
2 2 2 _1 +/ . * bs
_2
```

The negative radius may seem strange at first glance, but it corresponds to the fact that the tangencies are interior to that circle. Figure 2 shows the configuration that occurs with -2, 3, 6 and 7 as the curvatures.

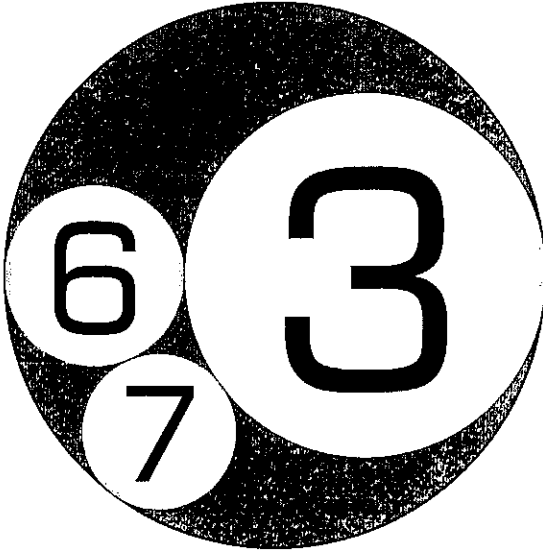


Figure 2. A Descartes configuration with curvatures -2, 3, 6, and 7.

The interchange of the curvature 34 and -2 curvature circles can be described geometrically in terms of circle inversions. However, for us, the main point is that by varying which curvature is replaced, many Descartes curvature lists can be produced from an initial one. Of course, we have not said anything about the centres of the circle and it takes bit of work with the law of cosines to realize a configuration by brute force from the list of curvatures. However, it turns out that if z_1, z_2, z_3 , and z_4 are complex numbers representing the centres of the circles, then Lagarias, Mallows and Wilks [6] showed that the product of the centres with the curvatures satisfies the same identity as the curvatures. This is known as the complex Descartes circle theorem.

$$(b_1 z_1)^2 + (b_2 z_2)^2 + (b_3 z_3)^2 + (b_4 z_4)^2 = \frac{1}{2} (b_1 z_1 + b_2 z_2 + b_3 z_3 + b_4 z_4)^2$$

The quadratic formula again applies, and the centre corresponding to the new circle, z'_4 , can be found from the following.

$$b'_4 z'_4 = 2b_1 z_1 + 2b_2 z_2 + 2b_3 z_3 - b_4 z_4$$

For example, this can be checked for the configuration in Figure 2.

```

      bz
      0
      0.166667
      0.333333
      0.285714
      0.285714

      +/*:*/"1 bz
      2.5j6

      -:*/:*/"1 bz
      2.5j6

```

It might seem natural to work with Descartes configurations by listing the curvatures and centres. However, the computations are uniform and can be done by matrix multiplication if we act on the curvature and the product of the curvature and centre which may be viewed as a pair of coordinates, rather than a complex number (these are curvature-centre coordinates). Moreover, round-off is something of a (persistent) problem, so we use exact rational representations. Below *br* gives the curvature-centre coordinates in rational form. The matrix *S1* produces an exchange of the type discussed, but distinguishing the first curvature rather than the last. Thus, the circle with curvature -2 from Figure 2 is replaced with the circle of curvature 34 in Figure 1.

```

      br
      0 0
      1r2 0
      2 0
      3r2 2

      S1
      2 2 2
      0 1 0 0
      0 0 1 0
      0 0 0 1

      S1 +/ . * br
      34 6 4
      3 1r2 0
      6 2 0
      7 3r2 2

```

If we apply *S1* again, we return to the original configuration *br*. However, we can use analogous matrices *S2*, *S3*, *S4* to modify other coordinates, leading to infinite chains of related Descartes configurations.

```

      S1 +/ . * S1 +/ . * br
_2      0 0
_3 1r2 0
6 _2 0
7 _3r2 _2

```

```

      S2 +/ . * br
_2      0 0
19 _15r2 _4
6 _2 0
7 _3r2 _2

```

In practice, we will put the Descartes configurations into a standard order so that we can avoid configurations that have already been checked and we will not pursue further variants when the sum of the curvatures is more than 2000. We define an array `SS` that contains all four transforming matrices. We then implement a function, `more_desc`, that takes two lists as its argument: a "done" list of configurations and those remaining "todo". The result is likewise a pair of "done" and "todo" lists created by applying all possible transformations, via `SS`, to the "todo" list and organizing the results. If we apply the function until it converges, we see several thousand Descartes configurations are produced from `br` (in less than a minute).

```
<"2 SS
```

_1 2 2 2	1 0 0 0	1 0 0 0	1 0 0 0
0 1 0 0	2 _1 2 2	0 1 0 0	0 1 0 0
0 0 1 0	0 0 1 0	2 2 _1 2	0 0 1 0
0 0 0 1	0 0 0 1	0 0 0 1	2 2 2 _1

```
L=:+/@: { .@: | : "2
```

```

more_desc=:3 : 0
'done todo'=.y
smoutput #todo
k=.2000>L todo
$m0=.(/:~)"2,/SS&( +/ . * )"2 k#todo
done=,~.done,todo
todo=,~.m0~.done
done;todo
)

```

```

md=:more_desc^:_ (i.0 4 3);,:br
1
4
12
36
108
324
804
1203
1059
651
...
6
3
0

```

```
$&.> md
```

5276	4	3	0	4	3
------	---	---	---	---	---

From the list of Descartes configurations we find and order the circles with curvature less than 2000. This results in about three thousand circles.

```

$d=:/:~.,./;md
5279 3

```

```

#d=:d#~2000>{."1 d
3035

```

Once the circles have been converted back to a more usual centre-radius format, J's graphics facilities may be used to draw the circles. We drew the circles using utilities derived from the *dwin2.ijs* script associated with the text *Fractals, Visualization and J*, 2nd edition [7]. A stand alone script that contains the drawing functions we used may be obtained from [8] and used to create these Apollonian packings. The default palette in that script uses 15 colours and selects the colour according to the residue of the curvature modulo 15.

Figure 3 shows the result of the plotting those thousands of circles.

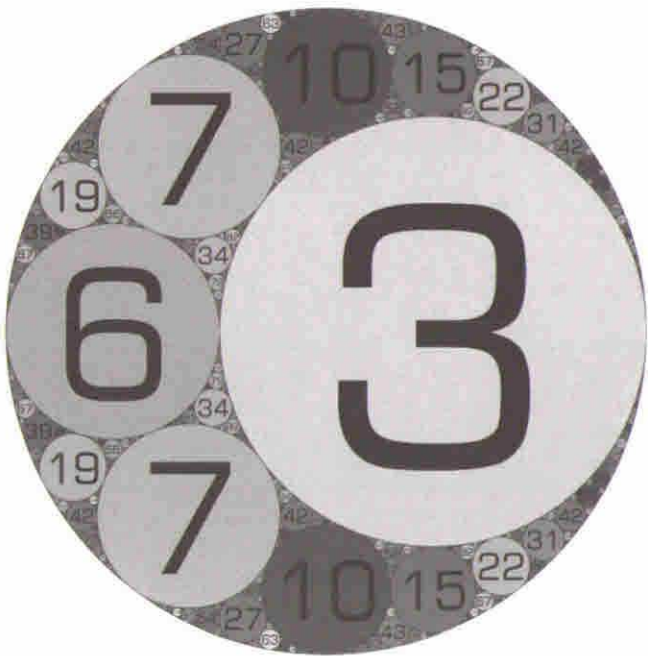


Figure 3. Apollonian packing with curvatures -2, 3, 6, 7.

Figure 4 Illustrates an Apollonian configuration based upon curvatures -8, 13, 21 and 24 and Figure 5 shows one with -1, 2, 2, and 3.



Figure 4. Apollonian packing with curvatures -8, 13, 21, 28.

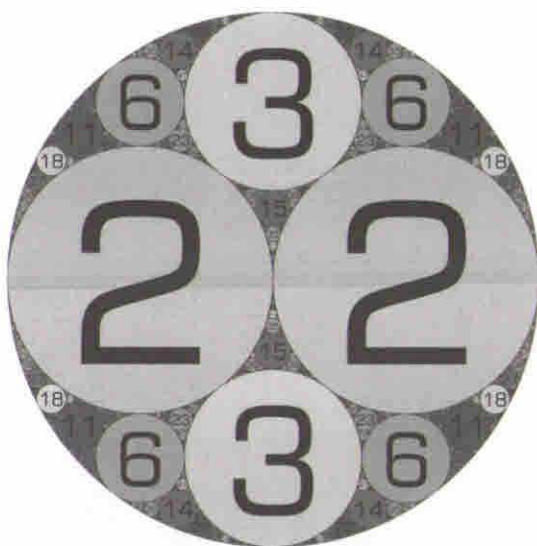


Figure 5. Apollonian packing with curvatures -1, 2, 2, 3.

A Special Configuration

There is a special configuration with curvatures 0, 0, 1, 1. The curvature of 0 corresponds to a straight line (a circle with infinite radius) and these lines are tangent to two circles of radius 1 (and each other at infinity) in an initial Descartes configuration. If we use 0 for the curvature and a unit vector perpendicular to the line in the positions for the curvature times the centre, then we will obtain an initial configuration that allows matrix multiplication to correspond to Descartes swapping.

```
      ]r0011=: 0 1,0 _1 0,1 0 1,:1 0 _1x
0 1 0
0 _1 0
1 0 1
1 0 _1

      S2 +/ . * r0011
0 1 0
4 3 0
1 0 1
1 0 _1
```

However, we also need to modify more_desc to limit the range of imaginary coordinates since this configuration is unbounded and has infinitely many repetitions of the circles of curvature 1. Figure 6 shows this configuration with three of the repetitions visible.

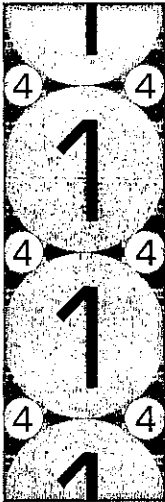


Figure 6.
Apollonian packing with
curvatures 0,0,1,1.

Initial Configurations

We have seen that obtaining the packing from a single Descartes configuration is possible once the details of the initial configuration are known. There is a theory that allows for canonical minimal Descartes configurations of curvatures, called root quadruples, to be identified. These configurations may be found by solving a simple Diophantine equation with some inequalities holding [1]. In particular, (a, b, c, d) is a root quadruple if $x^2 + m^2 = d_1 d_2$ where $x < 0 \leq 2m \leq d_1 \leq d_2$ and $a = x$, $b = d_1 - x$, $c = d_2 - x$, and $d = -2m + d_1 + d_2 - x$. For example, if $x = -7$, then we can look for other suitable parameters.

<@q:"0(*:m=:i.4)+*:x=:_7

7	7	2	5	5	53	2	29
---	---	---	---	---	----	---	----

So, for example seeing the relatively large factors of 5 and 10 available in the second box, we could take $x = -7$, $m = 1$, $d_1 = 5$, and $d_2 = 10$ giving $(a, b, c, d) = (-7, 12, 17, 20)$. We can obtain the rational curvature-centre coordinates we have used for Descartes configurations using function `to_rcf` defined in the script `ac3.ijs` in order to obtain an initial configuration.

```

to_rcf _7 12 17 20
_7      0      0
12      5r7     0
17 _48r35  2r5
20 _33r35 _8r5

```

The resulting Apollonian packing may be viewed at [8]. The simplicity of the process for swapping Descartes configurations via matrix multiplication allows us to explore and see some of the wonders of integer Apollonian packings of circles.

References

- [1] D. Austin, When Kissing Involves Trigonometry, <http://www.ams.org/featurecolumn/archive/kissing.html>.
- [2] R.L. Graham, J.C. Lagarias, C.L. Mallows, A. Wilks, and C. Yan, Apollonian circle packings: number theory, *Journal of Number Theory*, 100 (2003), 1-45.
- [3] R.L. Graham, J.C. Lagarias, C.L. Mallows, A. Wilks, and C. Yan, Apollonian circle packings: geometry and group theory I, the Apollonian group, <http://www.arxiv.org/abs/math?papernum=0010298>.

- [4] R.L. Graham, J.C. Lagarias, C.L. Mallows, A. Wilks, and C. Yan, Apollonian circle packings: geometry and group theory II, Super-Apollonian group and integral packings, <http://www.arxiv.org/abs/math?papernum=0010302>.
- [5] R.L. Graham, J.C. Lagarias, C.L. Mallows, A. Wilks, and C. Yan, Apollonian circle packings: geometry and group theory III, higher dimensions, <http://www.arxiv.org/abs/math?papernum=0010324>.
- [6] J.C. Lagarias, C.L. Mallows, and A. Wilks, Beyond the Descartes Circle Theorem, *American Mathematical Monthly*, 109 (2002), 338-361.
- [7] C. Reiter, *Fractals, Visualization and J*, 2nd ed. Jsoftware, Inc., 2000, <http://www.lafayette.edu/~reiterc/j/fv2/index.html>.
- [8] C. Reiter, Some integer Apollonian circle packings in J, <http://www.lafayette.edu/~reiterc/mvq/acp/index.html>.
- [9] F. Soddy, The Kiss Precise, *Nature* 137 (1936), 1021. <http://pballew.net/soddy.html>.

Contact:

Cliff Reiter

Department of Mathematics

Lafayette College, Easton, PA, 18042 USA

<http://www.lafayette.edu/~reiterc>

The Supposed Descent into Chaos of the Logistic Equation

$$x_{n+1} = kx_n(1 - x_n)$$

where k is less than or equal to 4 and x is between 0 and 1

by Sylvia Camacho

Foreword

Any publication which includes a discussion of chaos theory is almost certain to include, as a simple introduction to its concepts, a description of the logistic equation. In 1987 James Gleick [1] wrote an enthusiastic history which credits James Yorke with giving the field its name in his 1975 paper *Period Three Implies Chaos*. However it was a physicist turned biologist, Robert May [2], who devoted his time to a close investigation of the behaviour of the equation. It is not possible to appreciate why they found the output of the equation so intriguing without some sort of pictorial representation and when they started, computer graphics were not easy to access but nowadays the necessary software is cheap and common. As a modest desktop computer makes it an easy matter to reproduce the graphs which are so regularly used to illustrate logistic chaos, it ought to be a well-trodden, and by now arid, ground in which to sow seeds of doubt. Nevertheless Gérard Langlet planted some teasing questions during the 1990s in a few not very well known journals. Here, for example, is what he says in Vector 11/2 of 1994 [3 p83]:

The True Origin of Chaos for Iterated Applications

Successive iterations of the logistic equation are ALWAYS computed ... with computers.

In all computers, precision is limited. (on paper, with a slide rule or a calculator, precision is also limited.)

... with the usual IEEE standard, only 64 bits are available to code:

- a) the sign of the constant (1 bit)
- b) the exponent (11 bits, hence the maximum value 10^{308} because 308 is 2^{10} (i.e. 1024) divided by $\log_2 10$, then floored – rounded to the inferior integer knowing that one bit is also reserved for the sign of the exponent)
- c) the mantissa in 52 bits.

One may consider that "proofs of chaotic behaviour", based on iterated formulas or numeric integration should not be accepted as scientific results, unless the authors can prove that their calculations have been performed at least with $B+N$ bits for the mantissa of the floating point representation, given B as the number of necessary bits for the "accurate" binary encoding of the initial value(s), and N as the number of performed iterations:

In issue 11/4 of the same journal Donald McIntyre [4] relates the embarrassing story of an attempt to demonstrate the computation of π using Archimedes iteration for calculating the periphery of an inscribed polygon of increasingly many sides. The computer continued to produce more accurate estimates until it reached 3.14159 but the following eleven iterations fluctuated erratically until it halted at zero. McIntyre calls this cautionary tale, *The Perils of Subtraction*, and repeats Langlet's warning about the limitations of computer architecture. In consequence of this limited precision, Langlet advises that one should be very cautious about any results after the 51st iteration. His paper is a demonstration of the anomalies generated by a mind-set which looks for the characteristics of continuous functions despite using fundamentally discrete data, procedures and calculating equipment.

Historically the logistic equation was connected with Malthus' population studies and thus adopted the methods of statistics and probability. In these disciplines proportions of populations are normalised to be within the interval zero to one. Theoretically this allows infinite discrimination for infinite populations but in practice limits will be set by the calculation techniques used and mathematicians are notoriously uninformed about computer science. So, in order to get a feel for the equation, it is instructive to run it and look at some real computer output.

This is the J code used for the plots in the examples below.

```
g=: 13 : 'x.*y.*1-y.' NB. the basic logistic equation

n=: 4&g ^: (i.100)0.751 NB. as run for k=4,x=0.751
rnd=: 13 : '(.19j17 ":(x.*y.*1-y.)' NB. rounded
n=: 4&rnd ^@i.100)0.751 NB. run for rounded example
tru=: 13 : '(((10^17)*x.*y.*1-y.)%10^17' NB. truncated
n=: 4&tru ^:(i.100)0.751 NB. run for truncated example''
''int=: 13 : '<.(x.*y.*100-y.)%100' NB. integer for pop 100
n=: 2.4&int ^:(i. 100) 1 NB. integer simulation k=2.4, x=1
int=: 13 : '<.(x.*y.*1000-y.)%1000' NB. amend for diff. pops
```

note that the J argument x. is k of the logistic equation and J argument y. is x of the logistic equation.

APLIWIN code generating the samples of values in the text below.

<pre> ▽run1[□]▽ ▽ r←k run1 n;m [1] r←n [2] r←r,n+k×n×1-n [3] →(100=Pr)↑0 [4] →2 ▽ </pre>	<pre> ▽run2[□]▽ ▽ r←k run2 n;m [1] r←n [2] m←k×n×1-n [3] r←r,n+± 19 17 ▽m [4] →(100=Pr)↑0 [5] →2 ▽ </pre>
<pre> ▽run3[□]▽ ▽ r←k run3 n;m [1] r←n [2] m←L(10*17)×k×n×1-n [3] r←r,n+m*10*17 [4] →(100=Pr)↑0 [5] →2 ▽ </pre>	<pre> ▽runint[□]▽ ▽ r←k runint vec;m;n;pop;□io [1] □io←0 [2] r←m+vec[0] ◇ pop+vec[1] [3] m←k×m×pop-m [4] r←r,m←Lm+pop [5] →(pop=Pr)↑0 [6] →3 ▽ </pre>

Joey Tuttle pointed out that if you represent the variables as ratios, j calculates exact results -- e.g. for the case of $K=2$ and $x=0.75$ i.e. $3r4$

$2\&g \wedge: (i.10) 3r4$

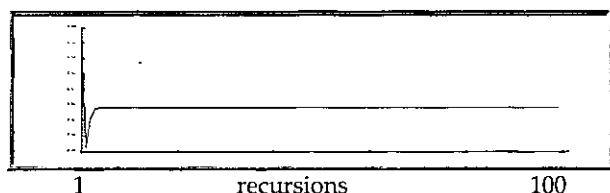
```

3r4 3r8 15r32 255r512 65535r131072 4294967295r8589934592
18446744073709551615r36893488147419103232
340282366920938463463374607431768211455r680564733841876926926749214863536422
912
1157920892373161954235709850086879078532699846656405640394575840079131296399
3...

```

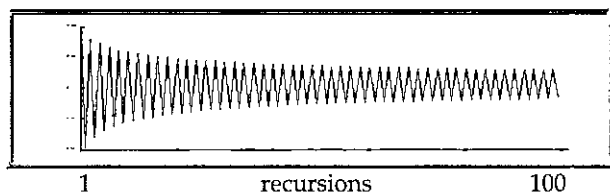
and went on to say, 'These kinds of experiments may be of interest in exploring precision of calculation and resulting chaos (or not). Of course, it doesn't take many iterations before the computation becomes quite onerous...'

For $x=0.75$ and $k=2$ the value stays constant after 6 iterations:



0.75 0.375 0.46875 0.498046875 0.49999237060546875 0.49999999988358468
0.4999999999999997 0.4999999999999997 0.4999999999999997

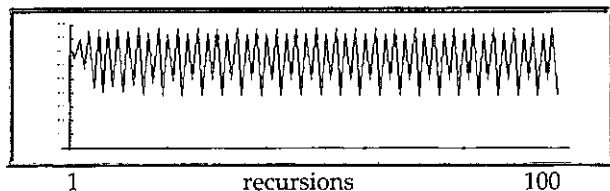
For $k=3$ the value never falls below 0.5625 and settles to a cycle of two gradually converging values, so no two are precisely the same:



0.75 0.5625 0.73828125 0.5796661376953125 0.73095991951413452 0.58997254673407348
0.72571482250255487 0.59715845670792045 0.72168070287040541 ...

For $k=3.5$ the first 51 values are unique but thereafter 4 values cycle if the numbers are compared at the program's default comparison tolerance of $1.4196976927394189e^{-34}$, but the cycle is of 12 values if tolerance is set to zero.

0.75 0.65625 0.78955078125 0.58156120777130127 0.85171719285410319
0.4420325568779035 0.86323921438260276 0.41320045597148339
0.8486304370475456 unique up to 51st and cycle thereafter ...



0.38281968301732014 0.82694070659143533 0.50088421030722548
0.87499726360246408

It is found that the values of the constant k for which the results have an apparently chaotic distribution lie between 3.57 and 4 but for $k=4$ there is, of course, a solution if the initial value is 0.75:

for $k=4$ and $x_n=0.75$

$1-x_n = 0.25$

therefore $x_{n+1} = 4 \times 0.75 \times 0.25$

$$= 4 \times 0.1875 = 0.75$$

and so does not change with recursion but:

for $k=4$ and $x_n=0.751$

$1-x_n = 0.249$

therefore $x_{n+1} = 4 \times 0.751 \times 0.249$

$$= 4 \times 0.186999 = 0.747996$$

and changes with each recursion, for example thus:

0.7510000000000000 0.7479960000000000 0.75399193593600000
0.74195238591793144 0.76583617179448144 0.71732451906261979
0.81108021365680400 0.61291640268494868 0.94899954401876202
0.19359763788377514 0.62447036995839122 0.93802850800968485

As shown here, after 3 recursions the limit of 17 significant decimal places has been reached. Thereafter the reliability of the results will depend upon the detailed working out of the IEEE (Institute of Electrical and Electronic Engineers) standard. For a large 'amplifier' like $k=4$, this will be sensitive at about recursion 50 as explained above.

Here, for example, are the 41st to 52nd and the 98th to 100th recursions:

0.99041787292736083 0.03796123965361185 0.14608073575029160
0.49896461757178029 0.99999571193290932 0.00001715219481266
0.00006860760245951 0.00027441158182557 0.00109734512043731
0.00438456381649586 0.01746135766653972 0.06862583461992357

0.12871194109343472 0.44858070925357962 0.98942422615654030

It is interesting to see what is changed by rounding to 17 significant places between each recursion. It makes 41 to 52 and 98 to 100 look like this:

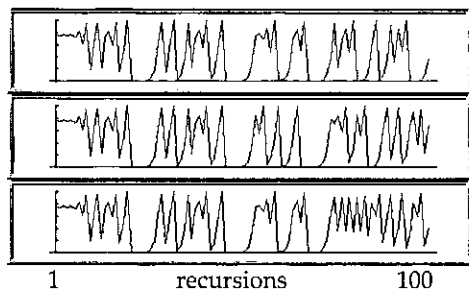
0.99041787287054966 0.03796123987650155 0.14608073657416103 0.49896461990444635
0.99999571195223092 0.00001715211752693 0.00006860729332718 0.00027441034546593
0.00109734017771292 0.00438454408898919 0.01746127944848360 0.06862553267402224

0.32725176800851702 0.88063219337526710 0.42047653346533304

So by the 100th recursion all similarity between the two sets of results has been lost. Of course, we need not round; we could merely truncate. In which case recursions 98 to 100 for each of the three trials are respectively:

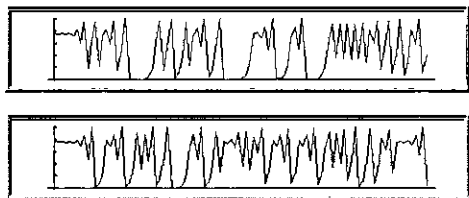
0.12871194109343472 0.44858070925357962 0.98942422615654030 (IEEE)
 0.32725176800851702 0.88063219337526710 0.42047653346533304 (rounded to 17)
 0.11041312655125879 0.39288827214573794 0.95410831102429791 (truncated at 17)

Viewed graphically with IEEE over rounded over truncated:



Langlet's warning about the 50th recursion is born out by the pictures. There is obvious divergence after 50 but the poor definition of the graph obscures the fact that none of the 3 sets of values is the same, even within the first 50. Nevertheless the way that these numbers jump around certainly looks chaotic and where there is no discernible pattern can we maintain that precise values are important? As a mathematical recreation and a demonstration of the deceptive power and limitation of computers, there may be no reason to quibble but, as Langlet says, the literature has many examples of similar mathematical researches being accepted uncritically as science.

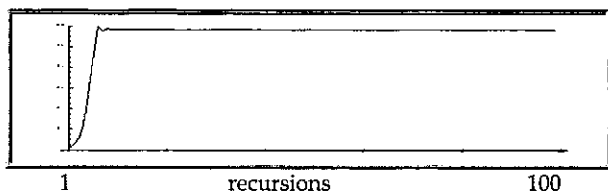
It seems to have become traditional to present the graphs in the form shown here, as connected points, as though imitating continuous differential equations, for which setting some intermediate value would enter the same curve. However difference equations do not smooth or average and we have known since the discovery of primes that some of the characteristics of numbers are unpredictable



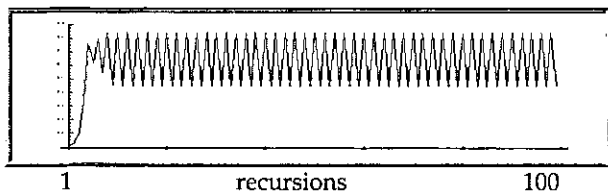
The graphs above show that starting value 0.7505 (top) is entirely dissimilar from starting value 0.751 (bottom).

What then is the significance of setting the starting value as a fraction of one? What is this 'one'? Robert May [2] is completely clear. For his example of goldfish, 'one' is the maximum population supportable by the pond. The feedback $x_n(1-x_n)$ represents the factors preventing the population reaching this 'extinction' value (except of course for $k=4$ and start population 0.5 which goes to zero!). The constant is interpreted as a characteristic rate of population growth for the given environment. The conclusion reached from the graphs is that if the rate of growth is less than 3 the population settles, after some fluctuation, to a single value. Between 3 and 3.57 the fluctuations seem to become a regular cycle. Beyond 3.57 there is no discernible pattern. May uses 0.01 as his initial value but what is the population? It could lie anywhere between one and infinity but we are imagining goldfish and, as we have seen, standard computing techniques set a limit of about 10^{17} significant places. A population of 100,000,000,000,000,000 fish is some shoal! However, let us look at the result of computing 100 generations using May's three examples.

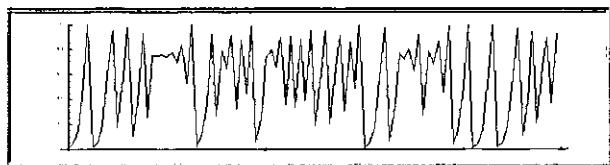
1. Initial value 0.01 with constant 2.4 produces a graph rising for the first 44 values to settle at 0.5833... recurring.



2. Initial value 0.01 with constant 3.4 produces a graph rising, then fluctuating before settling to a cycle of two. If examined recording only 4 significant places, the first part of the curve extends over 49 different values followed by a cycle with apparent values 0.8422 and 0.452. If examined for the maximum of 17 significant places there is no repetition among the hundred values.

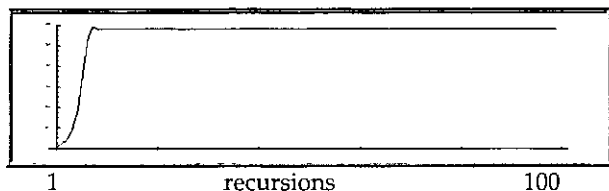


3. Initial value 0.01 with constant 3.99 has no apparent pattern.

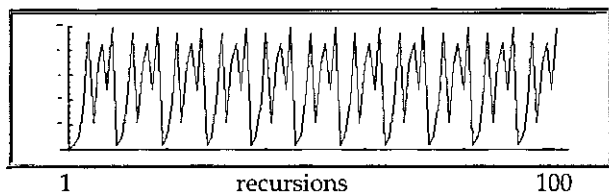


What then is the effect of assuming a maximum population of 100 and correcting to the nearest integer, on the reasonable grounds that fractional goldfish do not reproduce? If the algorithm is run using initial value 1 and constants as above, the results are as follows.

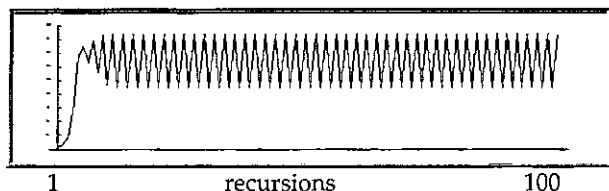
2.4 gives values 1 2 4 9 19 36 55 59 58 ... 58



3.4 gives values 1 3 9 27 67 75 63 79 56 83 47 84 45 84 45 ... 84 45

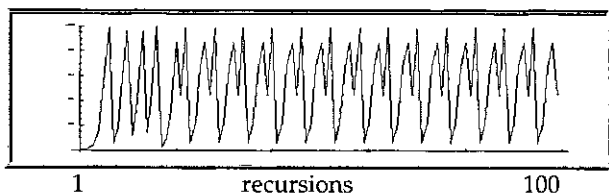


3.99 gives values 1 [3 11 39 94 22 68 86 48 99] cycle in brackets repeats.



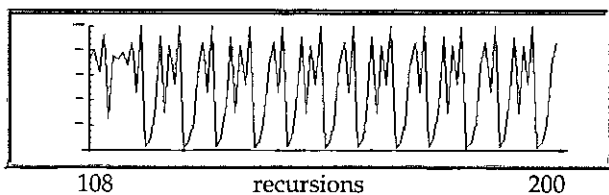
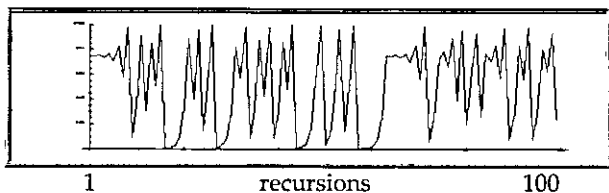
So the value of the constant that was supposed to yield chaos gives a cycle of nine among the hundred possible values. What then do we get for 3.99 with a population of one thousand? We get 1 3 11 43 164 547 988 47 178 583 970 116 409

964 138 474 994 23 89 323, 20 values, looking nicely chaotic, but what follows is 872 445 985 58 217 677, a cycle of 6 repeating for the next 980 generations.



One thousand is still a small population by the standards assumed for the logistic equation: not, one supposes, offering much scope for the development of chaos. What would a population of ten times the size (10,000) with a starting value of one and $k=3.99$ reveal? Would the repeating cycle be ten times as long? In fact it is much longer, being 168, following a non-repeating sequence of 47.

The two next graphs complete the investigation. They refer to a set of 10,000 recursions of a logistic equation look-alike, using only integer values. The constant k was set to 4, with initial value 7512. The first 108 iterations are apparently chaotic: they do not repeat. The remaining iterations up to 10,000 fall into a repeating sequence of 15 values, in no way chaotic.



The conclusion which these investigations support is that values of $k > 3$ which show cyclic patterns when graphed are deceptively simple, as the numbers on which they are based are mostly sets of slightly differing fractions which the display cannot resolve. The experiments with integers show that the cycle length and the non-repeating lead into it, is sensitive to a difference of 0.01 in the value of

k, but the effect of a fractional constant in this otherwise integer context is difficult to assess: the cycle is shorter for $k=4$, thus retaining integer throughout.

Taken together these trials suggest that interpretation based on low resolution graphs of a few hundred iterations may give false impressions of chaos. On a standard computer, it is not easy to run integral equivalents for the logistic equation for a population of 10^{17} , but it seems reasonable to suggest that cycles would be established and that they would be small by comparison with the population size. While both fractional and integral algorithms are mathematically legitimate, care must be taken in their application to real-world systems. Ilya Prigogine [5] noted that real populations fluctuate but not so wildly as to be called 'chaotic'. The chaos is an artifact of iterated irrational numbers which require a continuum for their realisation, but continuum is a mathematical concept for which there is no real-world warranty. It may still be true that the mathematical approximations provide insights into the working of physical systems with massive numbers of interacting entities but we must never lose sight of the primacy of the system being modelled: it is in no way constrained by the mathematical formalism and therefore no conclusions can be drawn unless independent evidence of applicability is available.

Summary

The Chaos is in the Continuum

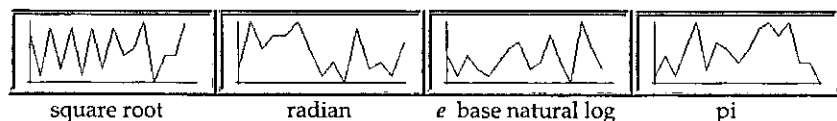
The Continuum is in the Imagination of the Mathematician

The Continuum is never in the Computer

No Computer-Generated Lines are Continuous.

We see a body moving and we know that it came from the position A to the position B, but we cannot understand how that happened. As soon as we think of *two* positions, we have left out an infinity of intermediate positions and yet we know that the body moved '*continuously*' from A to B, and we understand this term intuitively. But if we try to rationalise it we can only think of sharp positions, and that means *discrete* positions, which by its very nature contradicts the nature of a continuous change. As soon as we try to rationalise the concept of continuity, it evaporates into nothing.

Cornelius Lanczos [6 p139]



I created these plots in response to the following quotation from David Miller, who teaches philosophy at Warwick. In *Critical Rationalism* [7 p155] he refers to a 'recipe' that Ian Stewart gives for a function that will 'go chaotic': the recipe is *stretch and fold*. One is reminded of the Baker transformation [5 p269]. Miller suggests a process 'even more primitive than the logistic function':

Start with the decimal expansion of any real number, repeatedly multiply it by 10 (stretch), and take as the value of the operation the first digit after the decimal point (fold). Rational starting points will generate stable limit cycles, but irrational starting points will never do this. Yet every irrational number is as close as you like to a rational number and conversely.

Miller seems to suggest that this is a surprising consequence of the number system although its significance was known to Archimedes. I cannot do better than quote Cornelius Lanczos who, in the course of a chapter called *Evolution of the Number Field*, points out that there can be no exact representation of one-third as a decimal fraction and then goes on to explain the concept of a limit [6 p110]:

There will always remain a hair's breadth of space between the upper and the lower bounds. But we are satisfied with the fact that this hair's breadth can be made so terribly small that it becomes *smaller than any conceivable number*. Then we say that our sequence has a 'limit', and that our sequence defines a *definite number*, although in fact we never obtained that number. Of course, in our case we know what the limit is, namely $1/3$, which we can define without our sequence. . . .

There are two kinds of limit processes. In some processes, as in the one we have considered, the limit is *predictable*, in some others *not*. But this is immaterial because even a predictable limit, i.e. a number which can be exhibited by some finite means, without the infinite sequence, is completely replaceable by the infinite sequence which constitutes the limit process. . . .

It was the towering genius of Archimedes (287-212 b.c.) which pointed out the flaw in this argument. . . .

Archimedes pointed out that the proof required a *postulate*: 'Let A and D be two points of a straight line and B an arbitrary point to the right from D. then it is always possible to lay off the distance AD to the right a sufficient number of times to come eventually beyond the point B.' In arithmetical terms this can be stated as: 'Let ϵ be a positive number which may be chosen as small as we wish. Then we can always find a sufficiently large integer N such that N times ϵ shall become larger than 1.' This encapsulates the difference between mathematics and physics.

Planck discovered an absolute value for a 'least action' but it is only measurable because mathematics can postulate an arbitrarily smaller value. Computers, as part of the physical world, are subject to much coarser graining than this. All the 'irrational' numbers used to generate the picture above were necessarily defined to the computer as 'rational' numbers, that is to say an integer divided by 10^{17} , some number of one-hundred-million-billionth. The sequence of values produced by the logistic equation is analogous to the digits comprising the fractional part of π or any other such number. There, however, the digits measure increasingly smaller increments on the way to an unattainable limit. In the logistic interpretation, however, each expresses a number in the range one to 10^{17} . If any sequence repeats, the values are successive exact proportions of the population. If the numbers do not repeat, the results are the equivalent of an irrational expansion and could perhaps be called 'pseudo-chaotic', but if they are calculated using standard software, they will always be a computer construct.

That values of $k=4$ and $n=0.75$ should lead to a consistent value of 0.75 seems unremarkable, indeed satisfactory for those who feel equations should have tidy solutions. However, the precipitate reduction to zero when $k=4$ and $n=0.5$ makes an interpretation in terms of population dynamics somewhat strained. If a large population of 0.75 does not succumb, what is so fatal about a population of half the carrying capacity of the environment? It is this sort of anomaly that reinforces the message that such mathematical rigmaroles can carry scant weight as models of the real world, however realistic they may look in some of their manifestations.

Editor's Note

Full colour versions of all the pictures are included with the online version of this paper at www.vector.org.uk/archive/v223/chaos223.htm.

References

- [1] James Gleick, *Chaos*, Heinemann, London, 1988, ISBN 0-436-29554-x
- [2] Robert May, *The Best Possible Time to be Alive*, in Graham Farmelo Ed., *It Must be Beautiful*, Granta Books, 2002
- [3] Gérard Langlet, *Chaotic Behaviour Revisited*, Vector Vol.11 No.2
- [4] Donald McIntyre, *The Perils of Subtraction*, Vector Vol.11 No.4
- [5] Ilya Prigogine & Isabelle Stengers, *Order out of Chaos*, Fontana Paperbacks, London, 1985, ISBN 0-00-654115-1
- [6] Cornelius Lanczos, *Numbers Without End*, Oliver & Boyd, Edinburgh, 1968
- [7] David Miller, *Critical Rationalism*, Open Court, U.S., 1994, ISBN 0-6126-9198-9

Index to Back Numbers (Vols 13-22)

compiled by Gill & Adrian Smith

Author(s)	Title	Vol, Pg
Alis, David	Using a Defined Adverb to Trace Tacit Programs in J	Vol 13.3, 136
Alvord, Linda	Obituary: Remembering Ken	Vol 22.3, 59
APL97 (Toronto)	Conference Reports	Vol 14.2, 43
APL97 (Toronto)	CD Rom (with Abstracts)	Vol 14.3, 30
APL98 (Rome)	Conference Reports	Vol 15.1, 65
APL99 (Scranton)	Abstracts	Vol 16.1, 20
APL2000 (Orlando)	Abstracts	Vol 16.2, 7
APL2000 (Orlando)	Conference Report: APL 2000 User's Meeting in Orlando	Vol 17.3, 64
APL2000 (Naples Beach)	Conference Report: Notes and Pictures	Vol 19.3, 66
APL99 (Scranton)	Notes and Pictures (by Ray Cannon)	Vol 16.2, 61
APL Berlin 2000	APL Berlin 2000 Reports and Pictures	Vol 17.2, 56
APL2002 (Madrid)	APL2002 Conference Reports	Vol 19.2, 60
Apter, Stevan	A Conversation between Stevan Apter and Manfred von Thun	Vol 20.3, 101
Apter, Stevan	Functional Programming in Joy and K	Vol 21.4, 31
Askoolum, Ajay	Hacker's Corner: A Word6 APL Keyboard	Vol 13.1, 116
Askoolum, Ajay	Editorial: What is APL?	Vol 14.3, 3
Askoolum, Ajay	Book Review - It's APL, but not as we know it	Vol 15.1, 47
Askoolum, Ajay	Hacker's Corner: Context Sensitive Help and COM in APL+Win	Vol 15.2, 95
Askoolum, Ajay	DDE in APL+Win	Vol 15.3, 111
Askoolum, Ajay	Purging Name Pollution in APL+Win	Vol 16.1, 120
Askoolum, Ajay	Letter: APL+Win Application Migration	Vol 17.2, 112
Askoolum, Ajay	Review: APLX Beta 2.0.1 for Windows	Vol 21.1, 42
Askoolum, Ajay	APL Idioms	Vol 21.4, 92
Askoolum, Ajay	Developer Productivity with APLX v3 and SQL	Vol 21.4, 117
Baas, Michael	Hackers' Corner: Hack My Exe	Vol 21.1, 112
Baker, John & Neitzel, M.	Review: J3 - J grows up!	Vol 13.1, 48
Balako, Alexander	Namespaces: New Gem from Old Roots	Vol 16.1, 92
Balako, Alexander	APL+Internet Explorer	Vol 16.2, 113
Barman, Jonathan	Review: Advanced Windows Programming (Lescasse)	Vol 13.4, 63
Barman, Jonathan	Using VideoSoft FlexGrid Pro with APL+Win	Vol 15.3, 77
Barman, Jonathan	XML in Dyalog APL Version 9	Vol 17.4, 48
Barnetson, Paul	Letter: Memories of Ken Iverson and APL	Vol 22.4, 7

Author(s)	Title	Vol, Pg
Baronet, Dan	SHARP APL vs APL2 Coding Style	Vol 16.2, 87
Baronet, Dan	SHARP APL Operators	Vol 16.3, 85
Baronet, Dan	SHARP APL Tasks	Vol 16.4, 75
Baronet, Dan	Introduction to Tree-searching in APL: Using the TicTacToe game in 3D	Vol 17.1, 55
Baronet, Dan	Tools, Part 1: Basics	Vol 19.4, 106
Baronet, Dan	Tools, Part 2: Basics Too	Vol 20.1, 122
Baronet, Dan	Tools, Part 3: Object Comparison	Vol 21.1, 123
Baronet, Dan	Tools, Part 4: Regular Expressions	Vol 21.2, 126
van Batenburg, F.H.D.	A New Life For del: Structure Control	Vol 17.4, 125
van Batenburg, F.H.D.	Review: APLX, the Awakening of the Sleeping Beauty	Vol 19.1, 49
van Batenburg, et al	A Gui-quad for Graphical User Interaction	Vol 19.2, 76
van Batenburg, F.H.D.	Conformity of APL Implementations to ISO APL Standard Extended	Vol 21.3, 135
Bergquist, Gary	ZarkWin: Windows Programming without Quad-WI	Vol 17.1, 68
Bernecky, Bob	Letter: Strategies for Optimising the API Problem	Vol 13.4, 7
Bernecky, Bob	Obituary: Memories of Ken	Vol 22.3, 59
Berry, Paul	Expository Programming	Vol 22.3, 65
Bohrer, Douglas	Import/Export and the Wealth of Namespaces	Vol 13.3, 77
Breed, Larry	How We Got To APL1130	Vol 22.3, 68
Brisson, Gabriel & Reiter,	Parallel Processing with J	Vol 13.1, 86
Brooks, Fred	The Language, the Mind and the Man	Vol 22.3, 72
Brown, Jim	An APL to C++ Translator	Vol 20.4, 106
Brown, Jim	Ken at Yorktown Heights	Vol 22.3, 77
Brown, Robert G.	Letter: Leaving the Nest	Vol 13.3, 13
Budd, Timothy	Time-space Tradeoffs in Vector Algorithms	Vol 16.2, 101
Burke, Chris	APL and J (1) - Function Rank	Vol 13.1, 96
Burke, Chris	The J DLL and Extended Integers	Vol 13.1, 149
Burke, Chris	APL2 and J (2) - Indexing	Vol 13.2, 137
Burke, Chris	APL and J (3) Operators - Reduce, Scan, Outer Product	Vol 13.3, 118
Burke, Chris	OpenGL Graphics and J	Vol 14.1, 58
Burke, Chris	APL and J (4): Function Application and Axis	Vol 13.4, 113
Burke, Chris	APL and J (5): Operators - Inner Product, Each, Commute, Compose	Vol 14.1, 129
Burke, Chris	J4 and OOP	Vol 14.4, 103
Burke, Chns	Memory Mapped Files in J	Vol 15.4, 87
Buzin, Andrei	The API Problem as a Particular Recurrence Problem	Vol 14.1, 140
Byers, Bob	Review: Dyalog APL/W versions 7.2 & 8.0	Vol 13.1, 54
Byers, Bob	Review: NewLeaf Page Layout Tools	Vol 13.4, 73
Byers, Bob	Review: RainPro Publication Graphics	Vol 14.4, 33

Author(s)	Title	Vol, Pg
Camacho, Anthony	Hacker's Corner: Six Megabytes in 640K?	Vol 14.4, 112
Camacho, Anthony	GUI and Closed Systems	Vol 16.1, 86
Camacho, Anthony	BCS Specialist Groups Congress	Vol 17.2, 7
Camacho, Anthony	Review: The Cogwheel Brain by Doron Swade	Vol 17.3, 46
Camacho, Anthony	What, in Computing, is Worth Learning?	Vol 18.3, 71
Camacho, Anthony	Review: <How to Draw a Straight Line> by A B Kempe	Vol 19.1, 85
Camacho, Sylvia	40 Years On	Vol 15.3, 62
Camacho, Sylvia	The Dance of the Quirk	Vol 16.2, 82
Camacho, Sylvia	Can Anyone Tell Me Where I'm Wrong?	Vol 18.2, 86
Camacho, Sylvia	Guest Editorial: Should Vector Teach Mathematical Grammar?	Vol 20.1, 3
Camacho, Sylvia	How Wrong Was I?	Vol 21.2, 78
Camacho, Sylvia	Gérard Langlet: A Man of Distinction	Vol 22.2, 56
Camacho, Sylvia	The Supposed Descent into Chaos of the Logistic Equation	Vol 22.4, 116
Cannon, Ray	Vector on the Web	Vol 13.2, 16
Cannon, Ray	Vector on the Internet (progress report)	Vol 13.3, 15
Cannon, Ray	How to Ride a "BiCypher"	Vol 13.3, 85
Cannon, Ray	APL Characters on the World Wide Web	Vol 13.4, 14
Cannon, Ray	Hacker's Corner: Dynamic <EPW> in Dyalog APL	Vol 13.4, 111
Cannon, Ray	Conference: APL2000 1997 User's Meeting	Vol 14.3, 61
Cannon, Ray	Letter: Working with Compressed Data	Vol 14.4, 117
Cannon, Ray	APL98 - Observations	Vol 15.1, 74
Cannon, Ray	Utilities for the Dyalog Session	Vol 15.1, 138
Cannon, Ray	More Utilities for the Dyalog Session	Vol 15.3, 107
Cannon, Ray	Utilities for Dyalog APL/W Developers: UCMD Take 2	Vol 15.4, 109
Cannon, Ray	Report: Forests in Finland V (Report and Pictures)	Vol 17.4, 66
Cannon, Ray	My Yale 2001 Diary	Vol 18.1, 48
Cannon, Ray	Report: APL2000 User Conference at Naples, Florida	Vol 18.3, 57
Cannon, Ray	Hacker's Corner: Ghost Variables	Vol 18.3, 90
Cannon, Ray	Bach's Endlessly Rising Canon	Vol 18.3, 104
Cannon, Ray	Hacker's Corner: Packages in Dyalog APL	Vol 18.4, 126
Cannon, Ray	Review: Dyalog.NET First Impressions	Vol 19.1, 68
Cannon, Ray	Review: Dyalog APL CE (beta) - First Impressions	Vol 19.2, 46
Chamley, Anne	Functional Implementation of Moments Based Region Descriptors in J	Vol 16.3, 99
Charman-Williams, A	Array-based Bayesian Machine Learning Program: a J Implementation	Vol 16.2, 47
Chastney, Phil	An APL Unicode Font	Vol 16.1, 75
Chastney, Phil	Data Domains	Vol 16.2, 121
Chastney, Phil	Stack Operations	Vol 16.4, 84
Chastney, Phil	Reflexive Data Domains	Vol 17.3, 121
Chastney, Phil	APL Unicode Font - Extended	Vol 17.3, 139
Chastney, Phil	Function Models	Vol 18.2, 132
Chastney, Phil	Extending the Domains	Vol 18.3, 115
Chastney, Phil	A Girl's Best Friend	Vol 19.2, 109
Chastney, Phil	Getting to the Bottom of Things	Vol 19.4, 132
Di Chio, Paolo	APL98 Retrospective	Vol 15.1, 10

Author(s)	Title	Vol, Pg
Christensen, Gitte	Danish APL Group at Copenhagen	Vol 13.3, 74
Christensen, Gitte	Ken Iversen in Denmark	Vol 22.3, 81
Christiansen, Klaus Klug	Array-Based Logic	Vol 17.4, 85
Clark, Ian	DNA Fingerprinting in APL	Vol 14.1, 38
Clark, Ian	What Use Are User Defined Operators?	Vol 16.1, 61
Clark, Ian	Analysing Blackjack with APL (Finnish Forests Seminar)	Vol 16.4, 68
Clark, Ian	Magic Squares	Vol 17.1, 40
Clark, Ian	A Creative use of WATCHPOINTS to investigate Heritage ws in APL+Win	Vol 19.1, 92
Clark, Ian	Report: APL at FCRC2003 in San Diego	Vol 20.1, 56
Clark, Ian	Towards a General Theory of Formula Inversion	Vol 20.3, 100
Clark, Ian	TABULA: A Scientific Calculator for the 21st Century	Vol 20.4, 62
Clark, Ian	How to Win an Election	Vol 21.3, 63
Clark, Ian	Review: Mathematica 5.2	Vol 22.4, 59
Clark, John	Sudoku with Dyalog APL	Vol 21.4, 53
Clough, Eddie	Letter: Niven Number Computing Speed	Vol 14.4, 118
Clough, Eddie	Letter: Thoughts on the Urn Problem	Vol 21.4, 4
Cockshott, Paul	The Abstraction Mechanisms of Vector Pascal	Vol 18.4, 100
Cockshott, Paul	Efficient Compilation of Array Expressions	Vol 21.1, 91
Cox, Nicholas	Review: Fractals, Visualization & J by Cliff Reiter	Vol 13.1, 64
Cox, Nicholas	Letter: A Little Hook for Understanding	Vol 14.2, 15
Coxe, Angela	Inner Product Fractals from Fuzzy Logics	Vol 19.2, 65
Coxe, Angela & Reiter, C	Boolean Hexagonal Automata	Vol 19.3, 113
Crossley, David	APL98 - a Personal View	Vol 15.1, 66
Crossley, David	Data Conversion Between APL Versions	Vol 15.2, 121
Crossley, David	Bitmaps in Dyalog APLW	Vol 19.1, 138
Crossley, David	A Variable Editor	Vol 20.2, 111
Crossley, David	Notes about Execute Inverse Functions	Vol 20.3, 33
Crossley, David	Launching a Workspace in Windows	Vol 21.1, 131
Crossley, David	More on Execute Inverse	Vol 21.1, 140
Daintree, John	Dyalog.NET: An Introduction and Overview	Vol 19.1, 119
Daintree, John	Dyalog.Net: APLScript and Things Textual	Vol 19.2, 114
Daintree, John	Dyalog.Net: Putting It All Together (Web Services with Dyalog.NET)	Vol 19.4, 116
Daintree, John	Using Namespaces to Manage a CD Database	Vol 20.2, 70
Dazeley, Richard	J is for JPEG: Windows Bitmap to JPEG Baseline Compression	Vol 16.1, 103
Donnelly, Peter	OLE Automation with Dyalog APLW	Vol 13.2, 99
Donnelly, Peter	APL98 - The Dyalog OCX Workshop	Vol 15.1, 81
Donnelly, Peter	Driving the Windows Gui and COM from Dyalog version 9.0	Vol 17.2, 74

Author(s)	Title	Vol, Pg
Dumontier, Michel	Inversion of Boolean Matrices	Vol.18.1, 125
Dumontier, Michel	Matrix inversion modulo p, Linear systems modulo p	Vol.18.1, 134
Dumontier, Michel	MUCIPHER: a new Short Cipher that Escapes Cryptanalysis	Vol 18.2, 117
Dumontier, Michel	Playing with the Beast Number with J	Vol 18.4, 88
Dumontier, Michel	Hommage à Ken Iverson	Vol 22.3, 85
Dunne, Adam	CRCOMPARE in J	Vol 21.4, 7
Fil, Walter	Taking the Migraine out of Migration	Vol 17.1, 91
Fil, Walter	A First Look at the Causeway APL to C# Translator	Vol 22.1, 38
Finnish Forests	Conference Reports	Vol 16.4, 55
Garland, Simon	Matriks Reloaded	Vol 20.1, 49
Griffiths, Marc et al	Controlling APL Software Releases	Vol 13.4, 78
Guervos, Cano et al	Use of Multivariate Techniques for the Grouping Together of Districts	Vol 14.2, 112
Gustafsson, Tomas	NavSim – a Shipping Simulator: Part I	Vol 18.1, 57
Gustafsson, Tomas	Using Structures between GUI and Relational Databases	Vol 18.2, 78
Hager, Peter-Michael	Dyalog Next Generation Unnamed Window Refs	Vol 20.4, 98
Hager, Peter-Michael	Statistical Methods Discovered by the Power of APL	Vol 22.1, 115
Henriod, Claude	Letter: More on Niven Numbers	Vol 14.4, 120
Henriod, Claude	Letter: More Solutions	Vol 22.2, 10
Hill, Richard	Using J in Excel	Vol 21.1, 133
Hill, Richard	Ken Iverson at the End of the World	Vol 22.3, 93
Hoekstra, Bob	Review: Dyalog APL/M for Sun Solaris	Vol 14.3, 37
Hoekstra, Bob	APL98 - New Gems from Old Roots	Vol 15.1, 97
Hoekstra, Bob	Filecheck: Another Utility for the Dyalog Session	Vol 15.4, 113
Hoekstra, Bob	APLs for Linux	Vol 16.1, 7
Hoekstra, Bob	My SAX Experience – SHARP APL for Linux, Beta 5.1.0	Vol 16.3, 48
Hoekstra, Bob	Unix APL Power Shootout (Part 1)	Vol 16.4, 39
Hoekstra, Bob	Unix APL Power Shootout (Part 2)	Vol 17.1, 48
Hoesle-Klenzien, Conrad	Using OLE with Dyalog APL and Excel	Vol 18.4, 54
Hoffmann, Joachim (JoHo)	Hacker's Corner: A Special Locator in Dyalog APL/W	Vol 16.1, 108
Höjälvi, Kari	The Development and Q/A Process at Microsoft, US	Vol 17.4, 78
Holmes, Neville	Some Basic Functions for Composite Arithmetic	Vol 15.3, 49
Holmes, Neville	Letter: Sudoku Madness	Vol 22.2, 16
Holt, Dick	APL97 Survey Results	Vol 14.2, 92
Horton, Michael	Celtic Knotwork in J	Vol 21.1, 90
Hughes, Michael	Review: APL+Win Version 4.0	Vol 19.1, 74

Author(s)	Title	Vol, Pg
Hui, Roger	Extended Integers in J	Vol 13.2, 154
Hui, Roger	New Trace Facilities in J 3.03	Vol 13.3, 52
Hui, Roger	Honorary Doctorate for Ken Iverson	Vol 15.1, 8
Hui, Roger	The Tower of Hanoi	Vol 21.1, 78
Hui, Roger	Queens and Knights	Vol 21.3, 78
Hui, Roger	A Sudoku Solver in J	Vol 21.4, 49
Hui, Roger	A Lifetime of Working with Ken	Vol 22.3, 94
Hui, Roger et al	Ken Iverson Quotations and Anecdotes	Vol 22.3, 124
Iverson, Ken	APL in the New Millenium	Vol 22.3, 5
J User Conference (1996)	Conference Report	Vol 13.2, 48
Kai Jäger	Master and Slave: Dyalog APL Directs Excel!	Vol 21.3, 125
Korpela, Antti	APL and Bloomberg Data Control	Vol 22.4, 45
Jantunen, Veli-Matti	Dynamic Functions in Practice	Vol 18.4, 50
Jantunen, Veli-Matti	Data Compression with Dfns	Vol 20.2, 107
Johnson, Steph	Report: K Watch: Arthur Whitney in London	Vol 19.4, 53
Johnson, Steph	Report: Kx Systems Celebrates 10 Years	Vol 20.1, 54
Jones, William	Letter: Heron's Rule and Integer Area Triangles	Vol 14.2, 14
Kalliojärvi, Pertti	Using DyalogAPL based .NET extensions in ASP.NET applications	Vol 22.1, 77
Kangasniemi, Jouko	Rain-graphs to PowerPoint-land via OLE	Vol 19.3, 130
Kangasniemi, Jouko	Report: FinnAPL Forest Seminar VIII	Vol 20.4, 47
Karman, Jan	Bookbinders' Fun	Vol 13.4, 130
Kekäläinen, Kimmo	Translate your Dyalog GUIs with touching MAKE_-code	Vol 17.3, 131
De Kerf, Joseph	Swinging with APL: the Simple Pendulum	Vol 13.2, 128
De Kerf, Joseph	Niven Numbers and APL	Vol 14.2, 121
De Kerf, Joseph	Armstrong Numbers and APL	Vol 14.3, 138
De Kerf, Joseph	From Fibonacci to Horodam	Vol 15.3, 122
De Kerf, Joseph	From Horodam to Alanosov	Vol 16.2, 135
De Kerf, Joseph	Bell Numbers and APL	Vol 17.1, 131
De Kerf, Joseph	The Generalized Tribonacci Sequence	Vol 17.2, 131
Kertész, Adam	Letter: PROPER Capitals	Vol 16.3, 125
Kondrashev, Andrei	Internet Programming with the Internet Server Auxiliary Processor	Vol 14.4, 74
Korablev, J & Roche, G	Exploring ActiveX Data Objects from a Dyalog-APL Point of View	Vol 16.4, 116
Kromberg, Morten	OOO - Getting a Better Grip on OLE Objects	Vol 17.1, 121
Kromberg, Morten	R.I.P. OOF	Vol 17.2, 91
Kromberg, Morten	Why APL Programmers Don't Use Libraries	Vol 20.1, 62
Kromberg, Morten	OO for APLers (part 1)	Vol 22.1, 56
Kromberg, Morten	OO for APLers (part 2)	Vol 22.2, 107
Kromberg, Morten	OO for APLers (part 3)	Vol 22.4, 78
Landau, Eric	Strategies for Dealing with Multiple Arguments: The Case for Names	Vol 18.3, 64

Author(s)	Title	Vol, Pg
Langlet, Gérard	Obituary: Memories of Gérard Langlet (compiled Camacho)	Vol 13.4, 138
Langlet, Gérard	Paritons and Cognitons: Towards a New Theory of Information	Vol 19.3, 93
Lanzavecchia, Stefano	Editorial: GiochiAMO col mondo	Vol 14.4, 3
Lanzavecchia, Stefano	Editorial: APL98 and the Future of APL	Vol 15.1, 3
Lanzavecchia, Stefano	Editorial: You have performed an illegal operation	Vol 15.4, 3
Lanzavecchia, Stefano	Meeting: BAA Vendor Forum, May 21st 1999	Vol 16.1, 67
Lanzavecchia, Stefano	Funny Cube - Some Solutions	Vol 17.2, 35
Lanzavecchia, Stefano	The Puzzle Corner: the vier-neun puzzle	Vol 17.4, 38
Lanzavecchia, Stefano	Editorial: Cyberspace	Vol 19.4, 3
Lanzavecchia, Stefano	An APL Library	Vol 19.4, 8
Lanzavecchia, Stefano	Editorial: Quotable Quotes	Vol 20.2, 3
Lanzavecchia, Stefano	Editorial	Vol 20.3, 3
Lanzavecchia, Stefano	Wildheart	Vol 21.1, 3
Last, Phil	Letter: Locally defined niladic functions	Vol 18.1, 89
Last, Phil	Letter: Conditional Idioms	Vol 18.2, 104
Last, Phil	Lucidity Through Infix Notation	Vol 22.1, 72
Legrand, Bernard	Puzzle: Funny Cube	Vol 16.3, 44
Lescasse, Eric	Advanced Windows Programming with APL and Delphi	Vol 13.1, 122
Lescasse, Eric	APL+Win and VBxs	Vol 13.3, 92
Lescasse, Eric	Object Oriented Programming with APL+Win (Part I)	Vol 14.1, 106
Lescasse, Eric	Object Oriented Programming with APL+Win (Part II)	Vol 14.3, 109
Lescasse, Eric	A Grid Object for APL+Win	Vol 17.3, 76
Lescasse, Eric	APL2000 + Web Component (part 1 of 2)	Vol 21.3, 100
Lescasse, Eric	APL2000 + Web Component (part 2 of 2)	Vol 21.4, 62
Lescasse, Eric	APL+Santa Fe previewed at the APL2000 Conference	Vol 22.2, 41
Liebttag, David	The APL2 Interface	Vol 20.4, 101
Lucas, Jim	K-Lite: First Impressions	Vol 15.1, 36
Lucas, Jim	A Conference for Winners	Vol 16.3, 75
Mäenpää, Janne	Setting up a www-server with Dyalog APL and IIS	Vol 16.3, 131
Manktelow, Jonathan	Hacker's Corner: APL Meets MAPI	Vol 15.3, 70
Manktelow, Jonathan	Report: Vendor forum 19th May 2000	Vol 17.1, 13
Mansour, Paul	Show and Hide with Splitters	Vol 20.3, 137
Mansour, Steve	Arithmetic Boundary Checking	Vol 21.2, 119
Mansour, Steve	Letter: Ajay Askoolum's APL Idioms	Vol 22.2, 6
McCormick, Devon	The "Monty Hall" Problem Solved	Vol 21.1, 85
McCormick, Devon	Bayesian Financial Dynamic Linear Modelling	Vol 21.2, 65
McCormick, Devon	Report: Meeting of NYC JUG, June 2005	Vol 22.1, 20
McCormick, Devon	Letter: Reply to "Thoughts on the URN Problem"	Vol 22.2, 14
McCormick, Devon	Report: Meeting of NYC JUG 20060314	Vol 22.4, 49

Author(s)	Title	Vol, Pg
McDonnell, Eugene	At Play with J: Riding a Unicycle	Vol 13.1, 154
McDonnell, Eugene	At Play with J: Volutes	Vol 13.2, 144
McDonnell, Eugene	At Play With J: Extended Integers	Vol 13.3, 127
McDonnell, Eugene	At Play With J: Stumping the Rocket Scientist	Vol 13.4, 123
McDonnell, Eugene	At Play With J: Oh, No, Not Eigenvalues Again!	Vol 14.1, 135
McDonnell, Eugene	At Play With J: A Newer Random Link Generator	Vol 14.4, 122
McDonnell, Eugene	At Play With J: To Summarise	Vol 15.1, 132
McDonnell, Eugene	At Play With J: Maximum Infix Sums	Vol 15.2, 100
McDonnell, Eugene	At Play With J: Crosswords and Life	Vol 15.3, 99
McDonnell, Eugene	At Play With J: New Model Computer	Vol 15.4, 106
McDonnell, Eugene	At Play With J: New Big Deal	Vol 16.1, 113
McDonnell, Eugene	At Play With J: We'll Cross that Bridge when we Come to it	Vol 16.3, 126
McDonnell, Eugene	At Play with J: An Open and Shut Case	Vol 16.4, 99
McDonnell, Eugene	At Play with J: Blists in OLEIS	Vol 17.1, 110
McDonnell, Eugene	At Play With J: Someone Just Moved! Who Was It? or, Apter's Puzzle	Vol 17.2, 116
McDonnell, Eugene	Letter: DNA Analysis in APL - Charles Brenner	Vol 17.3, 112
McDonnell, Eugene	At Play With J: Four Cubes Redux	Vol 17.3, 113
McDonnell, Eugene	At Play with J: Erdős Numbers and Pierce and Engel Expansions	Vol 17.4, 113
McDonnell, Eugene	At Play with J: Boggle	Vol 18.1, 91
McDonnell, Eugene	At Play with J: The Counterfeit Coin Problem	Vol 18.3, 93
McDonnell, Eugene	At Play with J: Second Order Josephus	Vol 18.4, 132
McDonnell, Eugene	At Play with J: J be Nimble, J be Quick: Nim Addition	Vol 19.1, 105
McDonnell, Eugene	At Play with J: Beware Scholes	Vol 19.3, 137
McDonnell & Smith, R.	At Play with J: Pick a Card, Any Card	Vol 19.4, 101
McDonnell, Eugene	At Play with J: Greed	Vol 20.1, 117
McDonnell, Eugene	At Play with J: The Magical Matrix	Vol 20.2, 122
McDonnell, Eugene	At Play with J: Giddyap	Vol 20.3, 117
McDonnell, Eugene	At Play with J: Jacob's Ladder	Vol 20.4, 84
McDonnell, Eugene	At Play with J: The Google Test	Vol 21.1, 116
McDonnell, Eugene	At Play with J: Metlov's Triumph	Vol 21.4, 25
McDonnell, Eugene	At Play with J: Belgian Numbers	Vol 22.1, 96
McDonnell, Eugene	At Play with J: Token Counting: APL versus J	Vol 22.3, 49
McDonnell, Eugene	Obituary: Memories of Ken	Vol 22.3, 62
McIntyre, Donald B.	An Elementary Tutorial on Conjunctions	Vol 18.2, 64
McIntyre, Donald B.	A Tribute to Ken Iverson	Vol 22.3, 109
MacLeod, George	After the Lord Mayor's Show (Gui design tool)	Vol 17.1, 83
Morgan, Ellis	Sudoku with Dyalog APL	Vol 21.4, 57
Nabavi, Richard	Up, Down and Sideways: Extending the Reach of APL	Vol 22.2, 91
Neitzel, Martin	A Very Simple Structure Compiler for J	Vol 16.2, 129f
Neitzel, Martin	Sending MIDI Messages to a Sound-card with J	Vol 16.3, 136f
Neitzel, Martin	Literate Programming with J and noweb	Vol 16.4, 107f
Neitzel, Martin & Baker, J	Review: J3 - J grows up!	Vol 13.1, 48

Author(s)	Title	Vol, Pg
Ness, David	Trip: One Way from APL to J and K	Vol 19.3, 79
Nummi, Tapio	A Note on Cholesky Decomposition	Vol 14.2, 125
Parke, William	Hacker's Corner: How Big is that Doggie in the Window?	Vol 17.2, 108
Parke, William	Googling with APL+WIN	Vol 20.4, 75
Parkhouse, Graham	APL/J/K are Top of the League	Vol 22.4, 56
Pearson, Duncan	Editorial: Interesting Times	Vol 13.1, 3
Pearson, Duncan	Namespaces in Causeway	Vol 14.1, 94
Pearson, Duncan	Meeting: BAA Meeting at Network City	Vol 13.4, 13
Pearson, Duncan & Smith	Conference: APL97 Reports	Vol 14.2, 43
Pelle, Howard	Functional Programming in J (Part 1)	Vol 13.1, 20
Pelle, Howard	Sylvester's Algorithm	Vol 13.2, 12
Pelle, Howard	Functional Programming in J (Part 2)	Vol 13.2, 29
Pelle, Howard	Perfect Shuffles In J	Vol 13.2, 108
Pelle, Howard	Functional Programming in J (Part 3)	Vol 13.3, 22
Pelle, Howard	Functional Programming in J (Part 4)	Vol 13.4, 23
Pelle, Howard	The Josephus Problem Generalized in J	Vol 14.2, 33
Pelle, Howard	Volutas - a Survey of Algorithms	Vol 14.4, 53
Pelle, Howard	Towers of Hanoi, Generalized in J	Vol 15.3, 38
Pelle, Howard	A Tutorial in J Programming: The Second-Order Josephus Sequence	Vol 19.3, 46
Pelle, Howard	Solving the "Now I Know" Problem in J	Vol 20.2, 92
Pichler, Peter	The Take Care System	Vol 20.2, 63
Piper, David	Letter: Coding Style in Nlven: Numbers	Vol 14.3, 102
Pittenger, Donald	The IML Matrix Language: A Belated Introduction	Vol 14.1, 78
Pittenger, Donald	How to Win Programming Language Battles	Vol 14.2, 100
Pittenger, Donald	Using APL and J in Conjunction to Improve System Validity	Vol 14.3, 91
Pittenger, Donald	APL and J: Working Together, yet Separately	Vol 14.4, 67
Pittenger, Donald	Working with Existing Ascii Files Using J Memory Mapping	Vol 17.3, 103
Poehl, Oleg	The Manchester Umbrella Decoded	Vol 15.4, 59
Proctor, R & Brown, R	J/LAPACK Interface Makes Econometric Analysis Fly	Vol 16.1, 133
Quario, Gianluigi	Letter: Hidden Meaning inside APL	Vol 19.4, 11
Quario, Gianluigi	Function Arrays in APL	Vol 21.4, 39
Quario, Gianluigi	Letter: More Solutions	Vol 22.2, 7
Rama, H & Barghoorn, M	Turtlegraphics with APL2	Vol 13.4, 104
Ranne, Antero	The Investment Models of a Finnish Pension Company	Vol 15.4, 63
Reiter, Cliff & Brisson, G	Parallel Processing with J	Vol 13.1, 86
Reiter, et al	Word of Words from Iterated Function Systems	Vol 15.2, 104
Reiter, Cliff	Exact Horadam Numbers with a Chebyshevish Accent	Vol 16.1, 122
Reiter, et al	Fast Fourier Transforms, Diffraction Patterns and J	Vol 16.3, 111
Reiter, Cliff	CD Labels and More	Vol 17.2, 98
Reiter, Cliff	Hyperbolic Symmetry Is a Breeze	Vol 17.3, 91

Author(s)	Title	Vol, Pg
Reiter, Cliff	Web Image Galleries Made Routine	Vol 18.1, 69
Reiter, Cliff	Review of "J, the Natural Language for Analytic Computing"	Vol 18.3, 31
Reiter, Cliff & Zachary	The Image2 Addon	Vol 18.3, 80
Reiter, Cliff	Taming Barrel and Tilt Distortion	Vol 20.1, 87
Reiter, Cliff	Elliptic Curves and Factoring (I)	Vol 20.3, 123
Reiter, Cliff	Elliptic Curves and Factoring (II)	Vol 20.4, 135
Reiter, Cliff	Time(r) for the Game of Life	Vol 21.3, 88
Reiter, Cliff	Review of "Mathematical Computing in J (Peelle)"	Vol 21.4, 97
Reiter, Cliff	Integer Apollonian Circle Packings in J	Vol 22.4, 105
Reiter, Zach	Review: J5.01 Linux Beta h	Vol 19.1, 79
Renich, Richard	Source Code Management of APL Code	Vol 22.2, 138
Riebenbauer & Hoffmann	Interactive Design of Structures: A Program for Everyone	Vol 14.2, 71
Robertson, Graeme	New Foundations	Vol 20.1, 132
Robertson, Graeme	Bell's Theorem and Matrix Mechanics	Vol 20.3, 91
Rutiser, Bill	Regular Expressions and the PCRE Library	Vol 20.3, 64
Rutiser, Bill	Notes on Virtual Memory and the APL Workspace	Vol 21.3, 115
Rutiser, Bill	A Skeleton Key to Windows Performance Monitoring	Vol 21.3, 141
Rutiser, Bill	Object Oriented Programming in APL+Win	Vol 22.2, 69
Rutiser, Bill	Static Exception Handling for APL+Win	Vol 22.2, 79
Sandies, Jon	Meeting: BAA Vendor Forum	Vol 13.1, 76
Sandies, Jon	Review: Getting Started on the Internet	Vol 13.4, 51
Sandies, Jon	Review: ComLog APL Shareware	Vol 14.3, 36
Sandies, Jon	Zark News: Introduction and Crossword-1	Vol 14.3, 53
Sandies, Jon & Barman, J	Zark News: Character-Number Converter and Crossword-2	Vol 14.4, 38
Sandies, Jon	Zark News: Utility Corner and Crossword-3	Vol 15.1, 51
Sandies, Jon	Zark News: Utility Corner and Crossword-4	Vol 15.2, 36
Sandies, Jon	Meeting: BAA Vendors' Forum	Vol 15.2, 55
Sandies, Jon	Zark News: Utility Corner and Crossword-5	Vol 15.3, 22
Sandies, Jon	VideoSoft FlexGrid Pro with Dyalog APL	Vol 15.3, 93
Sandies, Jon	Zark News: Utility Corner and Crossword-6	Vol 15.4, 25
Sandies, Jon	Flexgrid with Dyalog APL - Update	Vol 15.4, 85
Sandies, Jon	Drag and Drop in Dyalog APL	Vol 15.4, 93
Scholes, John	Dynamic Functions in Dyalog APL	Vol 13.2, 88
Scholes, John	APL98 Workshop - Threads in Dyalog APL	Vol 15.2, 61
Scholes, John	Threads: an Introduction to Multithreading	Vol 15.2, 73
Scholes, John	D: a Functional Subset of Dyalog APL	Vol 17.4, 93
Scholes, John	dfns@dyalog.com	Vol 20.1, 74
Scholes, John	Hungarian Method Cost Assignment	Vol 20.1, 80
Scholes, John	A Note on Graphs	Vol 20.4, 55
Scholes, John	How to Write Computer Programs	Vol 21.3, 40
Schott, Brian	Graphical Simulation of Rearview Mirrors	Vol 20.1, 96
Schott, Brian	ACROSS and DOWN	Vol 21.1, 99

Author(s)	Title	Vol, Pg
Sexton, Sam	Letter: Kenecdots...	Vol 22.4, 7
Shaw, Ed	Conference: APL as a Tool of Thought (X)	Vol 14.3, 69
Shaw, Ewart	Hypergeometric Functions and CDFs in J	Vol 18.4, 139
Skom. & Komilovsky	Emulation of AP207 in Dyalog APL	Vol 14.1, 64
Skom. & Zalivne	Using OpenGL Graphics in Dyalog APL	Vol 14.4, 128
Small, Nicholas	Execute Inverse	Vol 20.3, 36
Smillie, Keith	Gregorian Calendar Calculations in J	Vol 13.2, 117
Smillie, Keith	The Logical Piano of W. S. Jevons	Vol 14.2, 85
Smillie, Keith	The Computer Construction of Weaving Designs	Vol 14.3, 71
Smillie, Keith	Discovering Array Languages	Vol 22.3, 13
Smillie, Keith	Kenneth Iverson, APL and J: Some Personal Recollections	Vol 22.3, 115
Smith, Adrian	Meeting: IBM GSE and APL-Club, Germany	Vol 13.1, 68
Smith, Adrian	Meeting: Eric Iverson at the BAA	Vol 13.1, 71
Smith, Adrian	Meeting: APL2000 at Orlando (October 1996)	Vol 13.3, 62
Smith, Adrian	Hacker's Corner: Clipboarding APL Arrays	Vol 13.3, 90
Smith, Adrian	HTML Basics for APLers	Vol 13.4, 94
Smith, Adrian	On First Encountering OLE	Vol 13.4, 133
Smith, Adrian	HTML Basics for APLers - Tables	Vol 14.1, 83
Smith, Adrian	Review: APL97 CD Rom	Vol 14.3, 30
Smith, Adrian	HTML Basics for APLers - Lists	Vol 14.3, 82
Smith, Adrian	HTML Basics for APLers - Forms and Servers	Vol 14.4, 91
Smith, Adrian	HTML Basics for APLers - Servers	Vol 15.1, 105
Smith, Adrian	Hacker's Corner: How Big is that Metafile?	Vol 15.1, 128
Smith, Adrian	Dyalog APLW 8.2. First Impressions	Vol 15.2, 26
Smith, Adrian	Meeting: Eric Iverson at the BAA	Vol 15.2, 49
Smith, Adrian	HTML Basics for APLers - Image Maps	Vol 15.2, 85
Smith, Adrian	Hacker's Corner: Don't Clip My Children	Vol 15.4, 75
Smith, Adrian	Guest Editorial: The Walls are Closing In ...	Vol 16.1, 3
Smith, Adrian	Hacker's Corner: Making Waves in Win32	Vol 16.2, 110
Smith, Adrian	APL2000 at Orlando	Vol 16.3, 58
Smith, Adrian	Hacker's Corner: Making Compressed GIFs in APL+Win and Dyalog APL	Vol 16.3, 119
Smith, Adrian	VML - Vector Graphics on the Internet (Finnish Forests Seminar)	Vol 16.4, 62
Smith, Adrian	Hacker's Corner: Using Watchpoints in APL+Win 3.6	Vol 16.4, 95
Smith, Adrian	Workshop: Dragging and Dropping with Lists, Trees and Graphs	Vol 17.3, 56
Smith, Adrian	Hacker's Corner: Compressing with GZIP	Vol 18.1, 84
Smith, Adrian	An Implementation of A	Vol 18.2, 14
Smith, Adrian	Getting Started with A	Vol 18.2, 56
Smith, Adrian	Formatting Dates in Dyalog APL and APL+Win	Vol 18.2, 97
Smith, Adrian	Update on A+	Vol 18.3, 8
Smith, Adrian	Report: Forests in Finland VI (Report and Pictures)	Vol 18.4, 48
Smith, Adrian	A PDF Primer for APLers	Vol 18.4, 77
Smith, Adrian	Report: APL Germany in Munich	Vol 18.4, 85
Smith, Adrian	Review: First Experiences with PocketAPL	Vol 19.2, 52

Author(s)	Title	Vol, Pg
Smith, Adrian	Provisional Unicode Reference	Vol 19.3, 122
Smith, Adrian	Report: Forests in Finland VII	Vol 19.4, 35
Smith, Adrian	Experiences with the Dyalog.NET Compiler	Vol 19.4, 65
Smith, Adrian	Hacker's Corner: From APL 2 APL2C	Vol 19.4, 96
Smith, Adrian	Report: APL Germany	Vol 20.1, 45
Smith, Adrian	Hacker's Corner: Square Me Up	Vol 20.1, 114
Smith, Adrian	Report: Dyalog 2003 Conference	Vol 20.2, 44
Smith, Adrian	Taking the RTF Challenge	Vol 20.2, 127
Smith, Adrian	Report: APL2000 at Naples Beach in November	Vol 20.3, 51
Smith, Adrian	A Layman's Tour of .Net	Vol 20.3, 82
Smith, Adrian	Report: BAA Anniversary K day	Vol 20.4, 40
Smith, Adrian	Report: APL Germany Spring Meeting	Vol 20.4, 50
Smith, Adrian	A Strategy for Compiling APL for the .Net Framework	Vol 20.4, 113
Smith, Adrian	Report: APL2000 and Dyalog at Naples Beach	Vol 21.1, 64
Smith, Adrian	Report: APL Germany in Ingolstadt	Vol 21.1, 74
Smith, Adrian	Report: Forests in Finland IX	Vol 21.2, 52
Smith, Adrian	Review: Built-In Charting In APLX Version3	Vol 21.3, 34
Smith, Adrian	Report: Dyalog APL Users Meeting, Elsinore, October 2005	Vol 22.1, 25
Smith, Adrian	Report: APL2000 at Naples Beach November 2005	Vol 22.2, 32
Smith, A & Manktelow, J	In Search of a Cool Smooth at Naples 2005	Vol 22.2, 98
Smith, Adrian	Enigma 1368 from New Scientist	Vol 22.3, 55
Smith, Adrian	Report: KX in London, February 2006	Vol 22.4, 29
Smith, Adrian	Report: Forests in Finland, March 2006	Vol 22.4, 35
Smith, Richard	An ODBC Browser Using SQL and Dyalog APL	Vol 14.3, 105
Smith, Richard	R.Net: A Reverse Polish Interpreter for .Net	Vol 20.2, 136
Smith, Richard	Rowan.net: An APL-like Interpreter for .Net	Vol 21.2, 101
Smith, Richard	Using Delegates in an Interpreted Language	Vol 22.1, 49
Spunde, Walter	The Hidden Variable Approach?	Vol 13.2, 27
Stokes, Roger	Letter: Antecedent Precipitation Index	Vol 13.3, 8
Sullivan, John	A PostScript Type 1 Font Downloader	Vol 14.2, 110
Sullivan, John	Hacker's Corner: the Windows Registry	Vol 14.3, 98
Sullivan, John	Functional Extensions to the Fibonacci Series	Vol 15.4, 123
Sullivan, John	Letter: Matrix Inversion modulo p	Vol 18.2, 106
Sutcliffe, Gordon	J Rational Numbers....	Vol 17.4, 131
Sutton, John	Letter: Random Link Generator	Vol 15.1, 130
Sykes, Alan	ARQUE - An insight into its methodology	Vol 22.1, 109
Sykes, Al & Stroud, Tom	APL and Nested Arrays - a Dream for Statistical Computation	Vol 14.2, 58
Taylor, Stephen	Persistence and Shadows	Vol 18.2, 110
Taylor, Stephen	Three Principles of Coding Clarity	Vol 18.4, 113
Taylor, Stephen	APL in Commercial Systems Development: a Survey	Vol 19.1, 88
Taylor, Stephen	Perfect Printing (RTF from APL)	Vol 19.2, 95
Taylor, Stephen	A Place for Us	Vol 19.4, 77
Taylor, Stephen	The Simple Art of Software	Vol 19.4, 81

Author(s)	Title	Vol, Pg
Taylor, Stephen	Guest Editorial: Time for a New Name?	Vol 20.4, 3
Taylor, Stephen	Report: APL Moot	Vol 20.4, 44
Taylor, Stephen	Editorial	Vol 21.1, 2
Taylor, Stephen	Editorial	Vol 21.2, 3
Taylor, Stephen	Review: The Solid Form of Language (Robert Bringhurst)	Vol 21.2, 44
Taylor, Stephen	Review: Kx Systems Users' Meeting	Vol 21.2, 50
Taylor, Stephen	Editorial: Please Mr President, don't forget about the Working Man	Vol 21.3, 2
Taylor, Stephen	Editorial: Discover, Learn, Profit	Vol 21.4, 2
Taylor, Stephen	Industrial FP: a Case Study	Vol 21.3, 52
Taylor, Stephen	Editorial: The Wind from Denmark	Vol 22.1, 2
Taylor, Stephen	Pair Programming With The Users	Vol 22.1, 128
Taylor, Stephen	Editorial: Winter in Naples	Vol 22.2, 3
Taylor, Stephen	Editorial: Kenneth E. Iverson	Vol 22.3, 3
Taylor, Stephen	Editorial: Profiting from Iverson's work	Vol 22.4, 3
Thomson, Norman	J-ottings 10: Interest Calculations	Vol 13.1, 29
Thomson, Norman	APL Operators and J	Vol 13.1, 143
Thomson, Norman	J-ottings 11	Vol 13.2, 37
Thomson, Norman	A Word (or Two) from our Spenser	Vol 13.2, 135
Thomson, Norman	J-ottings 12	Vol 13.3, 29
Thomson, Norman	J and APL Operators	Vol 13.3, 112
Thomson, Norman	J-ottings 13: Rational Fractions	Vol 14.1, 33
Thomson, Norman	Do APL2 and J Come up to Scratch?	Vol 14.1, 46
Thomson, Norman	Letter: Enumerating and Generating Combinations	Vol 14.2, 16
Thomson, Norman	J-ottings 14: Merge Alternate	Vol 14.2, 33
Thomson, Norman	J-ottings 15: Sortes	Vol 14.3, 56
Thomson, Norman	Letter: Calculating Medians for Nested Arrays	Vol 14.4, 9
Thomson, Norman	J-ottings 16: A Composition on Composition	Vol 14.4, 46
Thomson, Norman	J-ottings 17: Auto-stereograms and the Data Craftsman	Vol 15.1, 60
Thomson, Norman	J-ottings 18: Transposition	Vol 15.2, 43
Thomson, Norman	J-ottings 19: Getting to the Root of the Matter	Vol 15.3, 33
Thomson, Norman	J-ottings 20: A Case of Taken Identity	Vol 15.4, 44
Thomson, Norman	A Technical Note on Counting Moves for Generalized Towers of Hanoi	Vol 15.4, 83
Thomson, Norman	J-ottings 21: Time for Amendment of Data	Vol 16.1, 56
Thomson, Norman	J-ottings 22: Something for our very youngest readers	Vol 16.2, 41
Thomson, Norman	J-ottings 23: Just what do they sell at C. and A.?	Vol 16.3, 39
Thomson, Norman	J-ottings 24: One Foot in the Grade	Vol 16.4, 34
Thomson, Norman	J-ottings 25: The I-Spy Book of J	Vol 17.1, 35
Thomson, Norman	J-ottings 26: Here we go round...and round...and round...	Vol 17.2, 51
Thomson, Norman	J-ottings 27: Obverse to Adverse :: a Trip along the Brailleway	Vol 17.3, 40
Thomson, Norman	J-ottings 28: Just Say it in J	Vol 17.4, 31
Thomson, Norman	J-ottings 29: Power Steering Extra	Vol 18.1, 41
Thomson, Norman	J-ottings 30: Jaesthetics	Vol 18.2, 50
Thomson, Norman	J-ottings 31: Symphony in J minor, op.31	Vol 18.3, 52
Thomson, Norman	J-ottings 32: The Stylish Part of Vector	Vol 18.4, 41
Thomson, Norman	J-ottings 33: How to Tell a Fib in lots of Different Ways	Vol 19.1, 41
Thomson, Norman	J-ottings 34: Greed - Patterns for the Collapse of Western Capitalism	Vol 19.2, 38

Author(s)	Title	Vol, Pg
Thomson, Norman	J-ottings 35: How to do Things with Words	Vol 19.3, 40
Thomson, Norman	J-ottings 36: A Partial Solution to a Partial Problem	Vol 19.4, 46
Thomson, Norman	J-ottings 37: How Many Obtuse-Angled Triangles Are There?	Vol 20.1, 38
Thomson, Norman	J-ottings 38: Decline and Rise: A Matter of Rank	Vol 20.2, 37
Thomson, Norman	J-ottings 39: All Verbs are Monadic	Vol 20.3, 43
Thomson, Norman	J-ottings 40: Parallel Joins	Vol 20.4, 32
Thomson, Norman	J-ottings 41: Shortest Paths and Related Algorithms	Vol 21.1, 34
Thomson, Norman	J-ottings 42: I (continue to) Spy with very little J...	Vol 21.2, 38
Thomson, Norman	J-ottings 43: A Rippling Good Yarn	Vol 21.3, 30
Thomson, Norman	J-ottings 44: So Easy a Child of Ten ...	Vol 21.4, 101
Thomson, Norman	J-ottings 45: A bit of OUTReach (or do I mean outREACH?)	Vol 22.1, 102
Thomson, Norman	J-ottings 47: RU14 btt msgs? pls 1!1 on	Vol 22.2, 131
Thomson, Norman	J-ottings 46: Musical J-ers	Vol 22.3, 35
Thomson, Norman	J-ottings 48: Permting and Combing	Vol 22.4, 98
Toronto Globe and Mail	Obituary: Kenneth E. Iverson	Vol 22.3, 58
Tovar, Horacio	My Experience at the Dyalog Conference	Vol 22.1, 33
Towers, Andrew	An Approximation of the N-body Gravitation Problem in J	Vol 15.4, 50
Tuttle, Joey	What's Wrong with my Programming?	Vol 22.3, 119
VikingAPL 2000	Report: The VikAPL Experience - October 2000	Vol 17.3, 50
Waid, FM	APL and Web Services	Vol 19.3, 71
Weber, Adam	Hacker's Corner: APL Characters and Windows 95	Vol 13.2, 134
Weber, Adam	C Source Code Modification using APL	Vol 13.4, 89
Whitney, Arthur	Obituary: Memories of Ken	Vol 22.3, 63
Wickens, Mark	APL to Unicode Transcoding in Java	Vol 21.2, 137
Wilson, Alan	Solving Nonogram Puzzles with APL	Vol 13.1, 102
Wilson, Alan	The UK National Lottery	Vol 14.1, 51
Wilson, Alan	The Subset Sum Problem	Vol 15.4, 71
Wilson, Anne	APL and the Mountain	Vol 15.1, 119
Wilson, Anne	Over and Under: Celtic Knotwork Patterns	Vol 17.4, 101
Zippel, Michael	ZIMMERER: Roof Design with APL	Vol 20.2, 68

Index to Advertisers

Dyadic Systems Ltd	2
MicroAPL Ltd	6
Vector Back Numbers	11

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles to the Vector Working Group via the Editor:

The Editor, Vector
81 South Hill Park
London
NW3 2SS

Tel: +44 (0) 1439-788385
Email: editor@vector.org.uk

Please supply as much material as possible in machine-readable form. Our preferred format is HTML, using simple markup – we apply Vector's style sheet. Source code can be tagged with `<pre>` or `<code>` tags as usual. You can further distinguish language within these tags (which would be important if, for example, you were comparing J and APL expressions) with the class attribute. For example: `<code class="J">MEAN=:+/%#</code>` would be typical. Authors wishing to use MS Word should install a copy of the APL385 Unicode TrueType font and use Unicode APL symbols if their interpreter permits 'copy as Unicode' from the APL session. Otherwise we can convert from Dyalog APL, APL+Win, APLX, and APL2 mappings if necessary.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2006/2007 Committee

Chairman	Paul Grosvenor 01293-562700 paul@optima-systems.co.uk	Optima House, Mill Court Spindle Way, CRAWLEY West Sussex, RH10 1TT
Secretary	Anthony Camacho 0117-9730036 acam@blueyonder.co.uk	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stephen Taylor sjt@vector.org.uk	81 South Hill Park LONDON NW3 2SS
Education	Dr Alan Mayer 01792-295779 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Activities	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Rd, Blackwater Camberley, Surrey GU17 0BS
Projects	Ian Clark earthspot2000@hotmail.com	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stephen Taylor	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Anthony & Sylvia Camacho (0117-9730036), Ray Cannon (01252-874697), Stephen Taylor (077-1340 0852) Bob Hoekstra, Ian Clark	

Typeset by APL-385 with MS Word for Windows

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
Brook House, Gilling East
YORK YO62 4JU
Tel: 01439-788418
Web: www.causeway.co.uk

Dyalog Ltd
Grove House, Lutyens Close,
Chinham Court, BASINGSTOKE
Hants, RG24 8AG
Tel: 01256-888461
Email: sales@dyalog.com
Web: www.dyalog.com

Insight Systems ApS
Nordre Strandvej 119C
DK-3150 Høllerbak, Denmark
Tel: +45 70 26 13 26
Email: info@insight.dk
Web: www.insight.dk

MicroAPL Ltd
The Roller Mill, Mill Lane
Uckfield, E Sussex TN22 5AA
Tel: 01256-768050
Email: MicroAPL@microapldemon.co.uk
Web: www.microapl.co.uk/apl

Kx Systems
555 Bryant St., No. 375
Palo Alto CA 94301 USA
Tel: +1 866/kdb/fast
Email: general.enquiries@info@kx.com
In Europe: Simon Garland, simon@kx.com

Optima Systems Ltd
Optima House, Mill Court, Spindle Way, Crawley,
West Sussex, RH10 1TT
Tel: 01293 562700
Email: paul@optima-systems.co.uk

APL Italiana
via Montegrato 1, 144 Milano, Italy
Email: stf@apl.it

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

APL2000 Inc
One Research Court
Suite 140
Rockville MD 20850, USA
Tel: +1 (301) 208 7150
Email: sales@apl2000.com
Web: www.apl2000.com

HMW Computing
Hamilton House,
1 Temple Avenue,
LONDON EC4A 3HA
Tel: 0870-1010-469
Email: HMW@4xtra.com

Soliton Inc
44 Victoria Street, Suite 2100
Toronto, Ontario M5C 1W2, Canada
Tel: +1 (416) 864 9355
Email: sales@soliton.com
Web: www.soliton.com

First Derivatives plc
First Derivatives House
Kilmorey Business Park, Kilmorey Street
Newry, Co. Down, N. Ireland BT34 2DH
Tel: 028 3025 2242
Email: enquiries@firstderivatives.com

The Carlisle Group
544 Jefferson Avenue, Scranton PA, 18510
USA
Tel: +1 (570) 963-2036
Web: www.carlislegroup.com